

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Факультет _____ комп'ютерних наук
(повна назва)

Кафедра _____ програмної інженерії
(повна назва)

КВАЛІФІКАЦІЙНА РОБОТА
Пояснювальна записка

рівень вищої освіти _____ перший (бакалаврський)

Програмна система для безконтактного замовлення їжі в ресторані з
використанням QR-технологій
(тема)

Виконав:
здобувач _____ 4 _____ року навчання
групи ПЗП-22-1

_____ Денис ТОКАР
(Власне ім'я, ПРІЗВИЩЕ)

Спеціальність 121 – Інженерія програмного
забезпечення
(код і повна назва спеціальності)

Тип програми освітньо-професійна

Освітня програма Програмна інженерія
(повна назва освітньої програми)

Керівник _____ доцент кафедри ПІ Анастасія
ЧУПРИНА
(посада, Власне ім'я, ПРІЗВИЩЕ)

Допускається до захисту
В.о. зав. кафедри

_____ Ілона РЕВЕНЧУК
(підпис) (Власне ім'я, ПРІЗВИЩЕ)

2026 р.

Харківський національний університет радіоелектроніки

Факультет _____ комп'ютерних наук _____
 Кафедра _____ програмної інженерії _____
 Рівень вищої освіти _____ перший (бакалаврський) _____
 Спеціальність _____ 121 – Інженерія програмного забезпечення _____
 Тип програми _____ Освітньо-професійна _____
 Освітня програма _____ Програмна Інженерія _____
 (шифр і назва)

ЗАТВЕРДЖУЮ:

В.о. зав. кафедри _____
(підпис)

«___» _____ 2026 р.

ЗАВДАННЯ НА КВАЛІФІКАЦІЙНУ РОБОТУ

здобувачеві _____ Токару Денису Юрійовичу _____
 (прізвище, ім'я, по батькові)

1. Тема роботи _____ Програмна система для безконтактного замовлення їжі в ресторани з використанням QR-технологій _____

Затверджена наказом по університету від 05.06.2026 р. №479 Ст. _____

2. Термін подання студентом роботи до екзаменаційної комісії 10.07.2026 _____

3. Вихідні дані до роботи роботи _____ Розробити програмну систему для безконтактного замовлення їжі в закладах харчування з використанням QR. _____

4. Перелік питань, що потрібно опрацювати в роботі _____

Вступ, аналіз предметної галузі, формування вимог до програмної системи, архітектура та проектування програмного забезпечення, опис прийнятих програмних рішень, тестування розробленого програмного забезпечення, висновки, додатки. _____

КАЛЕНДАРНИЙ ПЛАН

№	Назва етапів роботи	Термін виконання етапів роботи	Примітка
1	Аналіз предметної галузі	20.04.2026	виконано
2	Створення специфікації ПЗ	04.05.2026	виконано
3	Проектування ПЗ	18.05.2026	виконано
4	Розробка ПЗ	01.06.2026	виконано
5	Тестування ПЗ	08.06.2026	виконано
6	Оформлення пояснювальної записки	22.06.2026	виконано
7	Підготовка презентації та доповіді	29.06.2026	виконано
8	Попередній захист	10.07.2026	виконано
9	Нормоконтроль, рецензування	10.07.2026	виконано
10	Здача роботи у електронний архів	10.07.2026	виконано
11	Допуск до захисту у зав. кафедри	12.07.2026	виконано

Дата видачі завдання «13» квітня 2026р.

Здобувач _____
(підпис)

Керівник роботи _____
(підпис)

доцент кафедри ПІ Анастасія ЧУПРИНА
(посада, Власне ім'я, ПРІЗВИЩЕ)

РЕФЕРАТ

Пояснювальна записка до кваліфікаційної роботи бакалавра: 108 с., 26 рис., 4 табл., 11 джерел, 2 додатки.

QR-ТЕХНОЛОГІЇ, БЕЗКОНТАКТНЕ ЗАМОВЛЕННЯ, АВТОМАТИЗАЦІЯ РЕСТОРАНУ, SAAS, ОДНОСТОРІНКОВИЙ ВЕБЗАСТОСУНОК, REACTJS, NODE.JS, MONGODB, WEBSOCKET, REST API, BYOD, FREEMIUM, KDS.

Об'єкт дослідження — процес автоматизації обслуговування клієнтів у закладах громадського харчування малого та середнього формату.

Предмет дослідження — методи та засоби розробки програмної системи для безконтактного замовлення їжі на основі QR-технологій із використанням технологічного стеку ReactJS, Node.js та MongoDB.

Мета роботи — проєктування та розробка програмної системи для безконтактного замовлення їжі в ресторані з використанням QR-технологій, що забезпечує повний цикл взаємодії гостя із закладом від сканування QR-коду до завершення оплати без обов'язкового залучення обслуговуючого персоналу.

Методи дослідження — порівняльний аналіз існуючих програмних рішень у сфері автоматизації ресторанного обслуговування; методології формування вимог до програмного забезпечення відповідно до стандарту IEEE Std 830-1998; методи об'єктно-орієнтованого проєктування; шаблони архітектурного проєктування веб-застосунків (трирівнева архітектура, multi-tenant SaaS, шарова організація серверного коду); методи проєктування документно-орієнтованих баз даних; принципи проєктування REST API та протоколу WebSocket для передачі даних у режимі реального часу.

У результаті виконання роботи спроектовано та реалізовано клієнт-серверну систему, що складається з односторінкового веб-застосунку для гостей і персоналу

та серверної частини з підтримкою REST API та WebSocket-з'єднань. Сформовано повний комплект функціональних та нефункціональних вимог, що містить 15 функціональних модулів, понад 80 функціональних вимог із унікальними ідентифікаторами та понад 70 критеріїв приймання у форматі Given/When/Then. Спроектowano схему документно-орієнтованої бази даних з 23 колекціями. Реалізовано інтеграцію з зовнішніми сервісами: платіжний шлюз LiqPay, Google OAuth 2.0, AWS S3, Resend, Google Translate API. Реалізовано модель монетизації Freemium із технічним розподілом функціоналу між безкоштовним та преміум-тарифами на рівні бази даних та проміжного програмного забезпечення.

Практична цінність роботи полягає у створенні програмного продукту, придатного для реального впровадження в закладах громадського харчування малого та середнього формату. Модульна побудова системи забезпечує її гнучкість та можливість подальшого розширення функціональності відповідно до потреб конкретного підприємства. Запропонована модель монетизації Freemium робить продукт фінансово доступним для закладів малого формату, що було визначено як ключова ринкова прогалина за результатами аналізу предметної галузі.

ABSTRACT

QR TECHNOLOGIES, CONTACTLESS ORDERING, RESTAURANT AUTOMATION, SAAS, SINGLE-PAGE WEB APPLICATION, REACTJS, NODE.JS, MONGODB, WEBSOCKET, REST API, BYOD, FREEMIUM, KDS.

The object of the study is the process of automating customer service in small and medium-sized restaurants.

The subject of the study is the methods and tools for developing a software system for contactless food ordering based on QR technology, using the ReactJS, Node.js, and MongoDB technology stack.

The objective of this work is to design and develop a software system for contactless food ordering in a restaurant using QR technologies, which ensures the full cycle of guest interaction with the establishment—from scanning a QR code to completing payment—without the mandatory involvement of service staff.

Research methods include a comparative analysis of existing software solutions in the field of restaurant service automation; methodologies for defining software requirements; object-oriented design methods; web application architectural design patterns (three-tier architecture, multi-tenant SaaS, layered server-side code organization); methods for designing document-oriented databases; principles for designing REST APIs and the WebSocket protocol for real-time data transmission.

As a result of this project, a client-server system was designed and implemented, consisting of a single-page web application for guests and staff, and a server-side component supporting REST API and WebSocket connections. A complete set of functional and non-functional requirements was developed, containing 15 functional modules, over 80 functional requirements with unique identifiers, and over 70 acceptance criteria. A document-oriented database schema with 23 collections was designed. Integration with external services has been implemented: LiqPay payment gateway, Google OAuth 2.0, AWS S3, Resend, and Google Translate API. A Freemium monetization model has been implemented with technical separation of functionality between free and premium plans at the database and middleware levels. The practical value of this work lies in the creation of a software product suitable for real-world implementation in small and medium-sized food service establishments. The system's modular design ensures its flexibility and the ability to further expand functionality according to the needs of a specific business. The proposed Freemium monetization model makes the product financially accessible to small-scale establishments, which was identified as a key market gap based on the results of an industry analysis.

ЗМІСТ

ПЕРЕЛІК УМОВНИХ СКОРОЧЕНЬ	8
ВСТУП.....	10
1 АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ.....	12
1.1 Загальна характеристика предметної галузі.....	12
1.2 Огляд існуючих програмних рішень.....	13
1.3 Порівняльний аналіз існуючих рішень	15
1.4 Постановка задачі	17
1.5 Рольова модель системи.....	17
1.6 Функціональні модулі системи.....	18
1.7 Позиціонування на ринку.....	20
1.8 Модель монетизації	21
1.9 Обґрунтування вибору технологічного стеку	22
2 ФОРМУВАННЯ ВИМОГ	24
2.1 Огляд функціональних вимог	24
2.2 Огляд нефункціональних вимог	43
2.3 Вимоги до тарифної моделі.....	45
3 АРХІТЕКТУРА ТА ПРОЄКТУВАННЯ	47
3.1 UML-діаграми	47
3.2 Опис структури бази даних.....	53
3.3 Опис алгоритмів функціонування	56
3.4 Опис компонентів програмної системи	58
3.5 UI/UX розробка дизайну інтерфейсу	61
4 ОПИС ПРИЙНЯТИХ ПРОГРАМНИХ РІШЕНЬ	69
4.1 Структура коду проєкту	69
4.2 Управління станом через контексти React	69
4.3 Найцікавіші модулі або компоненти системи.....	70
4.4 Інтерфейси системи	75
5 ТЕСТУВАННЯ.....	89
5.1 Стратегія тестування.....	89
5.2 Інструменти та середовище тестування.....	89
5.3 Тестові дані.....	90
5.4 Автоматизоване тестування серверної частини.....	91
5.5 Ручне функціональне тестування клієнтського інтерфейсу	92
5.6 Результати тестування	93
6 ВПРОВАДЖЕННЯ	95
6.1 Середовище розгортання.....	95
6.2 Конфігурація через змінні оточення	95
6.3 Збірка та розгортання	96
6.4 Моніторинг та логування	96
ВИСНОВКИ	98
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ	101
ДОДАТОК А	Ошибка! Закладка не определена.
ДОДАТОК Б	Ошибка! Закладка не определена.

ПЕРЕЛІК УМОВНИХ СКОРОЧЕНЬ

API – Application Programming Interface (програмний інтерфейс застосунку)

AWS S3 – Amazon Web Services Simple Storage Service (хмарне сховище файлів)

BYOD – Bring Your Own Device (принось власний пристрій)

CDN – Content Delivery Network (мережа доставки контенту)

CRM – Customer Relationship Management (управління відносинами з клієнтами)

CRUD – Create, Read, Update, Delete (базові операції з даними)

CSRF – Cross-Site Request Forgery (підробка міжсайтових запитів)

CSS – Cascading Style Sheets (каскадні таблиці стилів)

DOM – Document Object Model (об'єктна модель документа)

HMAC – Hash-based Message Authentication Code (код автентифікації повідомлень на основі хешу)

HoReCa – Hotel, Restaurant, Café

HTML – HyperText Markup Language (мова розмітки гіпертексту)

HTTP – Hypertext Transfer Protocol (протокол передачі гіпертексту)

HTTPS – Hypertext Transfer Protocol Secure (захищений протокол передачі гіпертексту)

IEEE – Institute of Electrical and Electronics Engineers

JSON – JavaScript Object Notation (текстовий формат обміну даними)

JWT – JSON Web Token (токен автентифікації)

KDS – Kitchen Display System (система відображення замовлень для кухні)

LTS – Long-Term Support (довгострокова підтримка)

NoSQL – Not only Structured Query Language

OAuth – Open Authorization (відкрита авторизація)

ODM – Object Document Mapper

POS – Point of Sale (точка продажу)

PWA – Progressive Web App (прогресивний вебзастосунок)

QR – Quick Response (швидкий відгук)

RBAC – Role-Based Access Control (контроль доступу на основі ролей)

REST – Representational State Transfer

REST API – Representational State Transfer Application Programming Interface

SaaS – Software as a Service (програмне забезпечення як послуга)

SPA – Single Page Application (односторінковий застосунок)

SRS – Software Requirements Specification (специфікація вимог до ПЗ)

TTL – Time to Live (час існування запису)

UI – User Interface (користувацький інтерфейс)

UML – Unified Modeling Language (уніфікована мова моделювання)

URL – Uniform Resource Locator (уніфікований локатор ресурсів)

UX – User Experience (досвід користувача)

WebP – Web Picture (формат зображень)

WSS – WebSocket Secure (захищений протокол WebSocket)

XSS – Cross-Site Scripting (міжсайтовий скриптинг)

ВСТУП

Розвиток інформаційних технологій та стрімке їх поширення суттєво змінюють підходи до організації обслуговування в закладах харчування. Сучасний споживач очікує швидкого та зручного обслуговування з мінімальним залученням персоналу, що формує попит на цифрові рішення, здатні замінити або доповнити традиційну модель роботи ресторану [1].

Існуюча практика прийому замовлень через офіціанта супроводжується низкою проблем: затримками в години пік, людськими помилками, витратами на друк і оновлення паперових меню, відсутністю інструментів для збору та аналізу даних про поведінку клієнтів. Зазначені чинники негативно впливають як на якість клієнтського досвіду, так і на економічну ефективність закладу [1, 2].

QR-технології є доступним і технологічно зрілим інструментом, що дозволяє усунути більшість із перелічених недоліків без значних інфраструктурних витрат з боку закладу [2]. Розміщення QR-коду на столику надає гостю можливість самостійно переглянути меню, сформувати та відправити замовлення безпосередньо на кухню, викликати офіціанта та здійснити оплату — все це в межах єдиного веб-інтерфейсу без встановлення додаткового програмного забезпечення.

Метою даної кваліфікаційної роботи є проєктування та розробка програмної системи для безконтактного замовлення їжі в ресторані з використанням QR-технологій.

Для досягнення поставленої мети необхідно вирішити такі завдання:

- дослідити предметну галузь та проаналізувати наявні програмні рішення у сфері автоматизації ресторанного обслуговування;
- сформувати функціональні та нефункціональні вимоги до системи;
- спроектувати архітектуру програмної системи та структуру бази даних;
- реалізувати клієнтський інтерфейс у вигляді SPA для гостей ресторану та панелі керування для персоналу;

- розробити серверну частину на основі REST API з підтримкою WebSocket для передачі замовлень у режимі реального часу;

- реалізувати розширені модулі системи: аналітику продажів, генерацію PDF-меню, систему зворотного зв'язку, CRM-елементи та фінансовий модуль з підтримкою онлайн-оплати і розділення рахунку.

Об'єктом дослідження є процес автоматизації обслуговування клієнтів у закладах громадського харчування.

Предметом дослідження є методи та засоби розробки програмної системи на основі QR-технологій з використанням стеку ReactJS, Node.js та MongoDB.

Практична цінність роботи полягає у створенні програмного продукту, придатного для реального впровадження в закладах громадського харчування. Модульна побудова системи забезпечує її гнучкість та можливість подальшого розширення функціональності відповідно до потреб конкретного підприємства.

1 АНАЛІЗ ПРЕДМЕТНОЇ ГАЛУЗІ

1.1 Загальна характеристика предметної галузі

Ресторанний бізнес належить до галузей із високою операційною інтенсивністю, де швидкість і точність обробки замовлень безпосередньо впливають на задоволеність клієнтів та фінансовий результат підприємства. Традиційна схема обслуговування передбачає послідовний ланцюжок взаємодій: гість — офіціант — кухня — офіціант — гість, кожна ланка якої є потенційним джерелом затримок і помилок [3].

Цифровізація ресторанного сервісу розвивається за кількома напрямками: автоматизація внутрішніх процесів (системи точок продажу — POS-системи, системи управління кухнею — KDS), діджиталізація взаємодії з клієнтом (мобільні застосунки, самообслуговування) та безконтактне обслуговування на основі QR-технологій. Останній напрям набув особливого поширення після 2020 року і на сьогодні є одним із найбільш доступних інструментів цифрової трансформації для малого та середнього бізнесу у сфері готельно-ресторанного бізнесу (HoReCa) [4].

QR-код (Quick Response Code) — двовимірний штрих-код, що кодує текстову інформацію, зокрема URL-адресу. Сканування QR-коду вбудованою камерою смартфона без встановлення додаткових застосунків є ключовою перевагою цієї технології з точки зору доступності для кінцевого користувача. Розміщення індивідуального QR-коду на кожному столику ресторану дозволяє системі автоматично ідентифікувати місце розташування гостя та прив'язати замовлення до конкретного столика [3].

Технологічну основу сучасних систем безконтактного замовлення складають:

- прогресивні вебзастосунки (PWA) або адаптивні односторінкові застосунки (SPA), що не потребують встановлення;
- програмні інтерфейси застосунків (REST API або GraphQL) для обміну даними між клієнтом і сервером;

- протоколи двостороннього зв'язку (WebSocket) для передачі подій у режимі реального часу;
- хмарні або локальні реляційні та документно-орієнтовані бази даних (БД) для зберігання меню, замовлень і аналітичних даних.

1.2 Огляд існуючих програмних рішень

1.2.1 Poster POS

Poster POS — хмарна система автоматизації ресторану українського походження, що охоплює функції POS-термінала, складського обліку, аналітики та управління персоналом [5]. Система надає можливість створення QR-меню для гостей, однак ця функція реалізована як статичний перегляд позицій без повноцінного циклу замовлення безпосередньо з боку клієнта. Прийом замовлень залишається прерогативою персоналу через планшет або касовий термінал.

З точки зору аналітики Poster POS пропонує розвинений дашборд із показниками виручки, популярності страв та ефективності персоналу. Проте система є комплексним корпоративним продуктом із щомісячною абонентською платою, що робить її фінансово недоступною для малих закладів. Відкритого API для глибокого налаштування клієнтської частини система не надає, що унеможливорює адаптацію інтерфейсу під фірмовий стиль конкретного закладу [5].

1.2.2 OddMenu

OddMenu — спеціалізований SaaS-сервіс для створення цифрових QR-меню ресторанів [6]. На відміну від Poster POS, продукт зосереджений виключно на взаємодії з гостем: сканування QR-коду відкриває адаптивне меню з фотографіями, описами та фільтрацією за категоріями. Інтерфейс оптимізований для мобільних пристроїв і не потребує реєстрації з боку клієнта.

Однак OddMenu є, по суті, інструментом для відображення меню, а не повноцінною системою замовлення [6]. Платформа не підтримує відправлення

замовлення на кухню в режимі реального часу, не має модуля онлайн-оплати, аналітики, системи зворотного зв'язку чи виклику офіціанта. Функціонал розділення рахунку та CRM-елементи також відсутні. Сервіс надає обмежені можливості для брендування інтерфейсу в рамках безкоштовного тарифу.

1.2.3 Choice QR

Choice QR — хмарна платформа, що надає ресторанам інструменти для управління взаємодією з гостями через QR-оплату, замовлення до столика, цифрове меню, бронювання та CRM для гостей [7]. Відмінністю від більшості конкурентів є те, що гості можуть самостійно розмістити або доповнити замовлення у будь-який момент без очікування, а заклад отримує миттєві сповіщення про нові замовлення, які можуть бути одразу передані на кухню.

Платформа також реалізує функцію оплати через QR-код із підтримкою розділення рахунку між гостями: кожен може самостійно обрати позиції для оплати та залишити чайові. При цьому гостям не потрібно реєструватися або встановлювати застосунок. Також є можливість додати функціонал резервації столиків. З точки зору CRM, система збирає клієнтську базу з усіх каналів, підтримує відправлення промоакцій, фільтрацію клієнтів і перегляд зворотного зв'язку, що, за даними самої платформи, забезпечує приріст виручки до 35% [7].

На додаток ця система надає можливість інтеграції з популярними сервісами доставки. А також платформа дає змогу інтегрувати схожі популярні POS системи, зокрема Syrve, Poster, Servio та інші, зі збереженням накопичених даних та автоматичною синхронізацією замовлень.

Проте основним недоліком Choice QR є обмежений функціонал в безкоштовному та найнижчих тарифах, зокрема відкритий API, CRM, пріоритетна підтримка, інтеграція з POS системами та власний домен, доступні лише на вищих тарифних планах [7]. Генерація PDF-меню для фізичного друку в системі відсутня.

1.2.4 Kavapp QR

Kavapp — українська система автоматизації, що охоплює продажі, склад, логістику, персонал і фінанси для закладів харчування та магазинів, синхронізована через три спеціалізовані застосунки: для менеджменту й аналізу, для продажів і фіскалізації, а також для управління логістикою та складом [8].

Серед маркетингових інструментів платформа включає QR-меню, програми лояльності та знижки. Проте QR-меню в Kavapp реалізоване як допоміжний інструмент у межах ширшої POS-системи, а не як самостійний канал взаємодії гостя з кухнею [8]. Самостійне оформлення замовлення клієнтом через QR-інтерфейс без участі персоналу не передбачено. Система орієнтована насамперед на автоматизацію роботи персоналу, а не на безконтактний клієнтський досвід. Функції виклику офіціанта через інтерфейс, розділення рахунку та генерації PDF-меню відсутні.

1.3 Порівняльний аналіз існуючих рішень

У таблиці 1.1 наведено порівняння розглянутих систем за ключовими функціональними та технічними характеристиками.

Таблиця 1.1 — Порівняльний аналіз існуючих рішень

Характеристика	Poster POS	OddMenu	Choice QR	Kavapp QR	Система, що розробляється
QR-меню для гостей	Так	Так	Так	Так	Так
Самостійне замовлення гостем	Через персонал	Ні	Так	Ні	Так
Передача замовлення на кухню в реальному часі	Через персонал	Ні	Так	Через персонал	Так
Виклик офіціанта через інтерфейс	Ні	Ні	Так	Ні	Так
Редагування замовлення «на ходу»	Ні	Ні	Так	Ні	Так

Продовження таблиці 1.1

Характеристика	Poster POS	OddMenu	Choice QR	Kavapp QR	Система, що розробляється
Онлайн-оплата	Так	Ні	Так	Ні	Так
Розділення рахунку (split bill)	Ні	Ні	Так	Ні	Ні
Аналітика та звітність	Розширена	Відсутня	Розширена	Базова	Базова
Генерація PDF-меню	Ні	Ні	Ні	Ні	Так
Система зворотного зв'язку	Частково	Ні	Так	Частково	Так
CRM / історія замовлень	Частково	Так	Частково	Так	Так
Орієнтація на малий бізнес	Частково	Так	Частково	Так	Так

Проведений аналіз дозволяє розподілити розглянуті рішення на три умовні категорії. До першої належать комплексні POS-платформи (Poster POS, Kavapp), які мають розвинений внутрішній функціонал для персоналу, але не забезпечують повноцінного самостійного замовлення гостем через QR-інтерфейс і є функціонально та фінансово надлишковими для закладів малого і середнього розміру. До другої категорії відносяться вузькоспеціалізовані сервіси цифрового меню (OddMenu), що є доступними у впровадженні, проте обмежені виключно функцією відображення інформації. Третю категорію формують повнофункціональні QR-платформи (Choice QR), які найближчі до концепції системи, що розробляється, однак орієнтовані на міжнародний ринок та потребують значних витрат на підключення.

Таким чином, проведений аналіз виявив прогалину на ринку: існуючі рішення або фінансово недоступні для малого бізнесу через надлишковий функціонал (Poster, Choice QR), або обмежуються лише переглядом меню (OddMenu). Система, що розробляється, покликана стати збалансованим інструментом, що поєднує автономність гостя (замовлення, оплата) із унікальними локальними запитами, як-от генерація PDF-меню та прямий виклик офіціанта.

1.4 Постановка задачі

Проведений аналіз предметної галузі та огляд наявних програмних рішень підтвердили відсутність на ринку доступного інструмента, що поєднував би повний цикл самостійного обслуговування гостя (замовлення, оплата, виклик персоналу) із фінансовою доступністю для закладів малого та середнього формату. Розв'язання цієї проблеми потребує проєктування та розробки програмної системи безконтактного замовлення їжі на основі QR-технологій.

Для досягнення поставленої мети в роботі мають бути розв'язані такі задачі:

1. сформулювати функціональні та нефункціональні вимоги до програмної системи з урахуванням рольової моделі користувачів;
2. спроектувати архітектуру програмної системи, зокрема схему бази даних та взаємодію клієнтської й серверної частин;
3. розробити клієнтську частину системи у вигляді прогресивного веброзширення (PWA) з підтримкою ролей гостя, офіціанта, кухаря та адміністратора;
4. розробити серверну частину системи з підтримкою REST API та обміну подіями в реальному часі за протоколом WebSocket;
5. реалізувати інтеграцію із зовнішніми сервісами: платіжним шлюзом LiqPay, Google OAuth 2.0, об'єктним сховищем AWS S3;
6. реалізувати модель монетизації Freemium із розподілом функціоналу між безкоштовним і преміум-тарифами;
7. провести тестування розробленої програмної системи та оцінити відповідність реалізованого функціоналу сформованим вимогам.

1.5 Рольова модель системи

Система обслуговує чотири типи користувачів із розмежованим доступом.

Гість є основним зовнішнім користувачем системи. Доступ до меню здійснюється через сканування QR-коду без обов'язкової реєстрації. Гість може

переглядати меню з фільтрацією за категоріями, формувати кошик, відстежувати статус приготування замовлення в режимі реального часу, доповнювати замовлення після його відправлення, викликати офіціанта через інтерфейс та здійснювати онлайн-оплату з можливістю розділення рахунку. Авторизація через Google-акаунт надає доступ до збереженої історії замовлень та системи відгуків.

Кухар працює з системою через інтерфейс KDS (Kitchen Display System) — спеціалізований дисплей на кухні, що отримує нові замовлення миттєво через WebSocket-з'єднання. Кухар керує станами приготування кожної позиції у замовленні.

Офіціант має доступ до мобільної панелі персоналу з інтерактивною картою залу, що відображає поточний стан кожного столика: вільний, зайнятий або очікує на рахунок. Офіціант може редагувати активні замовлення за зверненням гостя та отримує сповіщення про виклик.

Адміністратор має повний доступ до всіх модулів системи: управління персоналом та рівнями доступу, конструктор меню з управлінням цінами та категоріями, модуль аналітики, генератор PDF-меню, налаштування QR-кодів для столиків та налаштування параметрів ресторану.

1.6 Функціональні модулі системи

Модуль QR-менеджменту забезпечує генерацію унікальних QR-кодів для кожного столика з прив'язкою до його фізичного розташування в залі. Для підвищення гнучкості реалізовано динамічний QR з проміжним редиректом, що дозволяє змінювати цільову URL-адресу без перевипуску коду. Після сканування система автоматично ідентифікує столик і прив'язує до нього всі подальші дії гостя в межах сесії, а механізм Smart Session Recovery відновлює перервану сесію при повторному скануванні або перезавантаженні сторінки.

Модуль замовлень у реальному часі реалізує передачу підтверджених замовлень від клієнтського інтерфейсу до кухонного дисплея через WebSocket-

з'єднання без перезавантаження сторінки. Система підтримує одне активне замовлення на столик: повторне сканування QR-коду надає гостю доступ до перегляду меню без можливості розміщення нового замовлення. Статус замовлення оновлюється одночасно на кухонному дисплеї та в інтерфейсі гостя, а функція груп подачі дозволяє гостю об'єднувати страви в блоки для їх одночасного приготування та подачі.

Модуль кухонного дисплея (KDS) функціонує у двох режимах відображення: Order View, що показує замовлення як єдину картку, та Table View, що групує всі активні позиції по столиках. Персонал кухні послідовно переводить страви між статусами, а зміна статусу миттєво відображається в інтерфейсі гостя через WebSocket.

Модуль виклику персоналу підтримує два сценарії взаємодії гостя з офіціантом. Загальний виклик доступний у будь-який момент сесії для вирішення довільних питань. Виклик для готівкової оплати стає доступним лише після того, як усі страви замовлення отримали статус поданих, що унеможливлює передчасне закриття рахунку.

Фінансовий модуль підтримує як сповіщення для персоналу про оплату готівкою, так і повноцінну онлайн-оплату через інтегрований платіжний шлюз LiqPay. Онлайн-оплата, аналогічно до готівкової, стає доступною лише після подачі всіх страв. Операції скасування замовлення (Cancelled) є інструментами офіціанта або адміністратора і недоступні гостю.

Модуль меню забезпечує повноцінне управління структурою пропозиції закладу через CRUD-операції над стравами, категоріями, інгредієнтами та додатками (add-ons). Гість може переглядати детальний склад кожної позиції, а наявність модифікаторів і add-ons дозволяє персоналізувати замовлення без залучення персоналу.

Модуль аналітики та звітності надає адміністратору дашборд із показниками продажів, середнього чека та популярності окремих позицій меню у вигляді графіків і зведених таблиць за обраний період.

Генератор PDF-меню автоматично формує стилізований друкований документ на основі актуальних даних із бази даних, що призначений для фізичного друку та розміщення в залі.

Модуль авторизації реалізує повноцінну систему автентифікації, що підтримує два методи входу: за допомогою електронної пошти та пароля, а також через Google OAuth 2.0. Модуль також охоплює управління персоналом та процедуру створення нового ресторану.

Система відгуків дозволяє авторизованим гостям залишати оцінки окремим стравам та закладу в цілому після завершення замовлення.

1.7 Позиціонування на ринку

Система позиціонується як спеціалізоване SaaS-рішення (Software as a Service) для закладів харчування малого та середнього формату. Модель передбачає надання повного функціоналу платформи як хмарної послуги: заклад не купує програмне забезпечення і не розгортає власну інфраструктуру — натомість отримує доступ до готової системи без залучення технічних спеціалістів для впровадження.

Ключова відмінність від конкурентів полягає у цільовому фокусі: на відміну від комплексних POS-платформ, система не претендує на повну автоматизацію всіх бізнес-процесів закладу, а вирішує конкретну задачу — забезпечити повний цикл взаємодії гостя із закладом від сканування QR-коду до завершення оплати. Порівняно з вузькоспеціалізованими сервісами цифрового меню система надає значно ширший операційний функціонал за зіставну вартість.

Унікальною конкурентною перевагою системи є розширений real-time функціонал, що забезпечує миттєвий двосторонній зв'язок між гостем та персоналом. На відміну від статичних аналогів, система дозволяє клієнту викликати

офіціанта одним натисканням у веб-інтерфейсі та здійснювати редагування або доповнення замовлення «на ходу» без залучення персоналу. Додатковою перевагою є гібридна модель обслуговування, що поєднує самостійне замовлення гостем і класичну роботу офіціанта, дозволяючи закладу адаптувати систему під власні потреби без повної зміни бізнес-процесів.

1.8 Модель монетизації

Система базується на моделі Freemium із поділом функціоналу на два тарифні плани, що дозволяє закладам розпочати цифровізацію без початкових витрат і переходити на платний тариф у міру зростання потреб.

Базовий тариф (Free) надається без обмежень у часі та включає мінімально необхідний для роботи закладу функціонал:

- повний цикл прийому та обслуговування замовлень: QR-меню, самостійне замовлення гостем, кухонний дисплей (KDS), панель офіціанта з картою залу;
- виклик офіціанта через інтерфейс, включаючи виклик для готівкової оплати (cash_payment);
- оплата виключно готівкою через підтвердження офіціантом;
- управління меню з обмеженням: до 5 категорій, до 50 страв, до 3 зображень на страву;
- до 3 акаунтів персоналу сумарно (включно з адміністратором);
- базові налаштування ресторану: назва, адреса, тип кухні, контактні дані, мови інтерфейсу;
- система відгуків вимкнена: гості не мають доступу до форми відгуку.

Розширений тариф (Premium) орієнтований на заклади, які потребують повного функціоналу платформи та інструментів для збільшення прибутку:

- зняття обмежень на меню (необмежена кількість категорій, страв та зображень);

- інтеграція платіжного шлюзу LiqPay для безготівкової оплати онлайн через карту або Apple/Google Pay;
- багаторольовий персонал: підтримка ролей waiter, cook та комбінованої ролі waiter_cook, необмежена кількість акаунтів;
- повноцінний аналітичний модуль: виручка, середній чек, конверсія оплат, час приготування, топ страв, статистика відгуків за весь період;
- модуль управління відгуками для адміністратора (перегляд, фільтрація, видалення);
- активація системи залишення відгуків з боку гостей;
- автоматичний генератор PDF-меню для фізичного друку;
- інтеграція з Google Translate для автоматичного перекладу всіх двомовних полів вводу (назв та описів страв, категорій), що знімає необхідність ручного перекладу при веденні меню.

Розподіл функціоналу реалізовано на рівні бази даних (поле plan у документі ресторану) та контролюється проміжним програмним забезпеченням (middleware) на серверній частині, а також умовним рендерингом елементів інтерфейсу на клієнтській частині.

1.9 Обґрунтування вибору технологічного стеку

Вибір технологій для реалізації системи обумовлений вимогами до продуктивності, масштабованості та швидкості розробки.

ReactJS обрано для реалізації клієнтської частини з огляду на його компонентну архітектуру, що дозволяє незалежно розробляти та повторно використовувати елементи інтерфейсу як для мобільної версії гостя, так і для адміністративного дашборду. Віртуальний DOM забезпечує високу швидкість оновлення інтерфейсу при частих змінах стану — що важливо для відображення статусів замовлень у режимі реального часу. Декларативний підхід до побудови інтерфейсу та однонаправлений потік даних, покладені в основу React, спрощують

міркування про стан застосунку та зменшують кількість помилок при масштабуванні кодової бази [9].

Node.js обрано для серверної частини завдяки його асинхронній неблокуючій архітектурі, що є оптимальною для обробки великої кількості одночасних з'єднань — зокрема WebSocket-сесій від кількох столиків одночасно. Використання JavaScript як єдиної мови для клієнтської та серверної частин спрощує розробку та підтримку кодової бази [9].

MongoDB обрано як базу даних завдяки документно-орієнтованій моделі зберігання даних, що природно відповідає структурі меню ресторану: вкладені категорії, варіації страв і модифікатори зручно представляються у форматі JSON-документів без необхідності складних реляційних зв'язків. Гнучка схема дозволяє оперативно змінювати структуру меню без міграцій бази даних [10].

Google API використовується для реалізації автентифікації гостей за схемою OAuth 2.0, що забезпечує максимально швидкий вхід без необхідності повільної реєстрації акаунта та підвищує довіру користувачів завдяки знайомому та швидкому механізму авторизації.

2 ФОРМУВАННЯ ВИМОГ

2.1 Огляд функціональних вимог

Функціональні вимоги системи розподілені на п'ятнадцять логічних модулів, кожен з яких відповідає окремому бізнес-сценарію. Кожна вимога має унікальний ідентифікатор формату SYS-<МОДУЛЬ>-<номер>, а кожен критерій приймання — ідентифікатор формату AC-<МОДУЛЬ>-<номер> та сформульований за схемою Given/When/Then. Нижче наведено повний опис кожного модуля системи, описаного у специфікації вимог до програмного забезпечення (SRS).

2.1.1 Модуль QR-менеджменту та ідентифікація столика

Модуль забезпечує ідентифікацію столика через унікальний QR-код та керування життєвим циклом гостьової сесії. Ключовою особливістю є механізм Smart Session Recovery, що відновлює попередню сесію при повторному скануванні замість створення нової, а також гарантія одного активного замовлення на столик. Пріоритет: критичний.

Функціональні вимоги модуля:

- SYS-QR-01: short-code формату /qr/{shortCode}; 4–8 символів [A-Z0-9]; криптографічно безпечна генерація (crypto.randomBytes).
- SYS-QR-02: сервер при GET /qr/{shortCode} перевіряє cookies на активний session token для поточного tableId.
- SYS-QR-03: прив'язка short-code → tableId зберігається в БД та може змінюватись адміном.
- SYS-QR-04: адмін може тимчасово деактивувати столик без зміни фізичного QR.
- SYS-QR-05: session token має термін дії 24 години (Max-Age: 86400); після закінчення генерується нова сесія.
- SYS-QR-06: session token зберігається в HTTP cookie з атрибутами HttpOnly, Secure, SameSite=Strict.

- SYS-QR-07: адмін завантажує QR-зображення у форматі PNG або PDF для одноразового друку.
 - SYS-QR-08: карта залу в панелі адміна відображає поточний стан кожного столика.
 - SYS-QR-09: якщо session token активний — сервер повертає існуючу сесію (кошик, статус замовлення) замість створення нової (Smart Session Recovery).
 - SYS-QR-10: нова сесія створюється лише якщо: немає токена в cookies / token прострочений / token анульований / token належить іншому tableId.
 - SYS-QR-11: при переведенні столика в статус «Вільний» система інвалідує всі активні session token цього tableId автоматично.
 - SYS-QR-12: при GET /qr/{shortCode} для неіснуючого shortCode сервер повертає HTTP 404 з інформаційною сторінкою.
 - SYS-QR-13: при скануванні QR столика з активним замовленням — відповідь містить прапорець tableHasActiveOrder: true; новий Order не створюється.
- Критерії приймання модуля:
- AC-QR-01: given shortCode існує та активний, when гість сканує QR вперше, then HTTP 302 до /menu з новим sessionToken за ≤ 500 мс.
 - AC-QR-02: given гість має активний token для tableId T1, when повторно сканує QR, then повертається та сама сесія; новий Order не створюється.
 - AC-QR-03: given shortCode не існує, when GET /qr/{unknownCode}, then HTTP 404 «QR-код не знайдено».
 - AC-QR-04: given столик деактивований, when гість сканує QR, then «Столик тимчасово недоступний»; Session та Order не створюються.
 - AC-QR-05: given столик $\rightarrow$ «Вільний», then усі session token цього tableId анулюються; наступне сканування гарантовано створює нову сесію.
 - AC-QR-06: given за столиком вже є активне замовлення, when другий гість сканує QR, then відповідь містить tableHasActiveOrder: true; гість бачить меню без кнопки «Підтвердити замовлення».

2.1.2 Модуль меню: інгредієнти, модифікатори та добавки

Модуль реалізує відображення меню з підтримкою гнучкої кастомізації страв через знімні інгредієнти, добавки (add-ons) та обов'язкові варіанти вибору (ComponentGroups). Передбачено клієнтський пошук на основі кешованих даних для миттєвої реакції інтерфейсу. Пріоритет: високий.

Функціональні вимоги модуля:

- SYS-MENU-01: кожна страва повинна мати список інгредієнтів, що керується адміном.
- SYS-MENU-02: адмін позначає кожен інгредієнт як «можна прибрати» або «обов'язковий».
- SYS-MENU-03: гість може прибрати лише інгредієнти, позначені адміном як дозволені для виключення.
- SYS-MENU-04: кожна страва може мати перелік add-ons; кожен add-on має назву та ціну (0 або більше).
- SYS-MENU-05: гість може обрати один або кілька add-ons до страви.
- SYS-MENU-06: кожна страва може мати перелік ComponentGroups, в якій клієнт обов'язково має обрати одну.
- SYS-MENU-07: гість може залишити довільний текстовий коментар до кожної окремої позиції (максимум 300 символів), валідація на клієнті та сервері.
- SYS-MENU-08: виключені інгредієнти, add-ons, ComponentGroups та коментар передаються на KDS разом із позицією.
- SYS-MENU-09: позиції зі стоп-листа відображаються як недоступні при завантаженні та перевіряються перед створенням замовлення.
- SYS-MENU-10: при недоступності фото відображається placeholder без порушення відображення решти даних страви.
- SYS-MENU-11: пошук за назвою страви доступний глобально по всьому меню та локально в межах категорії; фільтрація case-insensitive з частковим

співпадінням; реалізується на клієнті без запиту до сервера на основі кешованих даних меню.

- SYS-MENU-12: поле пошуку присутнє на головній сторінці меню (глобальний) та на сторінці кожної категорії (локальний).

- SYS-MENU-13: результати пошуку оновлюються при кожному введеному символі.

Критерії приймання модуля:

- AC-MENU-01: given страва має 5 інгредієнтів (3 знімних, 2 обов'язкових), then знімні мають чекбокс, обов'язкові — ні.

- AC-MENU-02: given страва має ComponentGroup «Гарнір», then кнопка «Додати до кошика» неактивна до вибору варіанту.

- AC-MENU-03: given гість виключив «цибулю» та підтвердив замовлення, then на KDS відображається «Без цибулі».

- AC-MENU-04: given add-on 20 грн, when гість обирає, then ціна позиції збільшується на 20 грн в реальному часі.

- AC-MENU-05: given гість вводить 301+ символ у коментар, then символи понад 300 не вводяться; лічильник показує «300/300».

- AC-MENU-06: given кухар додав страву до стоп-листа, then при спробі замовлення позиція недоступна.

- AC-MENU-07: given меню завантажено, when гість вводить «піца» у глобальний пошук, then відображаються всі страви з «піца» у назві (case-insensitive) без запиту до сервера.

- AC-MENU-08: given гість перебуває в категорії «Піца» та вводить «маргарит» у локальний пошук, then відображаються лише страви цієї категорії, що містять «маргарит» у назві.

- AC-MENU-09: given пошуковий запит не знайшов жодної страви, then відображається «Страв не знайдено».

2.1.3 Модуль формування замовлення: кошик та групи подачі

Гість формує кошик зі стравами та може об'єднати їх у групи подачі — набори страв для одночасного приготування та подачі. На кухонному дисплеї кожна група відображається як монолітний блок зі спільним статусом, що дозволяє синхронізувати подачу складних замовлень.

Функціональні вимоги модуля:

- SYS-CART-01: гість може об'єднувати страви в іменовані групи подачі в межах одного замовлення.
- SYS-CART-02: страви без явного групування потрапляють до «Основної групи» за замовчуванням.
- SYS-CART-03: кожна новостворена група автоматично має назву «Група подачі ...» з відповідним номером.
- SYS-CART-04: на KDS страви — монолітні блоки за групами.
- SYS-CART-05: статус групи є спільним для всіх страв у ній; зміна статусу групи змінює статус кожної страви одночасно.
- SYS-CART-06: гість отримує WebSocket-сповіщення коли кожна з його груп змінює статус.
- SYS-CART-07: гість може змінювати кількість / видаляти позиції до підтвердження.
- SYS-CART-08: при підтвердженні сервер виконує фінальну валідацію на стоп-лист; за наявності недоступних позицій — HTTP 422 зі списком.
- SYS-CART-09: кнопка «Підтвердити замовлення» неактивна для порожнього кошика.
- SYS-CART-10: при помилці сервера під час підтвердження кошик зберігається на клієнті.

Критерії приймання модуля:

- AC-CART-01: given гість додав 3 страви, when натискає «Підтвердити замовлення», then замовлення з'являється на KDS за ≤ 300 мс.

– AC-CART-02: given гість створив групи «Стартери» та «Основне», when замовлення підтверджено, then на KDS відображаються 2 окремі блоки з відповідними назвами.

– AC-CART-03: given одна страва потрапила до стоп-листа, when гість натискає «Підтвердити», then замовлення не відправляється; відображається список недоступних страв з кнопкою «Видалити недоступні».

– AC-CART-04: given кошик порожній, when відображається сторінка кошика, then кнопка «Підтвердити замовлення» має атрибут disabled.

– AC-CART-05: given сервер повернув HTTP 500 при підтвердженні, when гість бачить помилку, then кошик незмінний; повторна спроба можлива.

2.1.4 Модуль управління замовленням столику

За кожним столиком може існувати щонайбільше одне активне замовлення (зі статусом open) одночасно. При спробі другого гостя розмістити замовлення за тим самим столиком система повертає HTTP 409 (TABLE_OCCUPIED). Другий гість, що сканує QR столика, отримує меню в режимі «тільки перегляд» — він може переглядати страви, але не може додавати позиції до кошика або підтверджувати замовлення. Пріоритет: критичний.

Функціональні вимоги модуля:

– SYS-ORDER-01: якщо декілька людей сканує QR, то лише перший, хто зробить замовлення, буде мати доступ до нього; всі інші зможуть лише переглядати меню.

– SYS-ORDER-02: сервер перевіряє наявність активного замовлення за tableId перед створенням нового; якщо є — HTTP 409 TABLE_OCCUPIED та tableHasActiveOrder: true.

– SYS-ORDER-03: відповідь QR-ендпоінту для столика з активним замовленням містить tableHasActiveOrder: true.

- SYS-ORDER-04: гість, який отримав `tableHasActiveOrder: true`, бачить меню без можливості розмістити замовлення (режим «тільки перегляд»).
- SYS-ORDER-05: гість бачить лише власне замовлення; доступ до чужого → HTTP 403.
- SYS-ORDER-06: столик → `free`, коли замовлення переходить у фінальний статус (`completed_cash` / `completed_epay` / `cancelled`) — автоматично.
- SYS-ORDER-07: офіціант може вручну закрити столик незалежно від стану замовлення; залишкове активне замовлення → `cancelled` з причиною «Столик закрито вручну».
- SYS-ORDER-08: при вході авторизованого гостя — система перевіряє наявність активного замовлення та відновлює стан замовлення та `restaurantId` автоматично.

Критерії приймання модуля:

- AC-ORDER-01: `given` за столиком є активне замовлення (`open`), `when` другий гість сканує QR, `then` він бачить меню з неактивним кошиком; кнопка «Підтвердити замовлення» неактивна.
- AC-ORDER-02: `given` замовлення → `completed_epay`, `then` столик → «Вільний» за ≤ 300 мс.
- AC-ORDER-03: `given` двоє ініціюють `POST /orders` одночасно, `then` другий → HTTP 409; лише одне замовлення створено.
- AC-ORDER-04: `given` авторизований гість входить до акаунта і має активне замовлення, `then` його замовлення `restaurantId` відновлюється автоматично; гість перенаправляється до меню.
- AC-ORDER-05: `given` неавторизований гість, який має активне замовлення, втрачає до нього доступ (очищення `cookie` або зміна пристрою), `then` після звернення до персоналу офіціант може ввімкнути режим відновлення і при повторному скануванні QR сесія з даними про замовлення буде відновлена.

2.1.5 Модуль передачі замовлення на кухню (KDS)

Модуль забезпечує миттєву передачу підтверджених замовлень на кухонний дисплей через WebSocket. Статуси страв змінюються однонаправлено за моделлю waiting → cooking → ready → served з короткочасною можливістю відкату. Пріоритет: критичний.

Функціональні вимоги модуля:

- SYS-KDS-01: нові замовлення на KDS без ручного оновлення (WebSocket ORDER_NEW), затримка ≤ 300 мс.
- SYS-KDS-02: картка замовлення містить: номер столика, orderId, групи подачі зі стравами, виключені інгредієнти, add-ons, коментарі та час надходження.
- SYS-KDS-03: групи подачі відображаються як монолітні блоки з єдиною кнопкою зміни статусу.
- SYS-KDS-04: статуси групи: очікує → готується → готово → подано (однаправлений перехід; зниження заборонено).
- SYS-KDS-05: кухар може додавати/знімати позиції зі стоп-листа.
- SYS-KDS-06: зміна стоп-листа відображається в меню всіх клієнтів та блокує замовлення з ними.
- SYS-KDS-07: KDS підтримує два режими: Order View (кожне замовлення — окрема картка) та Table View (всі замовлення столика — єдиний блок).
- SYS-KDS-08: при розриві WebSocket KDS відображає індикатор та автоматично відновлює з'єднання.
- SYS-KDS-09: після відновлення WebSocket сервер надсилає event replay пропущених подій від останнього підтвердженого event_id.
- SYS-KDS-10: спроба зниження статусу групи може бути виконана лише в першу хвилину після минулої зміни; в інших випадках відхиляється сервером (HTTP 400).

Критерії приймання модуля:

- AC-KDS-01: given замовлення з 2 групами, then 2 блоки на KDS, які мають сталий порядок за ≤ 300 мс.
- AC-KDS-02: given WebSocket KDS розірваний і відновлений, then актуальний стан замовлень без ручного оновлення за ≤ 5 с.
- AC-KDS-03: given кухар позначив групу «ready», then GROUP_STATUS_UPDATED $\rightarrow$ пристрій гостя за ≤ 300 мс.
- AC-KDS-04: given кухар помилково змінює статус групи до «ready», when кухар намагається повернути «cooking», then успіх, статус повертається, клієнт сповіщений про цю помилку.
- AC-KDS-05: given група «ready», when кухар намагається $\rightarrow$ «cooking», then HTTP 400; статус не змінюється.

2.1.6 Модуль виклику офіціанта

Система підтримує два типи виклику офіціанта: загальний виклик (call) — гість може викликати офіціанта у будь-який момент після підтвердження замовлення, та виклик для готівкової оплати (cash_payment). Офіціант отримує WebSocket-сповіщення за ≤ 300 мс. Непідтверджені виклики старші 3 хвилин візуально виділяються. Повторний виклик, поки є вже активний, ігнорується. Пріоритет: високий.

Функціональні вимоги модуля:

- SYS-CALL-01: кнопка загального виклику доступна в будь-який момент після підтвердження замовлення.
- SYS-CALL-02: кнопка виклику для готівкової оплати (cash_payment) є активною протягом усього замовлення та при натисканні закриває зміну замовлення.
- SYS-CALL-03: WebSocket-сповіщення на панель офіціанта за ≤ 300 мс для обох типів.

- SYS-CALL-04: офіціант підтверджує виклик одним натисканням.
- SYS-CALL-05: сповіщення містить: номер столика, orderId, тип виклику, час виклику.
- SYS-CALL-06: виклики сортуються за їх часом, де перший найдовший.
- SYS-CALL-07: повторний виклик від того самого гостя протягом 60 секунд ігнорується; гість отримує повідомлення.
- SYS-CALL-08: виклик, що надійшов під час відключення WebSocket офіціанта, доставляється після відновлення з'єднання (event replay).

Критерії приймання модуля:

- AC-CALL-01: given гість натискає «Викликати офіціанта», then на панелі офіціанта з'являється сповіщення за ≤ 300 мс.
- AC-CALL-02: given гість натиснув виклик, when повторно протягом 60 с, then новий WaiterCall не створюється.
- AC-CALL-03: given гість натискає «Оплатити готівкою», then офіціант отримує WAITER_CALL_CASH за ≤ 300 мс.
- AC-CALL-04: given офіціант підтверджує cash_payment виклик, then замовлення $\rightarrow$ completed_cash; столик $\rightarrow$ free; WebSocket PAYMENT_COMPLETED за ≤ 300 мс.

2.1.7 Модуль відстеження статусу замовлення

Гість бачить стан кожної страви в реальному часі через WebSocket. Статус замовлення (open / cancelled / completed_cash / completed_eraу) не є похідним від статусів страв і відображає фінансово-операційний стан. Редагування заблоковане для страв зі статусом cooking+; після переходу замовлення у фінальний статус (cancelled / completed_*) — будь-які зміни неможливі. Пріоритет: високий.

Функціональні вимоги модуля:

- SYS-STAT-01: інтерфейс відображає статус по кожній групі подачі: Прийнято / Готується / Готово.

- SYS-STAT-02: оновлення статусу через WebSocket без перезавантаження сторінки.
- SYS-STAT-03: гість може доповнити замовлення новими позиціями (лише коли `Order.status = open`).
- SYS-STAT-04: гість не може видаляти або зменшувати кількість позицій у групі зі статусом «Готується» або «Готово».
- SYS-STAT-05: редагування (видалення/зменшення кількості) дозволене лише для позицій у групах зі статусом «Очікує».
- SYS-STAT-06: після `cancelled` або `completed_*` — будь-які зміни → HTTP 409.
- SYS-STAT-07: при розриві WebSocket клієнт відображає індикатор втрати з'єднання і відновлює його автоматично (≤ 5 с).

Критерії приймання модуля:

- AC-STAT-01: given кухар змінив статус групи, when подія надходить на пристрій гостя, then статус оновлюється без перезавантаження за ≤ 300 мс.
- AC-STAT-02: given група має статус «Готується», when гість намагається видалити позицію, then відображається «Страву вже готують — зверніться до офіціанта»; позиція залишається.
- AC-STAT-03: given `Order.status = completed_epay`, when спроба зміни, then HTTP 409.

2.1.8 Модуль оплати

Система використовує модель пост-пей. Платіжний шлюз: LiqPay. Гість оплачує замовлення через LiqPay або готівкою. Оплата може бути здійснена і до закінчення замовлення, але в такому разі подальше редагування замовлення заблоковане. Пріоритет: критичний.

Функціональні вимоги модуля:

- SYS-PAY-01: кнопка онлайн-оплати (LiqPay) може бути натиснута при стані замовлення open незалежно від стану страв у ньому.
- SYS-PAY-02: карткові дані не зберігаються на серверах системи — обробляються виключно LiqPay.
- SYS-PAY-03: після закриття замовлення (completed_cash / completed_epay / cancelled) → столик «Вільний».
- SYS-PAY-04: всі транзакції (CASH, Cancelled, walk-out) зберігаються в БД та незмінному audit log.
- SYS-PAY-05: готівкова оплата підтверджується офіціантом через WaiterCall типу cash_payment або вручну офіціантом; після підтвердження Order.status → completed_cash.
- SYS-PAY-06: після completed_* або cancelled — будь-які зміни замовлення → HTTP 409.
- SYS-PAY-07: при LiqPay timeout Order.status без змін; інформативне повідомлення користувачу.

Критерії приймання модуля:

- AC-PAY-01: given LiqPay підтверджує, when webhook надходить, then Order.status → completed_epay і підтвердження для гостя за ≤ 3 с.
- AC-PAY-02: given LiqPay не відповідає > 30 с, then повідомлення про помилку; Order.status = open.

2.1.9 Модуль аналітики та звітності

Адміністратор має доступ до аналітичного дашборду для перегляду бізнес-метрик. Дані формуються з транзакцій MongoDB у реальному часі. Пріоритет: середній (розширений тариф).

Функціональні вимоги модуля:

- SYS-ANLT-01: загальна виручка за обраний період.
- SYS-ANLT-02: середній чек замовлення за обраний період.

- SYS-ANLT-03: топ-10 найпопулярніших страв за кількістю замовлень.
- SYS-ANLT-04: вибір довільного діапазону дат (максимум 365 днів за один запит).
- SYS-ANLT-05: дані формуються з транзакцій MongoDB.
- SYS-ANLT-06: кількість та відсоток cancelled випадків за обраний період.
- SYS-ANLT-07: середній час приготування замовлення (від cooking до ready) по стравах та категоріях.
- SYS-ANLT-08: для діапазонів > 365 днів — пропозиція вивантаження CSV.

Критерії приймання модуля:

- AC-ANLT-01: given адмін обирає діапазон до 90 днів, when запит виконується, then дашборд завантажується за ≤ 3 с.
- AC-ANLT-02: given транзакція з типом «cancelled», when вона включена в аналітику, then вона входить до статистики cancelled, але не до загальної виручки.

2.1.10 Модуль генератора PDF-меню

Адміністратор формує PDF-документ із поточним меню для фізичного друку.
Пріоритет: середній (розширений тариф).

Функціональні вимоги модуля:

- SYS-PDF-01: PDF генератор доступний лише на платному тарифі.
- SYS-PDF-02: PDF формується на основі поточних даних меню з MongoDB.
- SYS-PDF-03: PDF містить: назву закладу, категорії, назви страв, описи, інгредієнти та ціни.
- SYS-PDF-04: Формат A4, придатний для якісного друку.
- SYS-PDF-05: При помилці генерації адмін отримує повідомлення «Не вдалось згенерувати PDF. Спробуйте пізніше».

Критерії приймання модуля:

- AC-PDF-01: given меню містить 100 позицій, when адмін ініціює генерацію, Then PDF файл формується за ≤ 5 с і доступний для завантаження.

– AC-PDF-02: given PDF-рушій недоступний, when адмін ініціює генерацію, then відображається помилка; інші функції системи залишаються доступними.

2.1.11 Модуль системи відгуків

Авторизовані гості залишають відгуки: про ресторан (один на замовлення) та по одному на кожну замовлену страву. Доступно лише після `Order.status = completed_cash` або `completed_pay`. Пріоритет: середній (розширений тариф).

Функціональні вимоги модуля:

– SYS-REV-01: відгуки доступні лише авторизованим гостям для замовлень зі статусом `completed_cash` або `completed_pay`.

– SYS-REV-02: `RestaurantReview` — один на замовлення від одного акаунта.

– SYS-REV-03: `DishReview` — один на кожну `OrderItem` від одного акаунта.

– SYS-REV-04: обидва типи: оцінка 1–5 та необов'язковий коментар (≤ 500 символів).

– SYS-REV-05: адмін може переглядати та видаляти відгуки з фільтрацією за датою, стравою, рейтингом.

Критерії приймання модуля:

– AC-REV-01: given авторизований гість з `completed_pay` замовленням, when залишає `RestaurantReview` з оцінкою 4, then відгук у панелі адміна.

– AC-REV-02: given вже є `RestaurantReview`, when повторна спроба, then HTTP 409.

– AC-REV-03: given анонімний гість, when відкриває форму відгуку, then HTTP 401.

– AC-REV-04: given замовлення не `completed_*`, then HTTP 403 «Відгуки доступні лише для завершених замовлень».

2.1.12 Модуль авторизації та облікових записів

Модуль підтримує два паралельні методи входу: email+пароль із bcrypt-хешуванням та Google OAuth 2.0. Для гостей автентифікація опціональна —

система підтримує анонімне користування з прив'язкою сесії до QR-сканування.
Пріоритет: критичний.

Функціональні вимоги модуля:

- SYS-AUTH-01: система підтримує реєстрацію та вхід через email+пароль для всіх типів акаунтів.
- SYS-AUTH-02: система підтримує вхід через Google OAuth 2.0 для гостей та персоналу.
- SYS-AUTH-03: для персоналу (кухар, офіціант, адмін) наявність email+пароля є обов'язковою умовою.
- SYS-AUTH-04: гість може замовляти анонімно без будь-якої авторизації.
- SYS-AUTH-05: при вході нового користувача через Google — система автоматично створює акаунт; ім'я з Google-профілю.
- SYS-AUTH-06: при повторному вході через Google — система знаходить існуючий акаунт за Google ID.
- SYS-AUTH-07: ім'я в профілі можна змінити незалежно від методу створення акаунта.
- SYS-AUTH-08: гість і персонал можуть прив'язати Google-акаунт до існуючого email-акаунта.
- SYS-AUTH-09: відв'язати Google можна лише за наявності альтернативного методу входу.
- SYS-AUTH-10: паролі зберігаються виключно у вигляді bcrypt-хешів (salt-rounds ≥ 12).
- SYS-AUTH-11: JWT access-токен діє ≤ 1 годину; refresh-токен зберігається зашифровано.
- SYS-AUTH-12: API перевіряє роль користувача (RBAC) для кожного захищеного ендпоінта.
- SYS-AUTH-13: при недоступності Google OAuth система продовжує роботу через email+пароль.

– SYS-AUTH-14: при вході авторизованого гостя — система перевіряє наявність активного замовлення та відновлює restaurantId автоматично.

– SYS-AUTH-15: пряма реєстрація (email+пароль) активується одразу без email-верифікації; для onboarding нового ресторану обов'язкова email-верифікація (/onboarding/check-email → /onboarding/confirm).

– SYS-AUTH-16: кожен гість може за бажанням видалити свій акаунт.

Критерії приймання модуля:

– AC-AUTH-01: given нова реєстрація, then пароль у БД — bcrypt-хеш; відкритий текст відсутній; акаунт одразу активний.

– AC-AUTH-02: given Google OAuth недоступний, when гість натискає «Увійти через Google», then відображається помилка; кнопка email-входу залишається функціональною.

– AC-AUTH-03: given персонал роллю «cook», when GET /api/admin/analytics, then HTTP 403.

– AC-AUTH-04: given гість увійшов виключно через Google (без пароля), when намагається відв'язати Google, then HTTP 400 «Встановіть пароль перед відв'язуванням Google».

2.1.13 Модуль управління меню

Модуль надає адміністратору повний CRUD-функціонал для всіх сутностей меню: категорій, страв, інгредієнтів, додатків та груп компонентів. Пріоритет: критичний.

Функціональні вимоги модуля:

– SYS-MGMT-01: адмін може створювати, редагувати та видаляти (soft-delete) категорії.

– SYS-MGMT-02: адмін може створювати, редагувати та видаляти (soft-delete) страви.

- SYS-MGMT-03: unitPrice вже розміщених OrderItem залишається незмінним (snapshot ціни).
- SYS-MGMT-04: адмін керує інгредієнтами страви (знімні / обов'язкові).
- SYS-MGMT-05: адмін керує add-ons страви (назва, ціна).
- SYS-MGMT-06: адмін керує ComponentGroups (обов'язковий вибір варіанту).
- SYS-MGMT-07: адмін керує sortOrder категорій та страв.
- SYS-MGMT-08: будь-яка зміна меню → WebSocket MENU_UPDATED всім активним клієнтам.
- SYS-MGMT-09: видалення категорії зі стравами → HTTP 409.
- SYS-MGMT-10: зображення страв: JPG/PNG/WebP, ≤ 5 МБ.

Критерії приймання модуля:

- AC-MGMT-01: given адмін додає страву за 150 грн, when гість відкриває меню, then страва відображається за ≤ 500 мс після збереження.
- AC-MGMT-02: given адмін змінює ціну з 100 на 120 грн, when гість вже має її в підтвердженому замовленні, then unitPrice в OrderItem залишається 100 грн.
- AC-MGMT-03: given адмін видаляє страву (soft-delete), then MENU_UPDATED → клієнти; страва → «Недоступно».
- AC-MGMT-04: given адмін видаляє категорію зі стравами, then HTTP 409; категорія не видаляється.

2.1.14 Модуль управління персоналом та onboarding ресторану

Адміністратор керує обліковими записами персоналу. Перший адміністратор реєструється через процедуру Onboarding при створенні закладу в системі. Один обліковий запис адміністратора прив'язаний рівно до одного ресторану. Email-повідомлення для запрошення персоналу не надсилаються — тимчасові паролі відображаються адміністратору в інтерфейсі одноразово. Для onboarding нового ресторану надсилається email-підтвердження. Пріоритет: високий.

Процедура реєстрації нового ресторану в системі складається з таких кроків:

- Крок 1: власник або менеджер відкриває сторінку реєстрації та вводить: email, пароль, ім'я, назву та адресу ресторану.
- Крок 2: система створює Restaurant (restaurantId) та обліковий запис першого адміністратора з роллю administrator.
- Крок 3: на email надсилається підтверджувальний лист; акаунт активується після підтвердження email.
- Крок 4: адмін входить до системи та може розпочати налаштування: додавати столики, QR-коди, меню та персонал.

Функціональні вимоги модуля:

- SYS-STAFF-01: перший адміністратор реєструється через Onboarding при створенні ресторану; акаунт активується після email-верифікації (маршрут /onboarding/check-email → /onboarding/confirm).
- SYS-STAFF-02: адмін може створювати акаунти для кухарів та офіціантів (email + тимчасовий пароль).
- SYS-STAFF-03: при створенні акаунта система надсилає тимчасовий пароль співробітнику на вказаний email.
- SYS-STAFF-04: адмін може змінювати роль: cook ↔ waiter ↔ administrator.
- SYS-STAFF-05: при зміні ролі всі активні JWT токени анулюються.
- SYS-STAFF-06: адмін може деактивувати акаунт; всі активні сесії анулюються.
- SYS-STAFF-07: ресторан повинен мати мінімум одного активного адміністратора.
- SYS-STAFF-08: адмін не може видалити власний акаунт.
- SYS-STAFF-09: список персоналу з фільтрацією за роллю та статусом.
- SYS-STAFF-10: один адміністраторський акаунт прив'язаний рівно до одного ресторану; спроба реєстрації другого ресторану з тим самим email → HTTP 409.

- SYS-STAFF-11: адмін може скинути пароль будь-якого співробітника.

Критерії приймання модуля:

- AC-STAFF-01: given адмін створює акаунт кухаря, then система надсилає тимчасовий пароль на вказаний email; співробітник входить в акаунт з отриманим паролем.
- AC-STAFF-02: given адмін деактивує офіціанта, when офіціант намагається увійти, then HTTP 401 «Акаунт деактивовано».
- AC-STAFF-03: given єдиний адміністратор, when деактивація, then HTTP 409.
- AC-STAFF-04: given власник реєструє ресторан через Onboarding, then restaurantId та акаунт адміна створені; обліковий запис одразу активний.
- AC-STAFF-05: given адмін намагається зареєструвати другий ресторан зі своїм email, then HTTP 409 «Обліковий запис вже прив'язаний до ресторану».

2.1.15 Модуль офлайн-режиму клієнтського інтерфейсу (PWA)

Клієнтський інтерфейс реалізується як Progressive Web App (PWA) з підтримкою часткового офлайн-режиму через Service Worker. Весь функціонал, що не вимагає серверної взаємодії, повинен залишатися доступним при відсутності інтернет-з'єднання. Session token зберігається в HttpOnly cookie; кошик та черга замовлень — у localStorage. Пріоритет: середній.

Функціональні вимоги модуля:

- SYS-PWA-01: Service Worker кешує статичні ресурси (JS, CSS, шрифти, іконки) при першому завантаженні застосунку.
- SYS-PWA-02: дані меню (категорії, страви, інгредієнти, add-ons, ComponentGroups) кешуються після першого успішного завантаження.
- SYS-PWA-03: кошик зберігається в localStorage; відновлюється при повторному відкритті навіть без мережі.

- SYS-PWA-04: якщо гість натиснув «Підтвердити» в офлайн — замовлення ставиться в чергу в localStorage та відправляється при відновленні з'єднання з нотифікацією «Замовлення відправляється...».
- SYS-PWA-05: черга очікуючих запитів зберігається в localStorage; при закритті та повторному відкритті браузера черга не втрачається.
- SYS-PWA-06: клієнт відображає banner «Немає з'єднання» при відсутності мережі; banner зникає при відновленні.
- SYS-PWA-07: пошук у меню функціонує офлайн на основі кешованих даних.

Критерії приймання модуля:

- AC-PWA-01: given меню завантажено, when гість вимикає мережу, then меню, деталі страв, пошук та кошик залишаються доступними.
- AC-PWA-02: given гість додав страви до кошика офлайн, when мережа відновлена, then кошик збережений; гість може підтвердити замовлення.
- AC-PWA-03: given гість натиснув «Підтвердити» офлайн, then система показує «Замовлення відправляється...»; після відновлення мережі замовлення відправляється автоматично; гість отримує підтвердження.
- AC-PWA-04: given мережа відсутня, then відображається banner «Немає з'єднання»; при відновленні banner зникає; статус замовлення оновлюється автоматично за ≤ 5 с.

2.2 Огляд нефункціональних вимог

Нефункціональні вимоги визначають якісні характеристики системи та поділяються на шість категорій.

2.2.1 Вимоги до продуктивності

Час завантаження клієнтського інтерфейсу не повинен перевищувати 2 секунди при підключенні 4G. Затримка передачі замовлення через WebSocket — не більше 300 мс. Час відповіді REST API — не більше 500 мс для 95% запитів.

Система розрахована на одночасну обробку до 100 активних WebSocket-з'єднань на один ресторан. Доступність системи (uptime) — не менше 99,5% за календарний місяць.

2.2.2 Вимоги до безпеки

Усі мережеві з'єднання здійснюються виключно через HTTPS та WSS. Автентифікація персоналу реалізована через JWT-токени з обмеженим терміном дії. Паролі зберігаються виключно у вигляді bcrypt-хешів з параметром salt-rounds ≥ 12 . Карткові дані обробляються виключно платіжним шлюзом LiqPay і не потрапляють на сервери системи. Усі API-ендпоінти захищені перевіркою ролі користувача за моделлю RBAC. Реалізовано захист від типових векторів атак: XSS, CSRF, NoSQL-ін'єкцій. Фінансові операції фіксуються в незмінному audit log.

2.2.3 Атрибути якості

- Зручність (usability): гість виконує перше замовлення без інструкцій за не більше ніж три дії.
- Надійність (reliability): автоматичне відновлення WebSocket-з'єднання після розриву за не більше 5 секунд.
- Масштабованість (scalability): архітектура передбачає горизонтальне масштабування для обслуговування багатьох ресторанів.
- Супроводжуваність (maintainability): компонентна структура клієнтської частини та модульна архітектура серверної частини забезпечують незалежне оновлення модулів.
- Переносимість (portability): коректне відображення на пристроях з шириною екрана від 320 px до 1920 px.
- Відновлюваність (recoverability): Smart Session Recovery дозволяє відновити сесію після закриття браузера без втрати кошика та поточного статусу замовлення.

2.2.4 Вимоги до сумісності

Підтримуються браузери Chrome 90+, Safari 14+, Firefox 88+. Підтримувані мобільні платформи: iOS 13+, Android 8+. Сканування QR-коду здійснюється стандартною камерою без встановлення додаткових застосунків.

2.2.5 Вимоги до логування

Усі HTTP-запити та WebSocket-події логуються у структурованому JSON-форматі з обов'язковими полями: timestamp, request_id для наскрізного трасування, тип події, статус-код. Фінансові операції записуються в окремий незмінний audit log з мінімальним терміном зберігання 3 роки. Особисті дані (паролі, токени, платіжні дані) не потрапляють до логів.

2.2.6 Вимоги до моніторингу

Система надає health-check ендпоінт із перевіркою стану всіх критичних залежностей (база даних, платіжний шлюз, OAuth-сервіс). Метрики збираються в реальному часі: кількість активних з'єднань, час відповіді, частота помилок. При перевищенні порогів автоматично формуються алерти.

2.3 Вимоги до тарифної моделі

Система реалізує модель Freemium з технічним розподілом функціоналу на рівні бази даних та middleware. Кожен ресторан характеризується значенням поля plan $\in \{\text{free}, \text{premium}\}$, яке визначає набір доступних функцій. Детальний опис розподілу функціоналу між тарифами наведено у п. 1.8 цієї роботи.

З точки зору формування вимог тарифний розподіл породжує такі додаткові функціональні вимоги:

- REQ-PLAN-01: серверна частина перевіряє значення restaurant.plan перед обробкою запитів до тарифно-обмежених ендпоінтів (/payments/initiate, /admin/analytics/*, /admin/menu/pdf, /admin/reviews, /admin/staff для додавання понад

3 акаунтів). При недостатньому рівні підписки повертається HTTP 403 з кодом помилки PLAN_REQUIRED.

– REQ-PLAN-02: серверна частина перевіряє кількісні обмеження для тарифу free при операціях створення сутностей: максимум 5 категорій меню, 50 страв, 3 зображення на страву, 3 акаунти персоналу сумарно. При перевищенні ліміту повертається HTTP 403 з кодом PLAN_LIMIT_EXCEEDED.

– REQ-PLAN-03: клієнтська частина отримує значення plan у складі профілю ресторану та умовно рендерить елементи інтерфейсу: для тарифу free приховуються пункти меню «Аналітика», «Управління відгуками», «Генератор PDF», «Інтеграція платежів», а на формах редагування меню відображається залишок ліміту.

– REQ-PLAN-04: при спробі гостя залишити відгук для замовлення в ресторані з тарифом free система не відображає форму відгуку та не активує відповідну кнопку.

– REQ-PLAN-05: інтеграція з Google Translate (для автоматичного перекладу полів вводу) активується лише для тарифу premium.

3 АРХІТЕКТУРА ТА ПРОЄКТУВАННЯ

У цьому розділі описано процес проектування системи обробки замовлень з QR-меню: наведено UML-діаграми, описано структуру бази даних, алгоритми функціонування ключових підсистем, склад компонентів програмної системи, а також обґрунтовано рішення з UI/UX-проектування.

3.1 UML-діаграми

3.1.1 Use case діаграми

Діаграму use case для ролі «Гість» наведено на рисунку 3.1. Гість отримує доступ до системи двома способами: шляхом сканування QR-коду столика (для замовлення) або через реєстрацію/вхід за email чи Google OAuth. Гість має можливість переглядати меню, формувати кошик, оформлювати замовлення, відстежувати його статус, викликати офіціанта.

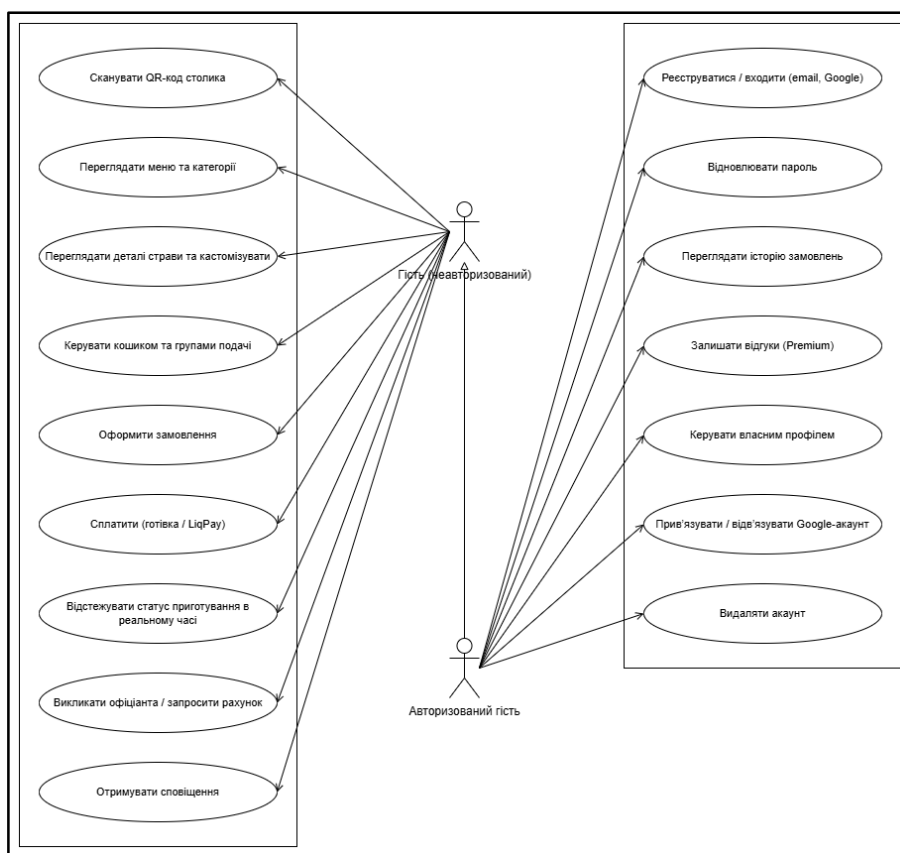


Рисунок 3.1 — Use case діаграма ролі «Гість»

Діаграму use case для ролі «Офіціант» наведено на рисунку 3.2. Офіціант отримує сповіщення про нові замовлення та виклики в реальному часі через WebSocket. Основні дії офіціанта: прийом викликів від гостей (обслуговування або готівкова оплата), перегляд активних замовлень по столах, зміна статусу замовлення. Офіціант також виконує функції кухаря при ролі waiter_cook.

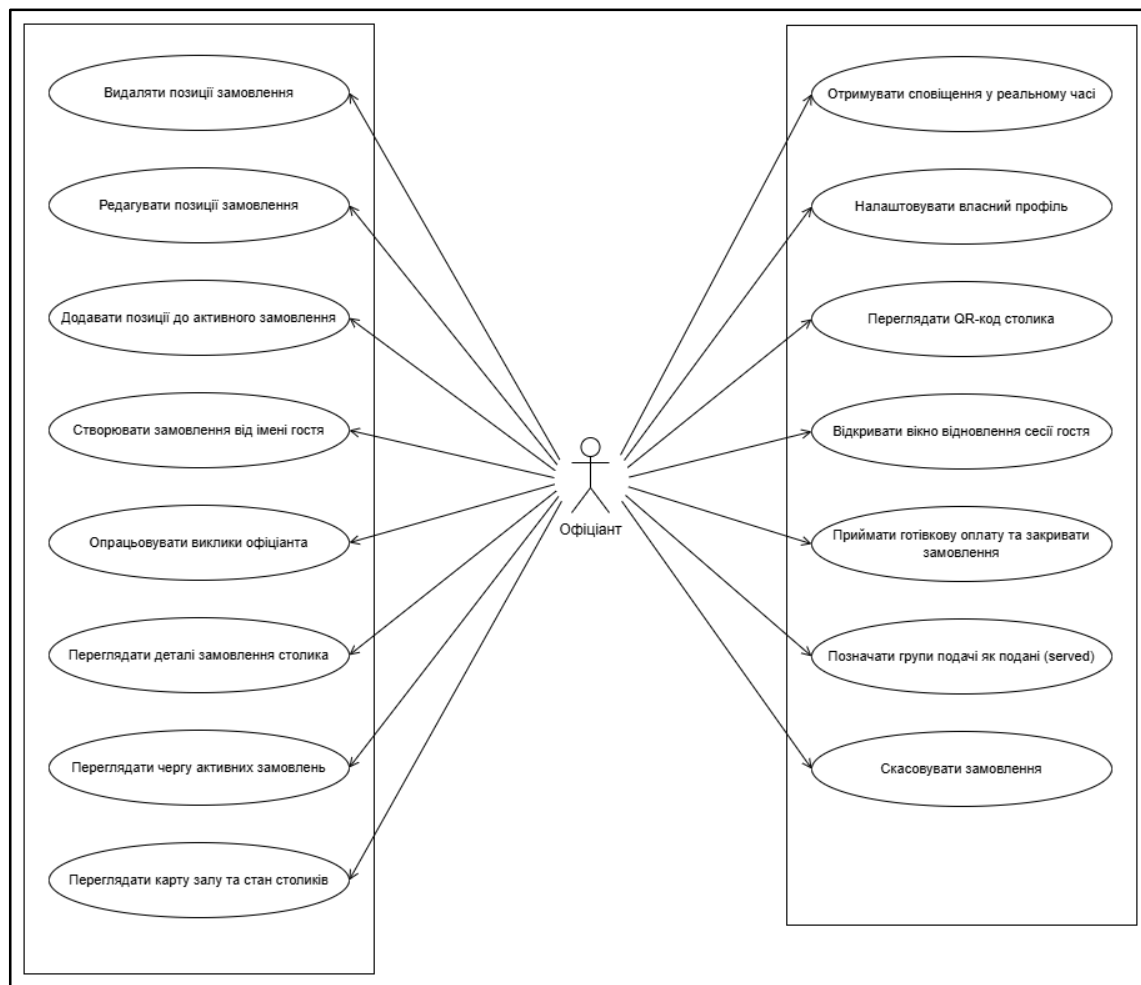


Рисунок 3.2 – Use case діаграма ролі «Офіціант»

Діаграму use case для ролі «Кухар» наведено на рисунку 3.3. Кухар працює з KDS (Kitchen Display System) — системою відображення замовлень для кухні. Основні функції: отримання нових замовлень у реальному часі, перегляд замовлень у двох режимах (Order View та Table View), зміна статусу груп страв (waiting → cooking → ready → served).

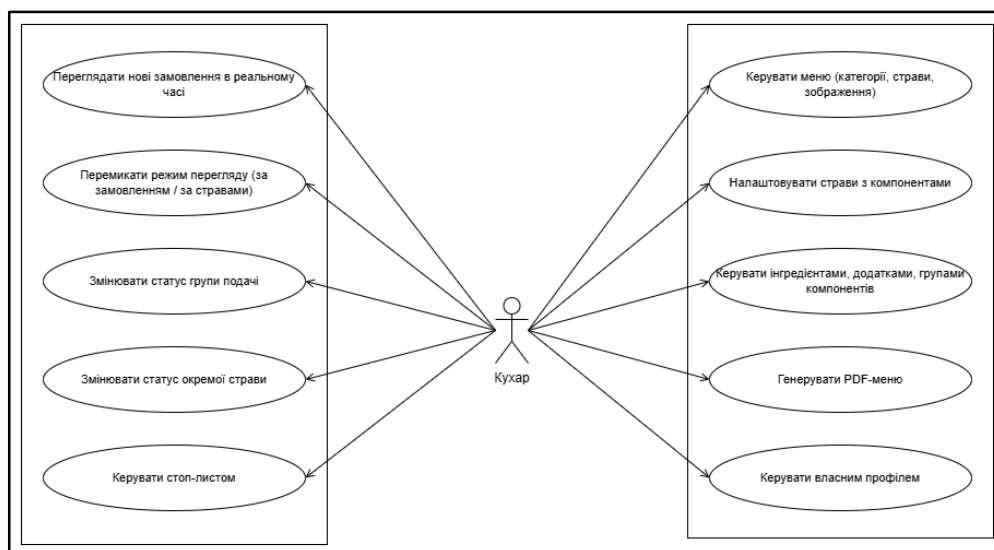


Рисунок 3.3 – Use case діаграма ролі «Кухар»

Діаграму use case для ролі «Адміністратор» наведено на рисунку 3.4. Адміністратор має найширші права: управління меню (CRUD для категорій, страв, інгредієнтів, add-ons, ComponentGroups), управління столиками та QR-кодами, управління персоналом та onboarding, перегляд аналітики, генерація PDF-меню, налаштування ресторану, управління відгуками, ініціювання преміум-підписки.

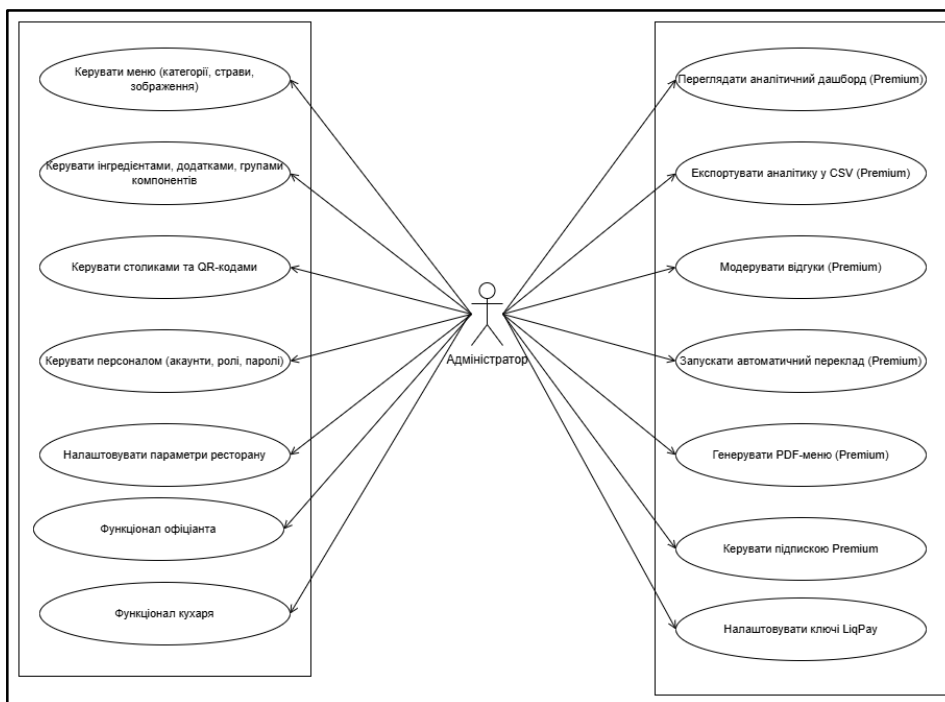


Рисунок 3.4 – Use case діаграма ролі «Адміністратор»

3.1.2 Діаграма станів замовлення

Діаграму станів об'єкта Order наведено на рисунку 3.5. Замовлення переходить зі стану open (активне) через такі стани:

- open_paid — замовлення оплачено онлайн, очікується підтвердження;
- completed_cash — закрито готівковою оплатою;
- completed_epay — закрито електронною оплатою (LiqPay);
- cancelled — скасовано гостем або офіціантом.

Стани completed_cash, completed_epay та cancelled є термінальними — повторне відкриття замовлення неможливе (API повертає HTTP 409). Перехід open → open_paid ініціюється успішним платіжним webhook від LiqPay.

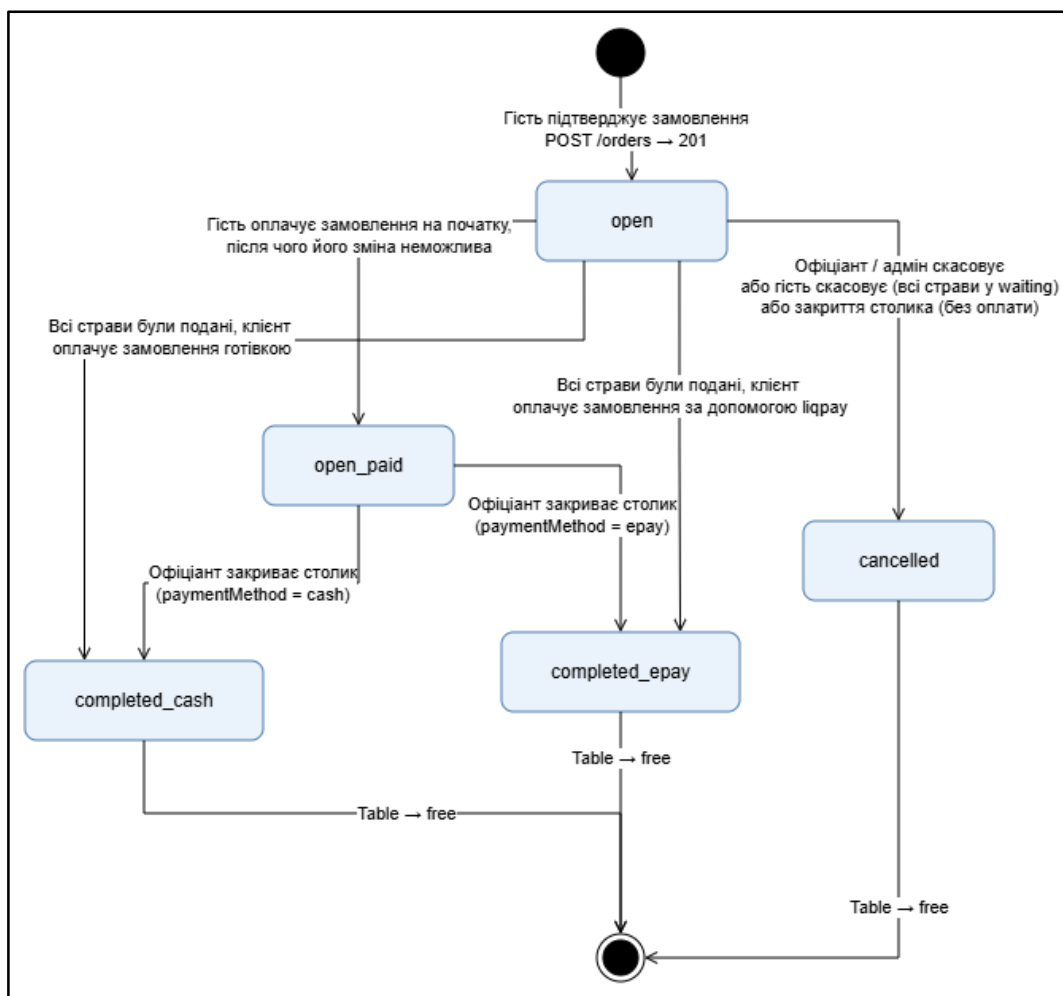


Рисунок 3.5 – Діаграма станів замовлення (Order State Diagram)

3.1.3 Sequence-діаграми

Для ілюстрації взаємодії компонентів системи розроблено три sequence-діаграми, що відображають ключові сценарії роботи.

Sequence-діаграму повного циклу замовлення наведено на рисунку 3.6. Сценарій охоплює: сканування QR-коду гостем та Session Recovery, перегляд меню, формування кошика, замовлення, сповіщення KDS, ініціювання оплати через LiqPay та отримання webhook-підтвердження.

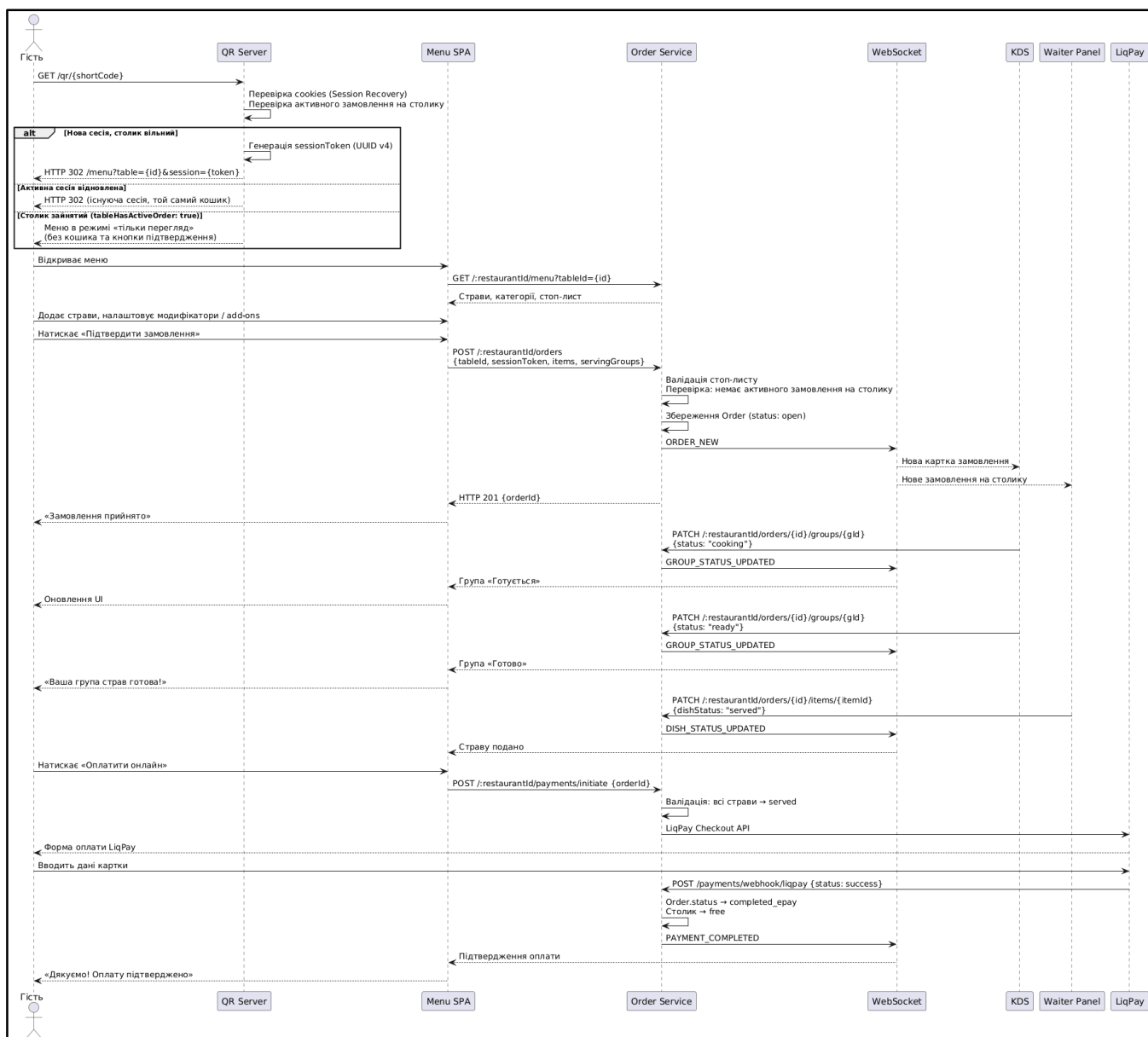


Рисунок 3.6 – Sequence-діаграма повного циклу замовлення

Sequence-діаграму обробки онлайн-оплати та fallback-механізму наведено на рисунку 3.7. Сценарій відображає взаємодію з LiqPay API: у разі таймауту webhook система виконує retry-запити з інтервалами 1, 5 та 15 хвилин. При отриманні підтвердження оплати замовлення переводиться у стан `open_paid`, а клієнт та офіціант отримують WebSocket-подію `PAYMENT_COMPLETED`.

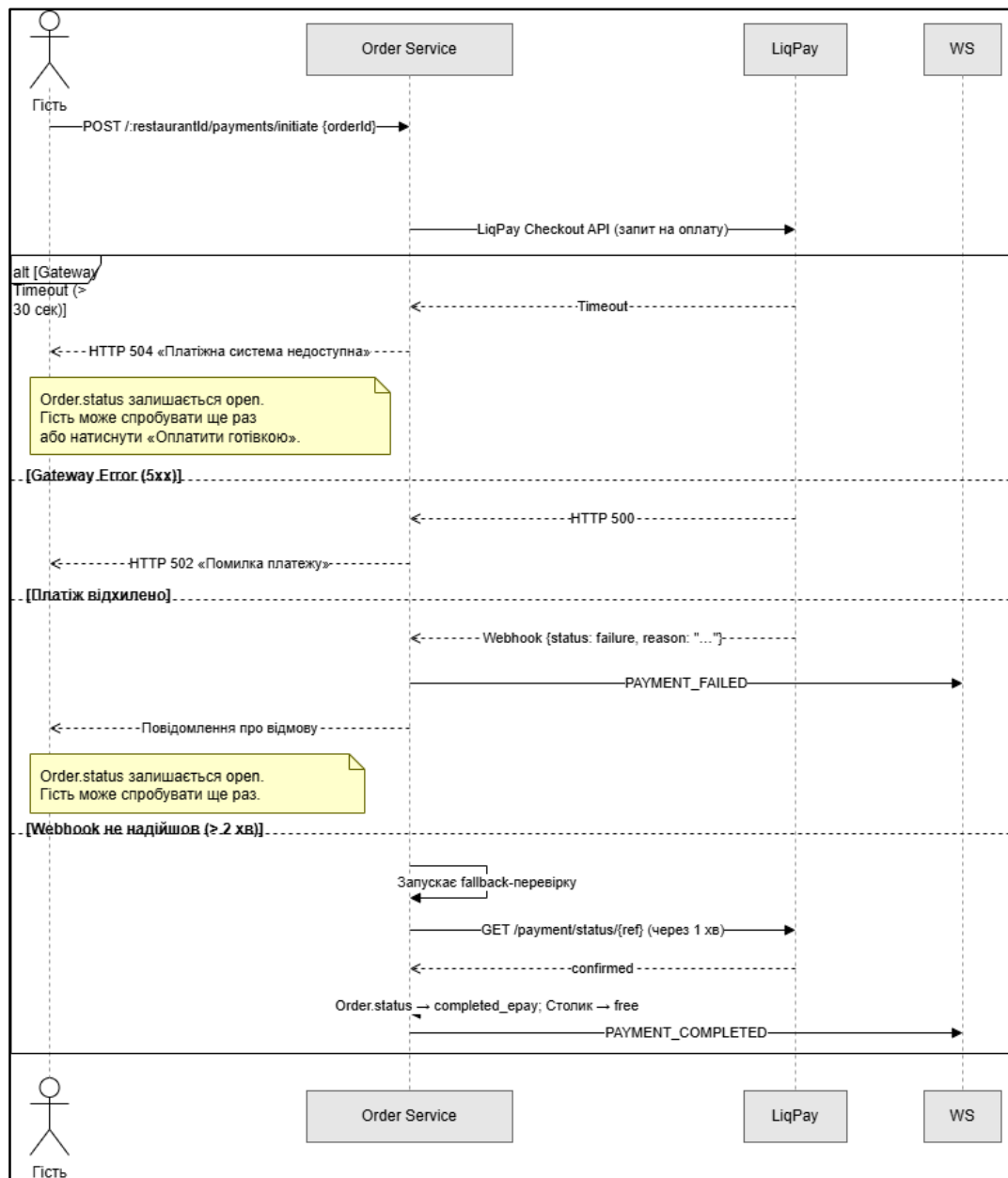


Рисунок 3.7 – Sequence-діаграма обробки помилок платіжного шлюзу та fallback-механізму

Sequence-діаграму відновлення WebSocket-з'єднання та event replay наведено на рисунку 3.8. При розриві з'єднання клієнт виконує повторне підключення з експоненційним backoff (3, 5+ секунд). Після відновлення відправляється запит REPLAY_REQUEST з полем last_event_id. Сервер повертає всі події, збережені в in-memory кеші з TTL 10 хвилин та event_id більшим за вказаний. При переповненні кешу дані запитуються через REST API.

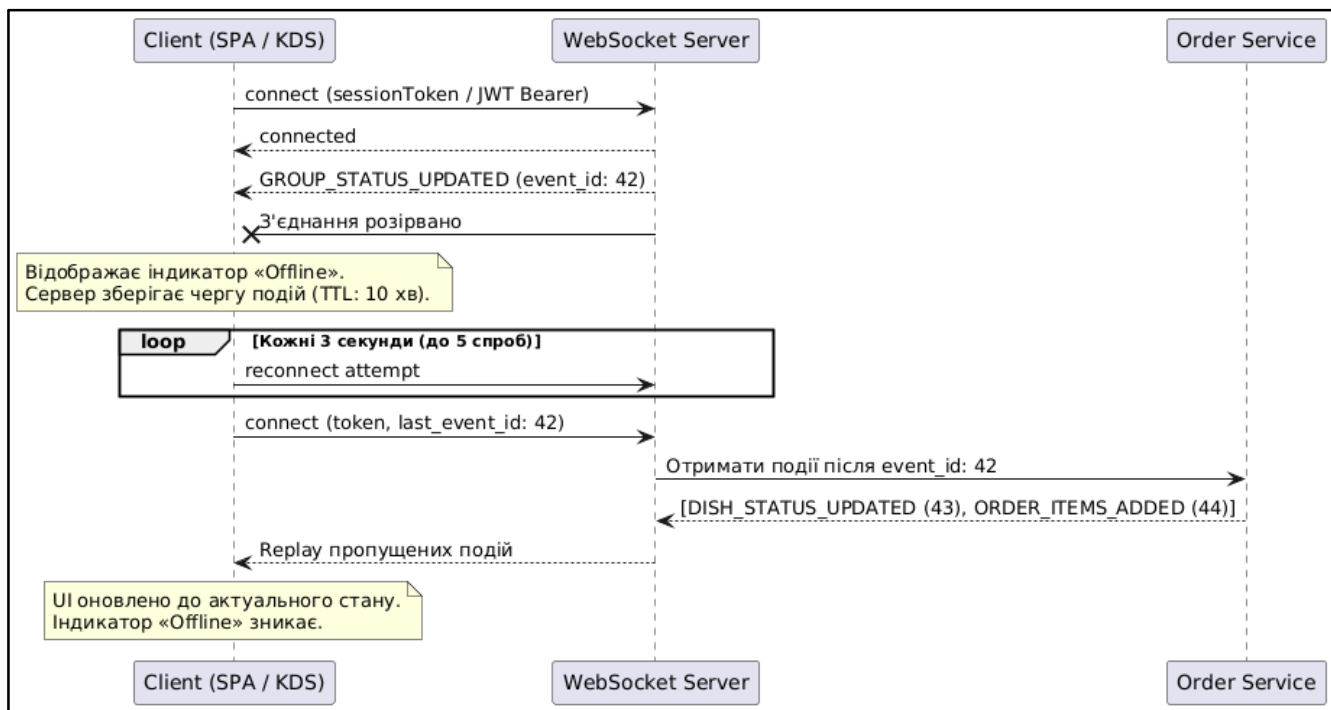


Рисунок 3.8 – Sequence-діаграма відновлення WebSocket-з'єднання та event replay

3.2 Опис структури бази даних

3.2.1 Вибір СУБД

Як систему управління базами даних обрано MongoDB 6.0+ — документоорієнтовану NoSQL СУБД. Вибір обумовлено такими чинниками: гнучка схема документів дозволяє зберігати вкладені об'єкти (наприклад, add-ons та ComponentGroups у пункті меню) без необхідності JOIN-запитів; MongoDB підтримує TTL-індекси для автоматичного видалення застарілих даних (сесії,

сповіщення, audit log); горизонтальне масштабування відповідає вимогам multi-tenant SaaS-архітектури.

Для взаємодії з MongoDB у back-end використовується ODM-бібліотека Mongoose, яка надає типізовані схеми, валідацію та хуки (pre/post hooks) для забезпечення цілісності даних.

3.2.2 Domain-модель

Доменну модель системи наведено на рисунку 3.9. Опис основних колекцій: restaurants — колекція ресторанів. Поля: name, address, plan (free/premium), enabledLanguages, settings. Поле plan визначає доступний набір функцій через middleware.

tables — столики ресторану. Поле shortCode генерується як 4–8 символів [A-Z0-9] (crypto.randomBytes) та є унікальним ключем для QR-коду. Індекс по полях (restaurantId, status) прискорює вибірки активних столиків.

sessions — сесії гостей. TTL-індекс по полю expiresAt (24 години) забезпечує автоматичне видалення.

orders — замовлення. Стан замовлення відстежується через поле status (open, open_paid, completed_cash, completed_pay, cancelled). Складений індекс (tableId, status) використовується для перевірки TABLE_OCCUPIED.

orderItems та servingGroups — позиції та групи подачі замовлення.

menuItems, categories, ingredients, addOns, componentGroups — сутності меню. Всі підтримують soft-delete (поле isDeleted), щоб не порушувати цілісність замовлень, де фіксується snapshot ціни (unitPrice в OrderItem).

waiterCalls — виклики офіціанта. Поле type: call або cash_payment; поле status: active або resolved.

payments — платежі. Зберігають payload для HMAC-SHA1 підпису LiqPay та статус транзакції.

reviews — відгуки. Тип RestaurantReview або DishReview, визначається полем type.

notifications — сповіщення з TTL 7 діб.

auditLog — журнал аудиту (оплати, скасування, walk-out) з TTL 3 роки.

users — облікові записи персоналу. Поля role (cook, waiter, waiter_cook, admin) та googleId (sparse-індекс для Google OAuth).

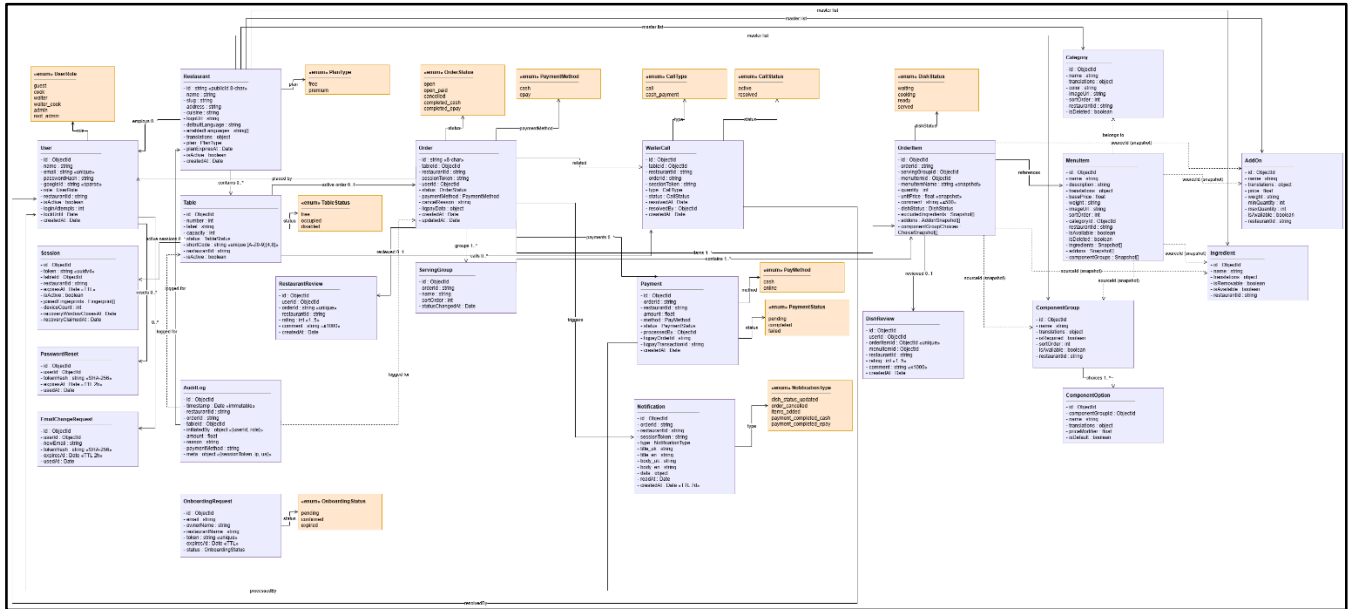


Рисунок 3.9 – Доменна модель (Domain Model) системи QR Restaurant

3.2.3 Цілісність даних

Оскільки MongoDB не підтримує зовнішні ключі на рівні СУБД, цілісність забезпечується засобами Mongoose: pre/post-хуки перевіряють консистентність посилань при видаленні або оновленні документів. Каскадні операції (наприклад, soft-delete позицій меню при видаленні категорії) реалізовано як MongoDB-транзакції з session.

3.3 Опис алгоритмів функціонування

3.3.1 Загальна архітектура системи

Систему побудовано за трирівневою архітектурою (three-tier architecture): рівень представлення — SPA-застосунок на ReactJS; рівень бізнес-логіки — сервер Node.js 18 LTS з Express.js; рівень даних — MongoDB 6.0+.

Загальну схему архітектури наведено на рисунку 3.10.

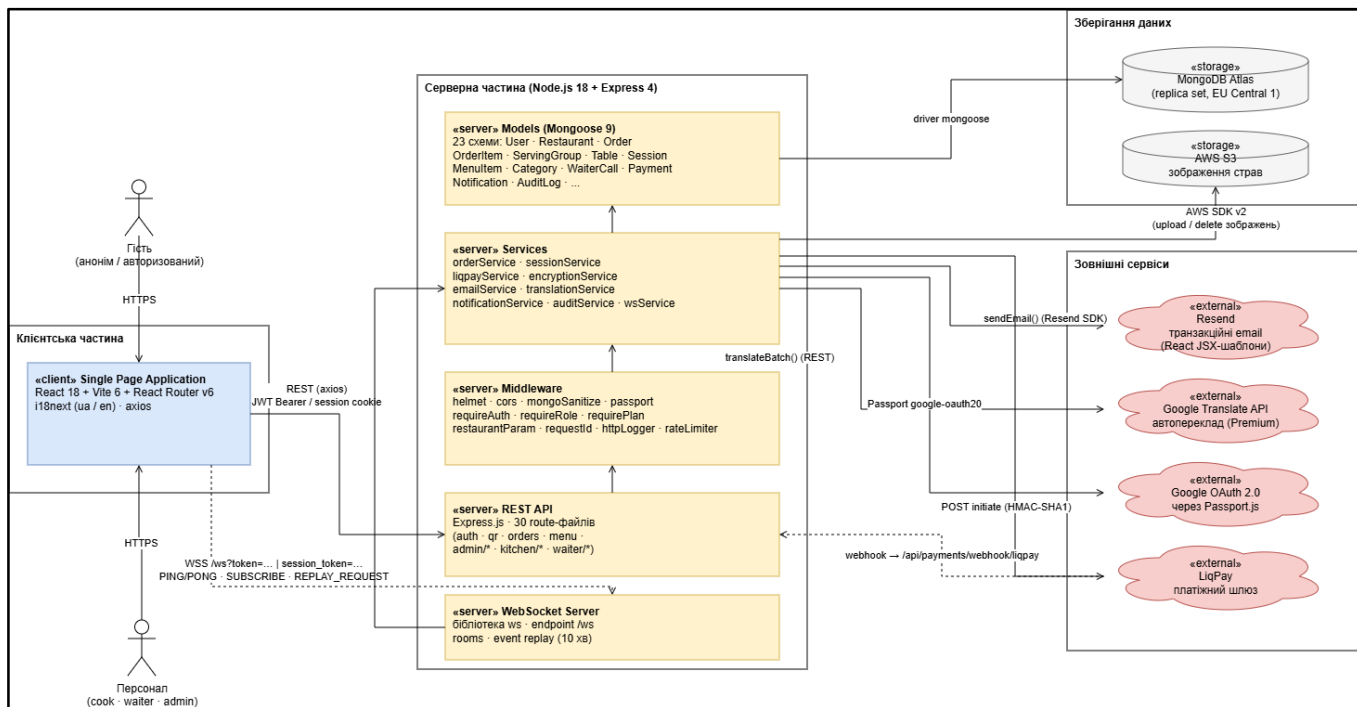


Рисунок 3.10 – Загальна архітектура системи QR Restaurant

Клієнтський застосунок (SPA/PWA) збирається за допомогою Vite та розгортається на CDN у вигляді статичних файлів (HTML, JavaScript, CSS). Backend сервер реалізовує REST API та WebSocket-сервер. Медіафайли зображень зберігаються у хмарному сховищі AWS S3. Зовнішні інтеграції: LiqPay (онлайн-оплата), Google OAuth 2.0 (автентифікація), Resend (email-повідомлення), Google Translate API (переклад меню для Premium).

Система реалізована у вигляді multi-tenant SaaS: кожен ресторан ідентифікується полем `restaurantId` у всіх API-маршрутах та документах MongoDB, що забезпечує повну ізоляцію даних.

3.3.2 Алгоритм Smart Session Recovery

Алгоритм забезпечує збереження кошика та замовлення гостя у разі втрати сесійного cookie (перезавантаження браузера, очищення cookie). При GET `/qr/{shortCode}` сервер перевіряє наявність валідного session token у cookies клієнта:

- якщо токен відсутній або прострочений — генерується новий сесійний токен (32 байти, base64), прив'язаний до `tableId`; встановлюється `HttpOnly Secure` cookie з `Max-Age 86400` (24 год);
- якщо токен дійсний і прив'язаний до того ж столика — відновлюється поточна сесія разом з активним замовленням (`tableHasActiveOrder: true`).

3.3.3 Алгоритм обробки онлайн-оплати

Алгоритм взаємодії з LiqPay реалізовано за схемою webhook. Послідовність кроків:

- 1) клієнт надсилає POST `/:restaurantId/payments/initiate` з `orderId`;
- 2) сервер створює документ `Payment` зі статусом `pending` та формує payload із HMAC-SHA1 підписом;
- 3) система перенаправляє клієнта на платіжний шлюз LiqPay;
- 4) LiqPay надсилає webhook POST `/api/payments/webhook/liqpay` після завершення оплати;
- 5) сервер верифікує підпис, оновлює `Payment.status` та `Order.status` → `open_paid`;
- 6) WebSocket-подія `PAYMENT_COMPLETED` розсилається підписникам `session:{token}` та `waiter:{restaurantId}`.

Fallback-механізм: при відсутності webhook сервер виконує retry-запити до LiqPay API через 1, 5 та 15 хвилин. Fallback доступний лише для Premium-плану.

3.3.4 Алгоритм event replay WebSocket

При розриві WebSocket-з'єднання (таймаут 3–5 секунд) клієнт виконує повторне підключення з використанням exponential backoff. Після відновлення надсилається повідомлення REPLAY_REQUEST з полем last_event_id. Відповідь містить всі події з event_id > last_event_id.

3.4 Опис компонентів програмної системи

3.4.1 Клієнтська частина (Front-end)

Front-end реалізовано у вигляді SPA на базі ReactJS 18 з використанням хуків (hooks). Збірник проекту — Vite, що забезпечує ES-модулі, Hot Module Replacement та оптимізацію tree-shaking і code-splitting.

Структуру каталогів клієнтської частини наведено на рисунку 3.11.

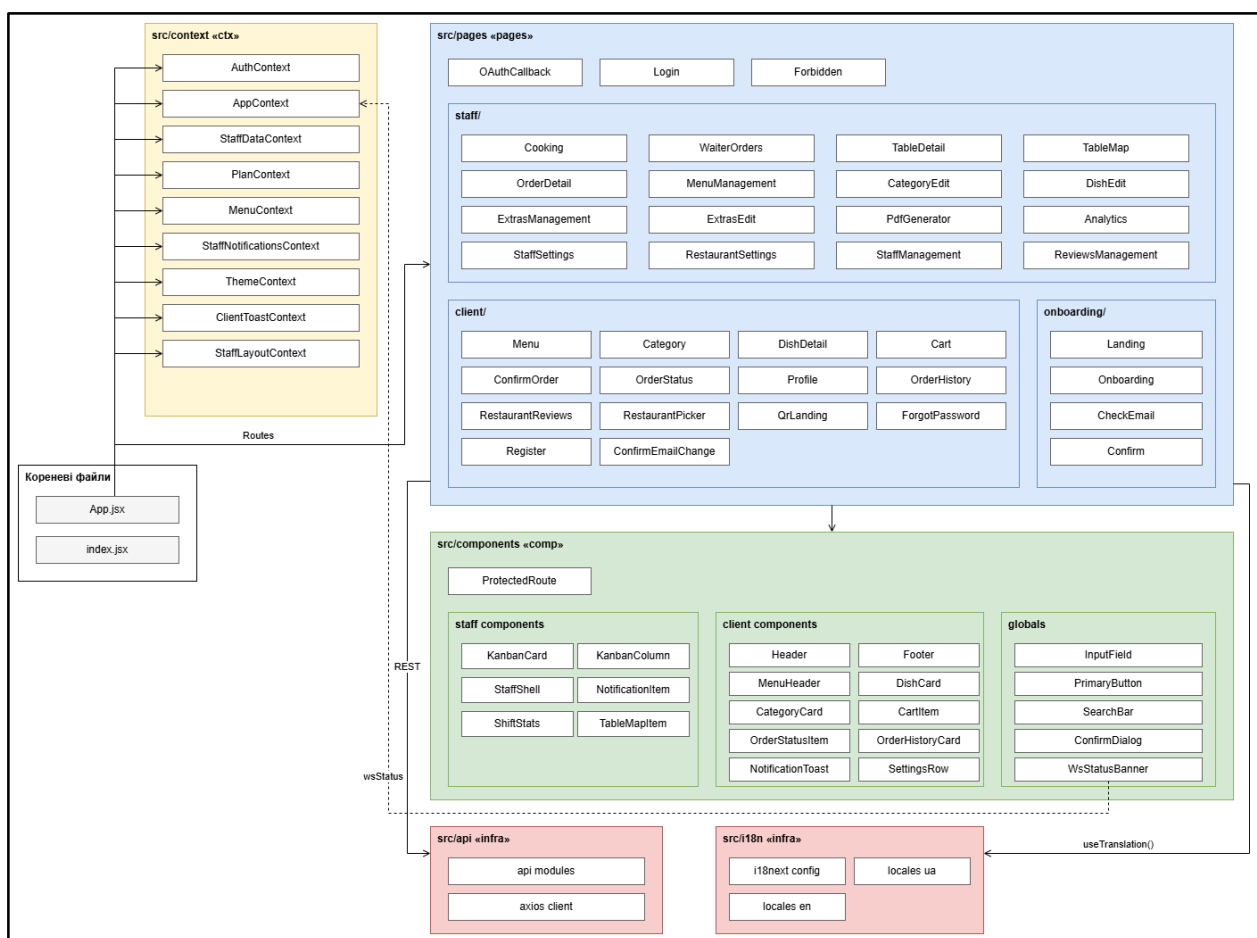


Рисунок 3.11 – Структура каталогів клієнтської частини

Основні модулі:

- src/pages — сторінки застосунку: pages/client (інтерфейс гостя), pages/staff (інтерфейс персоналу), pages/onboarding (реєстрація ресторану), pages/Login. Маршрутизація реалізована через React Router v6 з компонентом ProtectedRoute;
- src/components — перевикористовувані UI-компоненти: базові (InputField, Dropdown, PrimaryButton), layout-компоненти (StaffShell), real-time компоненти (NotificationToast, HttpErrorToast, WsStatusChip);
- src/context — контексти React: AuthContext, AppContext, StaffDataContext, PlanContext, MenuContext, ThemeContext, ClientToastContext, StaffNotificationsContext, StaffLayoutContext;
- src/hooks — кастомні хуки: useWebSocket, usePlan, useLocalField;
- src/i18n — файли локалізації у форматі JSON-словники (i18next);
- src/api — HTTP-клієнт Axios для взаємодії з REST API.

Для підтримки Progressive Web App реалізовано manifest.json та Service Worker із стратегіями: cache-first для статичних ресурсів та network-first з fallback для API-запитів.

3.4.2 Серверна частина (Back-end)

Back-end реалізовано на Node.js 18 LTS з використанням фреймворку Express.js для HTTP-сервера та бібліотеки ws для WebSocket-сервера. Взаємодія з MongoDB виконується через ODM Mongoose.

Структуру серверної частини наведено на рисунку 3.12.

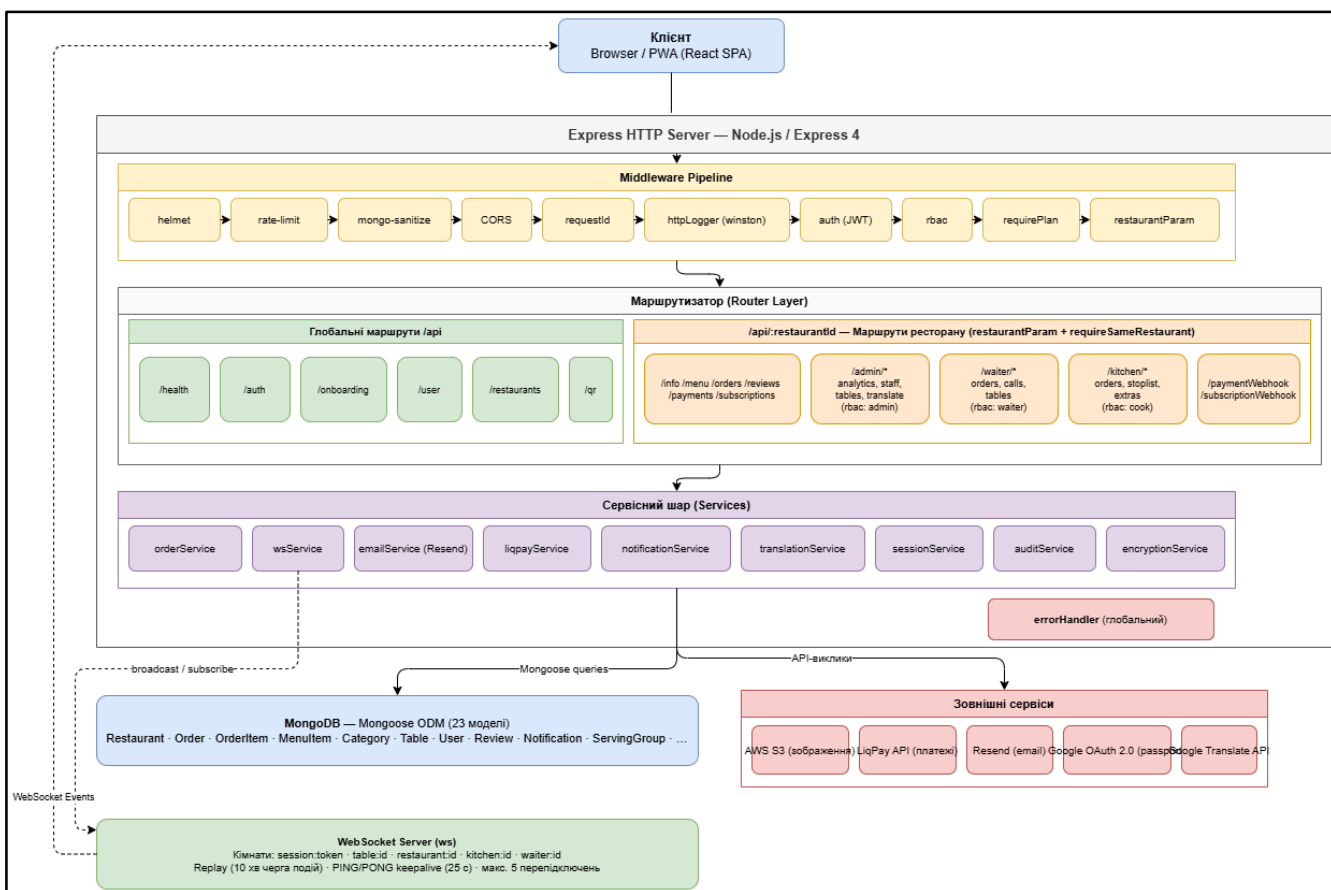


Рисунок 3.12 – Структура серверної частини (HTTP-сервер)

Компоненти серверної частини:

- Routes (маршрути) — модулі у `src/routes`: `auth.js`, `orders.js`, `menu.js`, `payments.js`, `waiterCalls.js`, `analytics.js` та ін. Кожен модуль відповідає за URL-простір конкретного ресурсу;
- Middleware (проміжне ПЗ) — модулі у `src/middleware`: `requireAuth` (перевірка JWT), `requireRole(['admin'])` (перевірка ролі), `requirePlan('premium')` (перевірка плану), `validateBody(schema)` (валідація тіла через Zod), `errorHandler` (обробка помилок);
- Services (сервіси) — модулі у `src/services`: `orderService`, `notificationService`, `paymentWebhookService`, `wsService` (broadcast WebSocket-подій);
- Models (моделі) — Mongoose-схеми у `src/models` відповідно до доменної моделі (рис. 3.9);

– Utils (утиліти) — генератор short-code, хешування, JWT-утиліти, підпис LiqPay.

REST API побудований за RESTful-принципами: іменники в URL, HTTP-методи (GET, POST, PUT/PATCH, DELETE), стандартизовані HTTP-статуси. Формат відповіді: успіх — об'єкт data та meta (request_id), помилка — об'єкт error.code та error.message.

3.4.3 WebSocket-компонент

WebSocket-сервер організовує з'єднання у кімнати (rooms): restaurant:{restaurantId}, table:{tableId}, session:{sessionToken}, kitchen:{restaurantId}, waiter:{restaurantId}. При підключенні клієнт підписується на відповідні кімнати залежно від ролі. Система підтримує 14 типів подій (ORDER_NEW, DISH_STATUS_UPDATED, WAITER_CALL, PAYMENT_COMPLETED тощо) із полями event, payload, timestamp та event_id.

3.5 UI/UX розробка дизайну інтерфейсу

Проектування користувацького інтерфейсу системи QR Rest ґрунтується на принципах орієнтованого на користувача дизайну (User-Centered Design), розмежування інтерфейсів за ролями та забезпечення однорідного візуального досвіду на всіх сторінках застосунку. Система охоплює два окремих інтерфейси: клієнтський (для гостей ресторану) та персоналу (для офіціантів, кухарів, адміністраторів), які реалізовані як єдиний React-SPA з ізольованими маршрутами та контекстами.

3.5.1 Загальні принципи дизайну

Під час проектування інтерфейсу були дотримані ключові принципи дизайну релевантні для предметної області.

Мінімалізм і ясність. Кожна сторінка вирішує одне завдання: сторінка меню — перегляд страв, сторінка кошика — перевірка замовлення, сторінка Cooking —

управління стравами на кухні. Відсутність зайвих елементів знижує когнітивне навантаження на персонал у напруженому ресторанному середовищі.

Консистентність. Усі інтерактивні елементи використовують спільні компоненти: `PrimaryButton`, `SecondaryButton`, `InputField`, `Dropdown`, `ConfirmDialog`. Це забезпечує однаковий вигляд та поведінку кнопок, форм і модальних вікон незалежно від сторінки.

Зворотний зв'язок у реальному часі. Зміни стану замовлення, нові виклики офіціанта та оплати відображаються миттєво через `WebSocket` без перезавантаження сторінки. Компоненти `WsStatusBanner` та `WsStatusChip` інформують користувача про поточний стан з'єднання (`idle` / `connecting` / `connected` / `reconnecting` / `failed`).

Розмежування доступу на рівні UI. Захищені маршрути (`ProtectedRoute`, `RequirePlan`) не лише блокують несанкціонований доступ, а й виключають рендеринг сторінки. Сторінки аналітики та відгуків взагалі не монтуються для безкоштовного тарифу — замість них відбувається перенаправлення на `/staff/map`.

3.5.2 Система кольорів та підтримка тем

Система стилів побудована на основі CSS-змінних, оголошених у `global.css`. Увесь колірний простір задається через набір змінних, що перевизначаються залежно від атрибуту `data-theme` на елементі `<html>`. Підтримуються дві теми: світла (`light`) та темна (`dark`).

Таблиця 3.1 - Основні колірні групи системи

Група	Змінні	Призначення
Фон	<code>--bg-color</code> , <code>--surface-color</code>	Основний фон сторінки та поверхня карток
Текст	<code>--primary-text</code> , <code>--secondary-text</code> , <code>--primary-text-inverted</code>	Основний і допоміжний текст

Продовження таблиці 3.1

Група	Змінні	Призначення
Акцент	--accent-color, --primary-btn-active	Кнопки СТА, посилання, активні елементи
Небезпека	--danger-color, --danger-bg, --danger-text	Скасування, видалення, помилки
Попередження	--warn-bg, --warn-border, --warn-text	Підтвердження, обмеження
Успіх / Інфо	--success-bg, --success-text, --info-bg, --info-text	Позитивні стани
Скелетон	--skel-base, --skel-peak	Shimmer-анімація під час завантаження

Окремо визначені палітри статусів для: стану замовлення, канбан-колонок, бейджів, інгредієнтів, нотифікацій офіціанта та кухні. Завдяки цьому при зміні теми не потрібна модифікація жодного компонента — переключення відбувається через ThemeContext, який записує вибір у localStorage і виставляє data-theme на <html>. Також такий підхід забезпечує легке додавання нових тем інтерфейсу в майбутньому.

3.5.3 Типографіка та компонентна система

Основний шрифт — системний стек без засічок, що забезпечує максимальну читабельність на мобільних пристроях. Для іконок використовується бібліотека react-icons з набором Material Design (префікс Md...)

Стилізація компонентів реалізована через CSS Modules — кожен компонент або сторінка має власний файл *.module.css із локальними класами (styles.myClassName). Це виключає конфлікти імен між модулями. Глобальні стилі (скидання, теми, змінні, анімації) зберігаються у global.css.

3.5.4 Клієнтський інтерфейс (гість)

Клієнтський інтерфейс орієнтований на гостя ресторану, який взаємодіє з системою через особистий смартфон після сканування QR-коду на столику. Оскільки переважна більшість таких користувачів — люди, що вперше бачать цей застосунок, — інтерфейс спроектований за принципом mobile-first і не вимагає реєстрації для виконання основного сценарію: перегляд меню та оформлення замовлення.

Кожна клієнтська сторінка є самодостатньою і має однакову структуру: шапка Header з назвою ресторану та посиланням на профіль, основний контент і підвал Footer. Постійна бокова панель відсутня навмисно — вона б займала цінний горизонтальний простір на мобільному екрані. Натомість для швидкого доступу до кошика або активного замовлення внизу екрана з'являється плаваючий рядок OrderBar, що показує кількість позицій і загальну суму без необхідності переходу на окрему сторінку.

Сторінка меню є точкою входу після сканування QR. Угорі закріплена горизонтальна стрічка категорій, при натисканні на яку сторінка прокручується до відповідного розділу. Нижче відображається сітка карток страв із фотографіями, назвами та цінами. Якщо ресторан підтримує кілька мов, гість може перемкнути мову в шапці, і назви страв зміняться без жодного повторного запиту до сервера.

Сторінка категорії показує список страв обраної категорії у детальнішому форматі з можливістю швидкого додавання до кошика.

Сторінка деталей страви дає гостю повний контроль над замовленням: тут можна виключити небажані інгредієнти, обрати додатки, вибрати варіант у рамках компонентних груп (наприклад, тип соусу або спосіб приготування), задати кількість та залишити коментар для кухні. Вся кастомізація відбувається на одній сторінці без додаткових переходів.

Сторінка кошика відображає поточний склад замовлення, розбитий на групи подачі. Гість може переміщати страви між групами — наприклад, якщо хоче

отримати салат раніше за основну страву. При наявності вже активного замовлення кошик автоматично переходить у режим додавання позицій до існуючого замовлення.

Сторінка підтвердження замовлення дає фінальний перегляд усіх позицій, вибір способу оплати (готівка або онлайн через LiqPay для преміум-ресторанів) і кнопку відправки замовлення на кухню.

Сторінка статусу замовлення — найважливіша сторінка для гостя після оформлення. Вона відображає кожну страву замовлення з кольоровим індикатором статусу: «очікує», «готується», «готово», «подано». Статуси оновлюються в реальному часі через WebSocket без жодних дій з боку гостя. На цій же сторінці відображаються push-подібні сповіщення від ресторану, доступна кнопка виклику офіціанта та, для преміум-ресторанів, кнопка онлайн-оплати.

Сторінка профілю дозволяє гостю змінити ім'я, email, пароль, прив'язати або відв'язати Google-акаунт, обрати мову інтерфейсу, переключити тему та налаштувати звукові сповіщення. Для гостей без акаунту доступна швидка реєстрація — це розблоковує перегляд історії замовлень.

Сторінка історії замовлень показує хронологічний список минулих замовлень з можливістю переглянути склад кожного.

Сторінка відгуків ресторану (доступна лише для преміум-ресторанів) відображає рейтинг і список коментарів гостей, а також форму для подачі власного відгуку.

Сторінка вибору ресторану призначена для гостей, які відкривають застосунок напряду без QR-коду. Вона показує список доступних ресторанів із пошуком за назвою чи адресою.

3.5.5 Інтерфейс персоналу

Інтерфейс персоналу повністю відокремлений від клієнтського і доступний лише автентифікованим користувачам з відповідною роллю. Усі сторінки

персоналу обгорнуті компонентом StaffShell, що формує єдиний каркас: ліворуч вертикальний Sidebar з навігацією, зверху StaffHeader з кнопкою дзвінка, перемикачами мови і теми та меню профілю, праворуч — колапсована панель сповіщень RightPanel, що відкривається по натисканню на дзвінок і показує стрічку останніх подій. Кількість непрочитаних сповіщень відображається як лічильник на іконці дзвінка і оновлюється в реальному часі.

Пункти навігації в бічній панелі динамічно адаптуються до тарифного плану ресторану: «Аналітика» та «Відгуки» для безкоштовного плану позначені іконкою замка і при натисканні відкривають вікно пропозиції переходу на преміум замість переходу на сторінку.

Карта столів є основною сторінкою входу для офіціанта та адміністратора. Вона відображає всі столи ресторану у вигляді сітки, де кожен стіл показує свій номер, кількість активних позицій і колірний статус — вільний, зайнятий або з активним викликом офіціанта. Дані оновлюються через WebSocket миттєво при будь-якій зміні.

Сторінка столика відкривається після натискання на конкретний стіл і дає повний контекст поточного замовлення: склад позицій з їх статусами, можливість редагування або видалення окремих страв, кнопки закриття рахунку (готівка або підтвердження онлайн-оплати), блок активних викликів офіціанта та QR-код столика для повторного сканування гостем.

Список замовлень офіціанта відображає всі активні замовлення ресторану у вигляді черги з пріоритизацією за викликами. Тут офіціант бачить, хто з гостей натиснув кнопку виклику або запросив рахунок, та може позначити виклик як оброблений.

Канбан кухні — головний робочий інструмент кухаря. Сторінка містить три колонки: «Очікує», «Готується» і «Готово», між якими кухар переміщає картки страв одним натисканням. Підтримується два режими перегляду: по страві та по групі подачі — перший зручний для кухні з поділом по станціях, другий — для

відстеження повної готовності страв для конкретного столика. При надходженні нового замовлення або додаванні позицій до існуючого картки з'являються на канбані миттєво без перезавантаження.

Деталі замовлення — розгорнута сторінка конкретного замовлення зі статусами кожної страви, можливістю підтвердити подачу, відредагувати позиції та закрити рахунок.

Управління меню дає адміністратору та кухарю змогу переглядати каталог страв, вмикати й вимикати їх доступність та переходити до редагування. Для безкоштовного тарифу в шапці відображаються лічильники використаного ліміту категорій і страв.

Редактор категорії та редактор страви — форми створення і редагування відповідних сутностей з підтримкою введення назв і описів кількома мовами через вкладки, завантаженням зображень і прив'язкою інгредієнтів, додатків та компонентних груп. Кнопка автоматичного перекладу через Google Translate API доступна лише для преміум-плану.

Управління додатками охоплює редагування майстер-каталогу інгредієнтів, додатків і компонентних груп із можливістю вмикати та вимикати їх доступність у меню.

Генератор PDF-меню дозволяє адміністратору налаштувати зовнішній вигляд меню для друку: обрати шаблон, формат аркуша, мову, шрифт, кольори, вміст і порядок категорій. Сторінка доступна на безкоштовному плані, проте при спробі генерації одразу відкривається вікно пропозиції преміуму.

Аналітика (лише преміум) показує дашборд з виручкою, кількістю замовлень, топ-стравами та топ-категоріями за обраний період з можливістю експорту у CSV.

Налаштування персоналу — сторінка особистого кабінету, де будь-який співробітник може змінити ім'я, email, пароль, прив'язати Google-акаунт, обрати мову та тему, а також налаштувати звукові сповіщення.

Налаштування ресторану доступні лише адміністратору і дозволяють змінити назву закладу, логотип, увімкнуті мови, ключі LiqPay для онлайн-оплати та запустити автоматичний переклад меню.

Управління персоналом дає адміністратору змогу створювати облікові записи для нових співробітників, призначати їм ролі, деактивувати та скидати паролі. На безкоштовному плані кнопка додавання нового співробітника блокується при досягненні ліміту у три акаунти.

Модерація відгуків (лише преміум) відображає всі відгуки гостей про ресторани і страви з можливістю видалення небажаних.

3.5.6 Преміум-інтерфейс та обмеження плану

Розмежування між безкоштовним і преміум-планом реалізовано на рівні інтерфейсу максимально ненав'язливо. Блоковані функції не приховані повністю — користувач бачить їх, але при спробі використати отримує вікно UpgradeModal з переліком переваг та актуальною ціною підписки, яка завантажується динамічно з сервера. Такий підхід інформує адміністратора про можливості преміуму без агресивного нав'язування і дозволяє прийняти зважене рішення щодо переходу на платний план.

3.5.7 Мультимовність в інтерфейсі

Перемикання мови інтерфейсу відбувається миттєво і не вимагає повторних запитів до сервера, оскільки backend при кожному запиті повертає одночасно назви та описи на всіх підтримуваних мовах. Хук useLocalField() автоматично вибирає потрібне мовне поле залежно від поточної мови, встановленої в i18next. Текстовий контент інтерфейсу — підписи кнопок, повідомлення про помилки, назви статусів — зберігається у файлах просторів імен (menu, staffSettings, cooking тощо) окремо для кожної мови, що дозволяє додавати нові мови без змін у коді компонентів.

4 ОПИС ПРИЙНЯТИХ ПРОГРАМНИХ РІШЕНЬ

У цьому розділі розглянуто ключові програмні рішення, прийняті при реалізації системи, з наведенням фрагментів реального коду. Основну увагу приділено тим механізмам, що визначають нефункціональні характеристики системи: єдиному WebSocket-з'єднанню, підтримці багатомовності, технічному розподілу тарифів, офлайн-режиму та системі сповіщень персоналу.

4.1 Структура коду проєкту

Проєкт організовано як два незалежні застосунки: клієнтська частина (каталог QR Rest) та серверна частина (каталог QR Rest API). Клієнтська частина побудована на Vite-збірці та містить такі ключові каталоги: `src/pages` (сторінки, згруповані за ролями), `src/components` (багаторазові компоненти інтерфейсу), `src/context` (дев'ять глобальних контекстів React), `src/hooks` (користувацькі хуки, зокрема `useWebSocket` та `usePlan`), `src/i18n` (інфраструктура багатомовності), `src/utlis` (допоміжні модулі, зокрема черга офлайн-замовлень) та `src/api` (обгортки HTTP-запитів на основі `Axios`).

Серверна частина організована за шаровою архітектурою (детально розглянуто у підрозділі 3.3) із каталогами `src/routes`, `src/middleware`, `src/services`, `src/models` та `src/utlis`, а також каталогом `scripts`, що містить службові скрипти, зокрема `db-seed.js` для наповнення бази даних тестовими даними.

4.2 Управління станом через контексти React

Глобальний стан клієнтської частини інкапсульований у дев'яти контекстах React, кожен з яких відповідає за окрему предметну область, що відповідає принципу єдиної відповідальності та спрощує супроводжуваність коду [9]. Контексти об'єднані в ієрархію провайдерів, у якій зовнішні провайдери надають дані внутрішнім.

Перелік реалізованих контекстів:

- AuthContext — стан автентифікації (профіль користувача, JWT-токени, методи login, logout, deleteAccount).
- AppContext — центральний контекст (близько 1200 рядків коду): кошик, життєвий цикл замовлення, відновлення активного замовлення при вході, метадані ресторану, єдине WebSocket-з'єднання та сесія гостя.
- StaffDataContext — кешовані ресурси панелі персоналу (столики, категорії, страви, персонал) із ледачим завантаженням та узгодженням через WebSocket.
- PlanContext — похідний контекст тарифного плану ресторану (plan, isPremium).
- MenuContext — клієнтський кеш меню з обмеженням актуальності даних (5 хвилин для списку, 3 хвилини для деталей страви).
- ThemeContext — керування світлою / темною темою.
- ClientToastContext — глобальні toast-повідомлення для гостьового інтерфейсу.
- StaffNotificationsContext — сповіщення персоналу в реальному часі (розглянуто у підрозділі 4.7).
- StaffLayoutContext — стан бічної панелі сповіщень у каркасі сторінок персоналу.

Універсальний хук usePlan інкапсулює логіку визначення тарифу: він спершу звертається до PlanContext (для персоналу), потім до поля restaurantPlan у AppContext (для гостьового потоку), а за відсутності даних повертає значення free за замовчуванням. Такий підхід дозволяє використовувати єдиний інтерфейс перевірки тарифу як на сторінках персоналу, так і в гостьовому інтерфейсі.

4.3 Найцікавіші модулі або компоненти системи

4.3.1 Архітектура єдиного WebSocket-з'єднання

Для уникнення дублювання з'єднань реалізовано єдине глобальне WebSocket-з'єднання, що належить AppContext і використовується всіма компонентами через

тонкий хук-обгортку `useWebSocket`. До рефакторингу кожна сторінка відкривала власне з'єднання, що для користувача-персоналу при навігації означало 4–6 паралельних з'єднань до того самого сервера. Реалізацію хука наведено у лістингу 4.1.

Лістинг 4.1 — Хук `useWebSocket`: підписка на єдине глобальне з'єднання

```
export function useWebSocket({ onMessage, enabled = true }) {
  const { wsStatus, wsLatency, wsSubscribe,
    addWsListener, removeWsListener } = useApp();
  const handlerRef = useRef(onMessage);

  // Зберігаємо актуальне посилання на обробник без повторної підписки
  useEffect(() => { handlerRef.current = onMessage; }, [onMessage]);

  useEffect(() => {
    if (!enabled) return;
    const fn = (msg) => handlerRef.current?.(msg);
    addWsListener(fn);
    return () => removeWsListener(fn);
  }, [enabled, addWsListener, removeWsListener]);

  return { status: wsStatus, latency: wsLatency, subscribe: wsSubscribe };
}
```

Кожен компонент-споживач лише реєструє власний слухач (`listener`) на спільному з'єднанні через `addWsListener` та автоматично відписується при демонтуванні. Використання `ref` для збереження обробника дозволяє оновлювати функцію-обробник без повторної підписки при кожному перерендерингу. Таким чином, увесь застосунок персоналу використовує одне `WebSocket`-з'єднання, що знижує навантаження на сервер та спрощує керування станом з'єднання.

4.3.2 Підтримка багатомовності

Багатомовність меню реалізована за схемою суфіксних полів: базове поле (наприклад, `name`) зберігає значення мовою-джерелом, а додаткові поля з мовним суфіксом (`name_en`, `name_pl`) — переклади. Хук `useLocalField` повертає функцію-локалізатор, що обирає коректне поле залежно від активної мови інтерфейсу. Реалізацію наведено у лістингу 4.2.

Лістинг 4.2 — Хук `useLocalField`: вибір локалізованого поля об'єкта

```

export function useLocalField() {
  const { i18n } = useTranslation();

  return (obj, field) => {
    if (!obj) return '';
    const lang = i18n.language;

    // Мова-джерело -> базове поле (наприклад, 'name')
    if (lang === SOURCE_LANG) return obj[field] ?? '';

    // Спершу поле поточної мови, потім відкат до мови-джерела
    const localKey = fieldFor(field, lang);
    return obj[localKey] ?? obj[field] ?? '';
  };
}

```

Перевага такого рішення полягає у незалежності від кількості мов: додавання нової мови не потребує зміни структури документів, а лише реєстрації нового мовного коду. За відсутності перекладу для поточної мови функція повертає значення мовою-джерелом, що гарантує відсутність порожніх полів в інтерфейсі. Окремий хук `useFallbackField` додатково повідомляє, чи показане значення є відкатом до мови-джерела, що використовується для візуальної індикації неперекладених полів в адміністративній панелі.

4.3.3 Реалізація тарифних обмежень

Технічний розподіл функціоналу між тарифами `free` та `premium` реалізовано на серверній стороні через окреме middleware `requirePlan`, що перевіряє значення поля `plan` у документі ресторану перед обробкою запиту до тарифно-обмеженого ендпоінта. Реалізацію наведено у лістингу 4.3.

Лістинг 4.3 — Middleware `requirePlan`: перевірка тарифного плану ресторану

```

function requirePlan(requiredPlan) {
  return function (req, res, next) {
    const plan = req.restaurant?.plan || 'free';
    if (plan === requiredPlan || (requiredPlan === 'free')) {
      return next();
    }
    return res.status(403).json({
      error: {
        code: 'PLAN_REQUIRED',
        message: `This feature requires the ${requiredPlan} plan`,
        requiredPlan, currentPlan: plan,
      },
    });
  };
}

```

```

    meta: {},
  });
};
}
module.exports = requirePlan;

```

Middleware повертає функцію-обробник Express, яку приєднують до маршрутів преміум-функціоналу (ініціація онлайн-оплати, аналітика, генерація PDF-меню, управління відгуками). За недостатнього рівня підписки повертається відповідь HTTP 403 з машинозчитуваним кодом помилки `PLAN_REQUIRED`, який клієнтська частина використовує для відображення вікна пропозиції оновлення тарифу. Такий централізований підхід забезпечує єдину точку контролю тарифних обмежень і дозволяє додавати нові обмеження без модифікації кожного контролера окремо.

4.3.4 Офлайн-черга замовлень

Для підтримки офлайн-режиму (вимога SYS-PWA-04) реалізовано чергу замовлень у `localStorage`. Коли гість підтверджує замовлення без мережі, корисне навантаження запиту зберігається в черзі, а інтерфейс одразу отримує синтетичний плейсхолдер; при відновленні з'єднання `AppContext` послідовно відправляє накопичені замовлення. Черга переживає закриття та повторне відкриття браузера завдяки зберіганню в `localStorage`. Функцію додавання замовлення до черги наведено у лістингу 4.4.

Лістинг 4.4 — Функція `enqueueOrder`: постановка замовлення в офлайн-чергу

```

const KEY = 'pendingOrderQueue';

export function enqueueOrder({ restaurantId, payload }) {
  const items = readQueue();
  const entry = {
    id: uuid(),
    restaurantId,
    payload,
    queuedAt: new Date().toISOString(),
  };
  items.push(entry);
  writeQueue(items);
  return entry;
}

```

Кожен запис черги містить локально згенерований ідентифікатор (для дедуплікації та відображення в інтерфейсі), ідентифікатор ресторану, корисне навантаження запиту та час постановки в чергу. Послідовна відправка по одному запиту за раз гарантує, що сервер не отримає дублікатів замовлення для одного столика. Функції `readQueue` та `writeQueue` інкапсулюють роботу з `localStorage` із безпечним опрацюванням винятків (вимкнене сховище, перевищення квоти).

4.3.5 Система сповіщень персоналу за ролями

Сповіщення персоналу в реальному часі реалізовані в контексті `StaffNotificationsContext`, який підписується на єдине `WebSocket`-з'єднання та фільтрує події залежно від ролі користувача. Відповідність ролей і типів подій задана декларативно у структурі `ROLE_EVENTS`, наведеній у лістингу 4.5.

Лістинг 4.5 — Структура `ROLE_EVENTS`: фільтрація подій за роллю персоналу

```
const ROLE_EVENTS = {
  cook:      ['ORDER_NEW', 'ORDER_ITEMS_ADDED'],
  waiter:    ['ORDER_NEW', 'WAITER_CALL', 'WAITER_CALL_CASH'],
  waiter_cook: ['ORDER_NEW', 'ORDER_ITEMS_ADDED',
    'WAITER_CALL', 'WAITER_CALL_CASH'],
  admin:     ['ORDER_NEW', 'ORDER_ITEMS_ADDED', 'WAITER_CALL',
    'WAITER_CALL_CASH', 'ORDER_VOID',
    'ORDER_CANCELLED', 'PAYMENT_COMPLETED'],
};
```

При надходженні `WebSocket`-події обробник перевіряє, чи дозволений її тип для поточної ролі користувача, і лише тоді формує об'єкт сповіщення та відтворює звуковий сигнал. Такий підхід гарантує, що кухар отримує лише сповіщення про нові та доповнені замовлення, офіціант — про нові замовлення та виклики, а адміністратор — повний набір подій, включно зі скасуваннями та підтвердженнями оплати. Тексти сповіщень формуються через систему перекладів `i18next`, що забезпечує їх відображення мовою інтерфейсу персоналу.

4.4 Інтерфейси системи

4.4.1 Інтерфейс гостя

4.4.1.1 Головна сторінка меню та корзина

Головна сторінка гостя відображає меню ресторану, організоване за категоріями. Кожна позиція меню містить назву, опис, ціну, фотографію та кнопку додавання до кошика. Підтримується пошук страв (case-insensitive) та фільтрація за категоріями. При наявності перекладу (Premium) гість може обрати мову меню.

Екран меню та кошика наведено на рисунку 4.1.

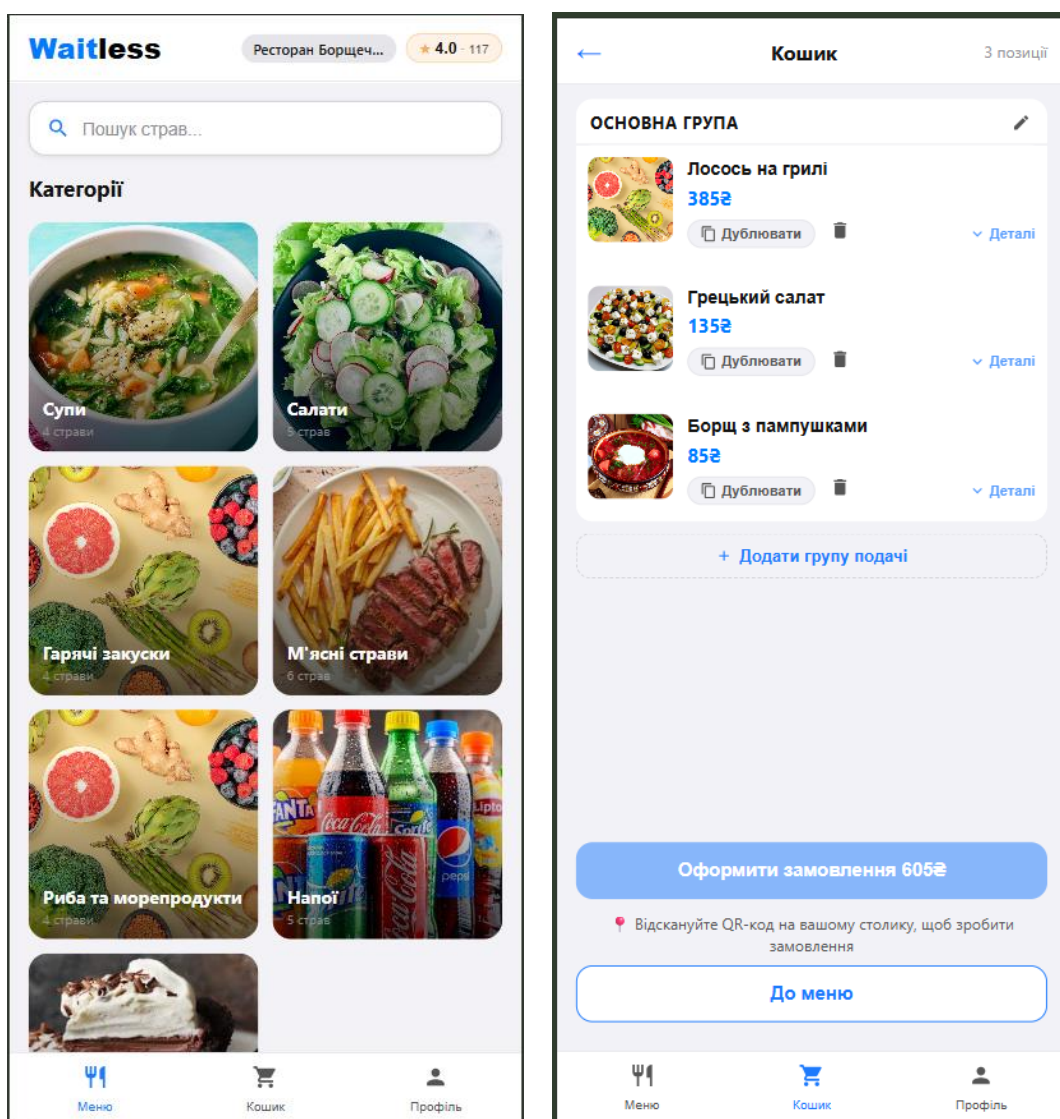


Рисунок 4.1 – Інтерфейс гостя: сторінка меню та кошика

4.4.4.2 Оформлення замовлення та відстеження статусу

Після підтвердження кошика гість переходить на сторінку статусу замовлення. Сторінка відображає поточний стан замовлення та страв у реальному часі через WebSocket. Гість бачить: перелік замовлених страв та їх статус (очікує / готується / готово), загальну суму, кнопки виклику офіціанта та ініціювання оплати.

Екрани оформлення та відстеження замовлення наведено на рисунку 4.2.

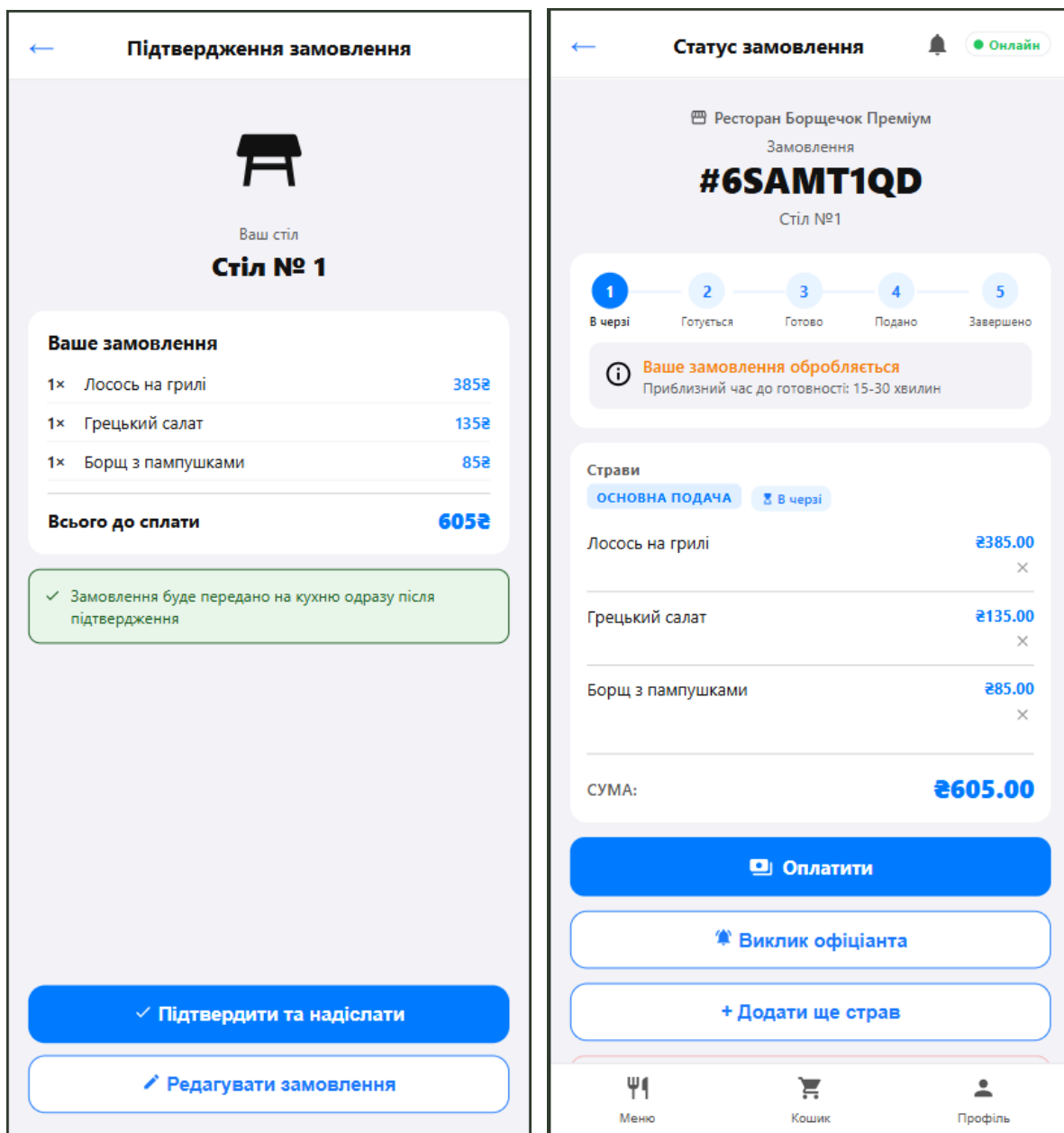


Рисунок 4.2 – Інтерфейс гостя: оформлення та відстеження статусу замовлення

4.4.4.3 Сторінки деталей страви та профілю

Сторінка деталей страви надає повну інформацію про позицію та дозволяє налаштувати замовлення: гість переглядає зображення, опис і оцінку страви, прибирає дозволені інгредієнти, обирає платні додатки й обов'язкові групи компонентів, додає коментар та задає кількість порцій, при цьому ціна перераховується автоматично. Сторінка профілю містить історію замовлень, налаштування облікового запису та застосунку, а також під'єднання до столика за QR-кодом, де введення коду зайнятого столика заблоковано з міркувань безпеки.

Екрани деталей страви та профілю наведено на рисунку 4.3.

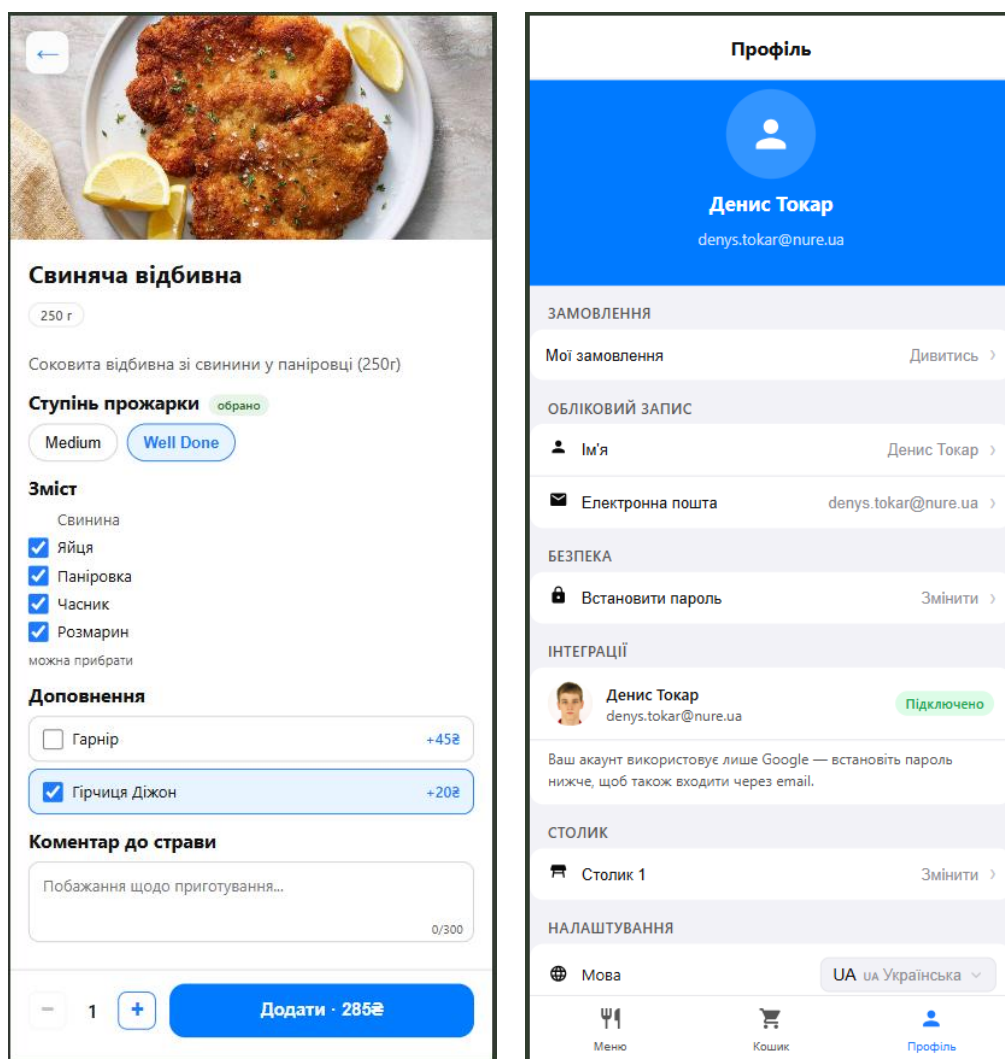


Рисунок 4.3 – Інтерфейс гостя: деталі страви та сторінка профілю

4.4.2 Інтерфейс офіціанта

Інтерфейс офіціанта доступний після авторизації з роллю waiter або waiter_cook. Усі дані оновлюються в реальному часі через WebSocket без періодичного опитування, а компонент WsStatusChip у заголовку відображає стан з'єднання (Online / Reconnecting / Offline). Toast-сповіщення інформують про нові замовлення, виклики гостей та готовність страв.

4.4.2.1 Карта залу

Головний екран офіціанта — карта залу, що відображає всі столики у вигляді сітки з їх номерами, кількістю місць та поточним станом. Кольорове кодування дозволяє швидко оцінити ситуацію: вільний столик, зайнятий, виклик офіціанта чи запит на оплату. У картці кожного зайнятого столика наведено перелік замовлених страв із позначкою статусу приготування (очікує / готується / готово). Карту залу наведено на рисунку 4.4.

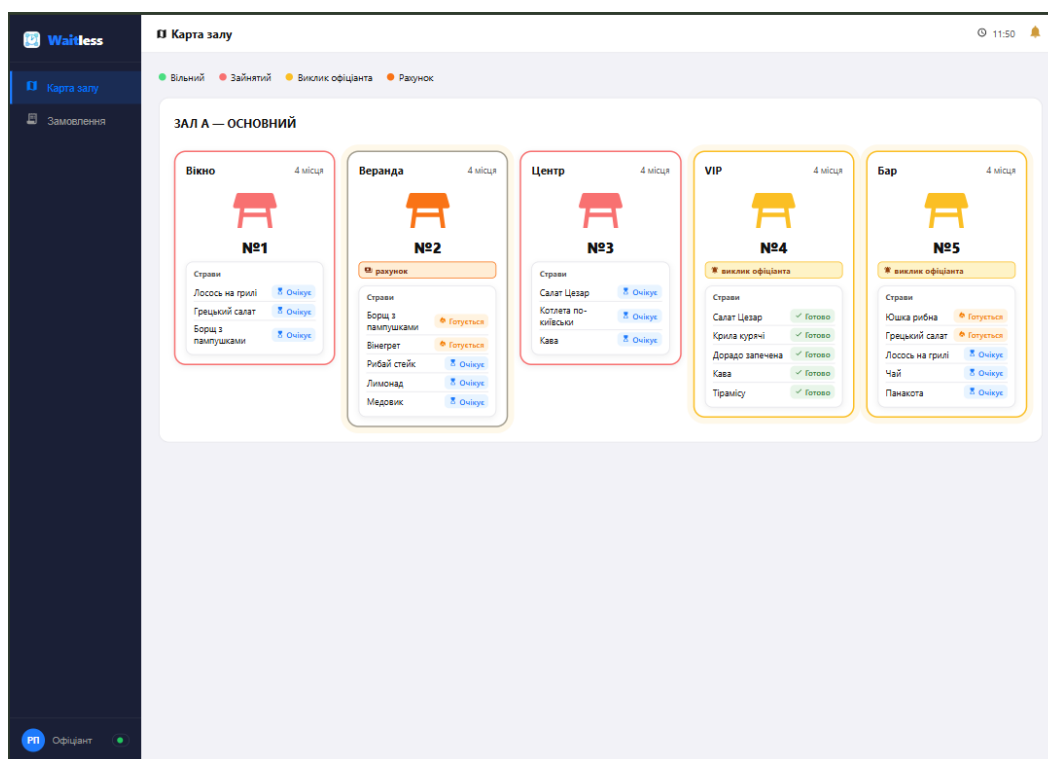


Рисунок 4.4 – Інтерфейс офіціанта: карта залу

4.4.2.2 Деталі столика та управління викликами й оплатою

Перехід до столика відкриває його активне замовлення, згруповане за подачами, із загальною сумою та статусами страв. Тут офіціант може додати позиції, скасувати замовлення, відредагувати чи видалити окрему страву. При виклику від гостя у верхній частині з'являється банер із типом виклику — обслуговування (call) або готівкова оплата (cash_payment) — та зворотним відліком: без відповіді виклик автоматично скасовується через 60 секунд. Підтвердження готівкової оплати переводить замовлення у статус `completed_cash` і надсилає гостю WebSocket-подію `PAYMENT_COMPLETED`. Сторінка також містить QR-код столика, кнопку відкриття вікна відновлення сесії та історію замовлень за день. Екран деталей столика наведено на рисунку 4.5.

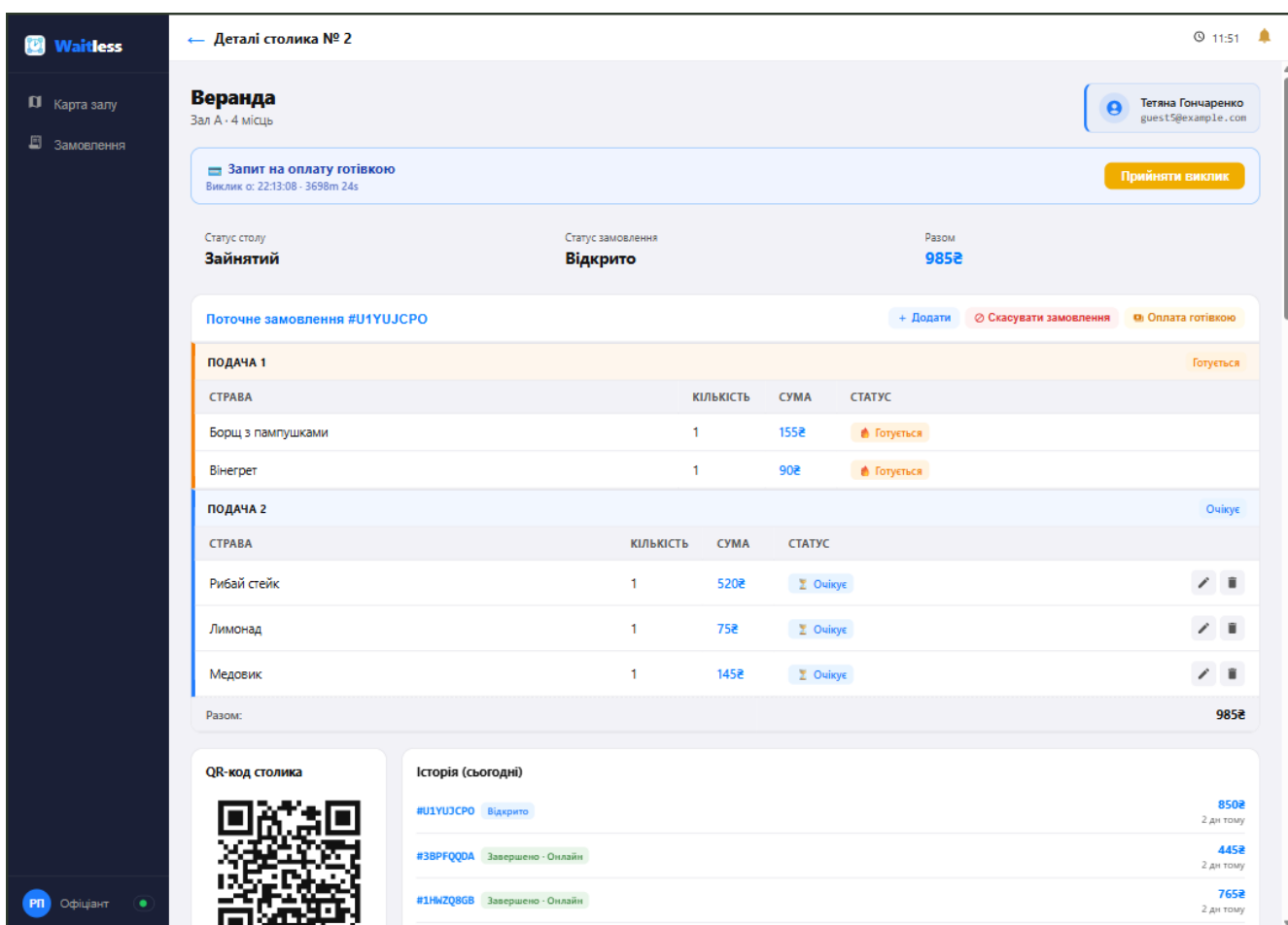


Рисунок 4.5 – Інтерфейс офіціанта: деталі столика, виклики та оплата

4.4.2.3 Черга замовлень

Окремий розділ «Замовлення» подає всі активні замовлення у вигляді карток, згрупованих за столиками, із сумою, переліком страв та зведеним статусом. Картки з активним викликом виділяються та містять кнопку прийняття виклику, а кнопка скасування дозволяє анулювати замовлення. Відкриття картки переводить до детального перегляду замовлення з повним складом кожної страви, обраними модифікаторами й можливістю поетапного редагування. Чергу та деталі замовлення наведено на рисунку 4.6.

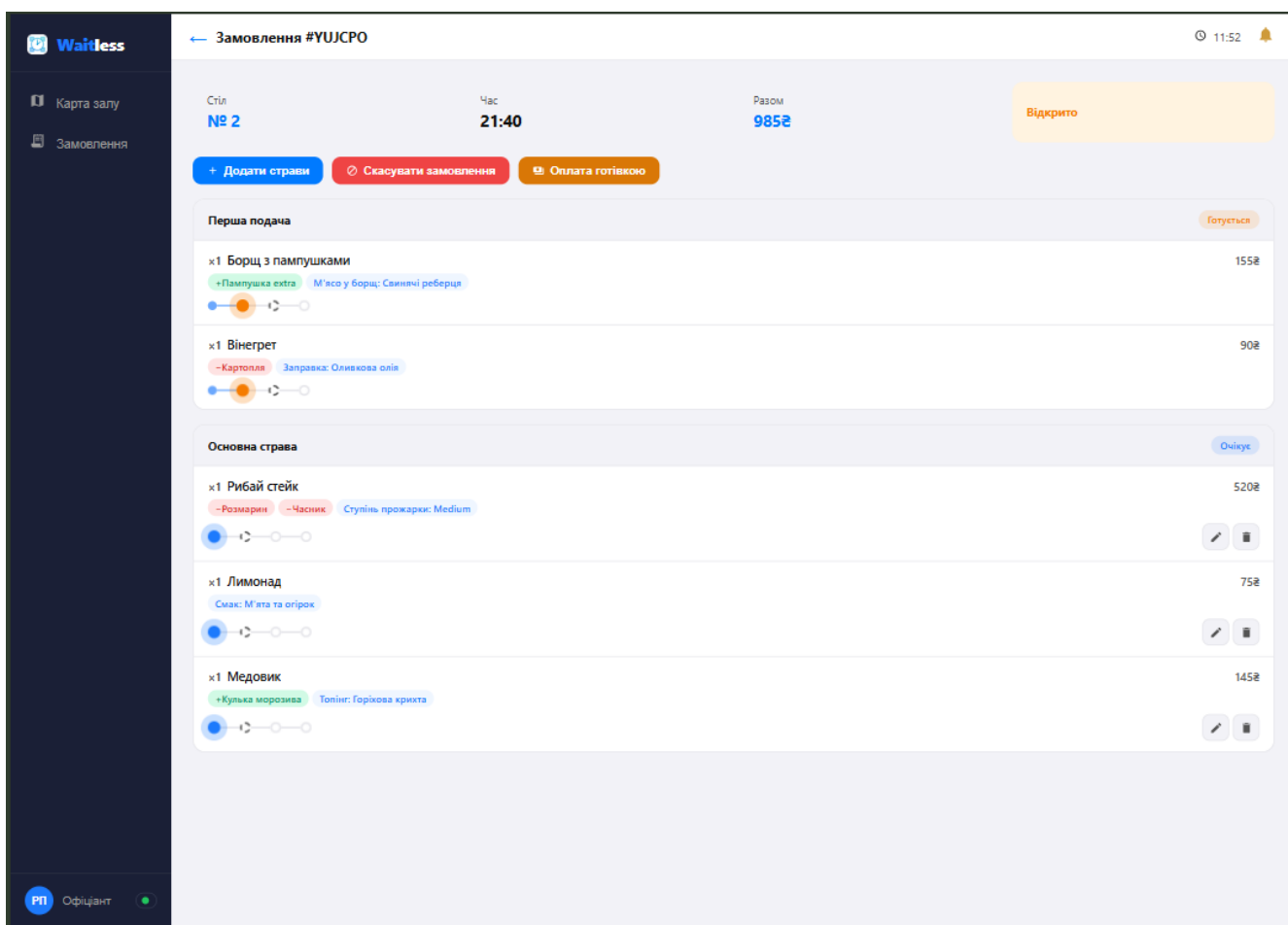


Рисунок 4.6 – Інтерфейс офіціанта: черга та деталі замовлення

4.4.3 Інтерфейс кухаря

4.4.3.1 Активні приготування страв

Головним робочим екраном кухаря є дошка приготувань, на якій усі позиції розподілені за стовпцями «Нові», «Готуються», «Готові» та «Подані» відповідно до їхнього стану. Кожна картка містить назву страви, столик та внесені гостем зміни складу — прибрані інгредієнти й додані доповнення, тож кухар бачить точний склад замовлення без додаткових уточнень. Перехід між станами виконується кнопками «Почати готувати», «Готово» та «Подати», а оновлення надходять миттєво через WebSocket, тому дошка завжди відображає актуальну чергу без перезавантаження. Перемикач «По замовленнях / По столах» змінює спосіб групування: у першому режимі позиції впорядковані за станом приготування, а в другому — за столиками, де кожен блок містить подачі замовлення (наприклад, «Основна подача» та «Перша подача»), що допомагає готувати й видавати страви одного столу разом. Екран активних приготувань наведено на рисунку 4.7.

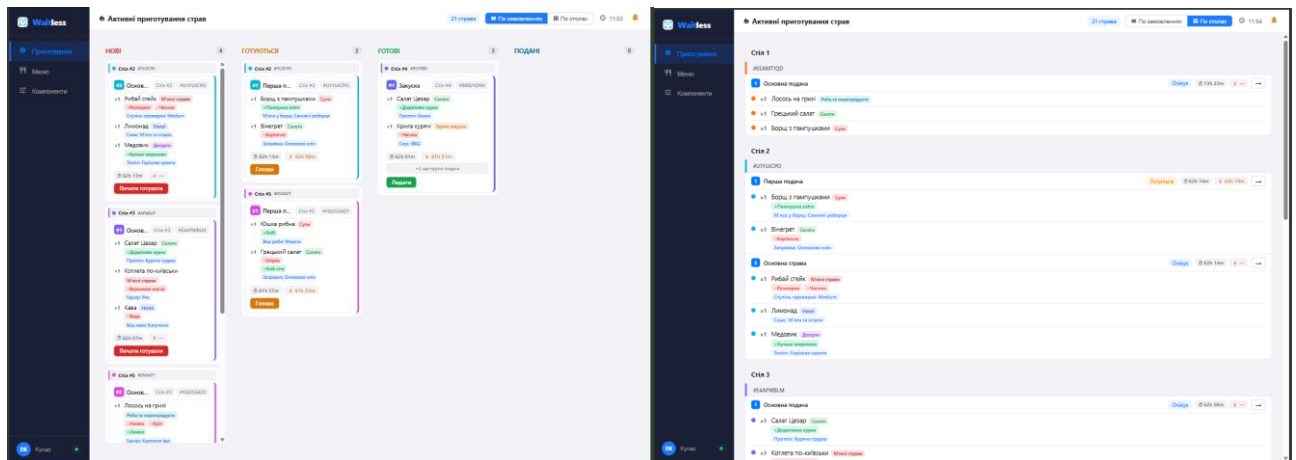


Рисунок 4.7 – Дошка приготування страв (перегляд за замовленнями та за столами)

4.4.3.2 Управління меню та компонентами

Розділ управління меню містить бічну панель категорій і список страв обраної категорії, де для кожної позиції відображається ціна, перемикач доступності та дії

редагування й видалення; перемикач дозволяє швидко прибрати страву з продажу, якщо закінчилися інгредієнти, а кнопки «Додати страву» та «Створити PDF меню» розташовані над списком. Окремий підрозділ «Компоненти» поділений на вкладки «Інгредієнти», «Доповнення» та «Групи компонентів» і слугує спільним довідником, з якого складаються страви: для кожного компонента показано, у яких стравах він використовується, та перемикач доступності, що дозволяє тимчасово вимкнути компонент одразу в усіх пов'язаних позиціях. Екран управління меню та компонентами наведено на рисунку 4.8.

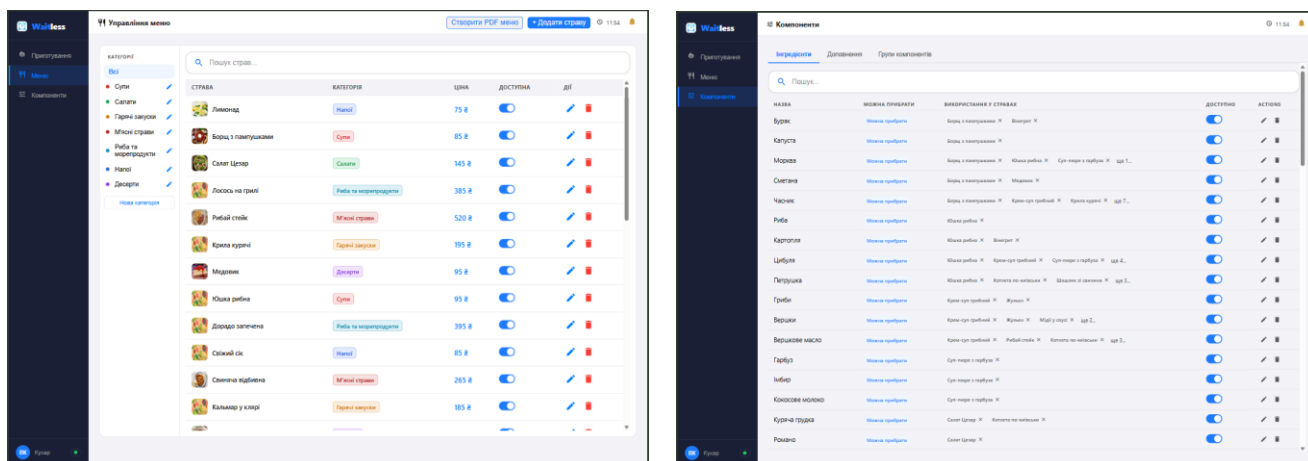


Рисунок 4.8 – Сторінки управління меню та компонентами

4.4.3.3 Редагування страви

Сторінка редагування страви має вкладки мов (українська та англійська), завдяки чому назву, опис та інші текстові поля можна заповнити кожною мовою окремо. Окрім основної інформації, тут задаються інгредієнти страви, причому кожен інгредієнт можна відредагувати окремо та позначити прапорцем «Можна прибрати» для тих, що гість має право виключити, а збоку відображається живий попередній перегляд картки страви у вигляді, який бачитиме клієнт. Екран редагування страви наведено на рисунку 4.9.

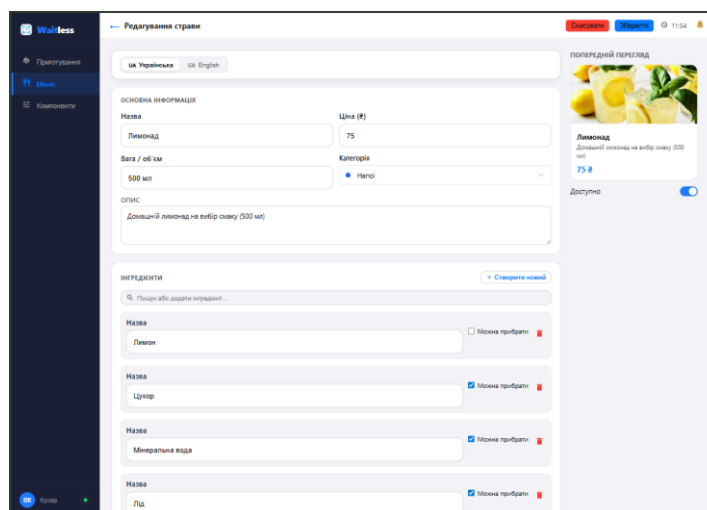


Рисунок 4.9 – Редагування страви

4.4.3.4 Генерація PDF-меню

Кнопка «Створити PDF меню» формує друкований варіант меню ресторану: застосунок збирає актуальні категорії та страви з цінами й засобами бібліотек jsPDF і html2canvas перетворює їх на файл menu.pdf, готовий до друку чи розміщення поза застосунком. Для безкоштовного тарифу функція обмежена й супроводжується вікном пропозиції оновлення плану. Екран генерації PDF-меню наведено на рисунку 4.10.

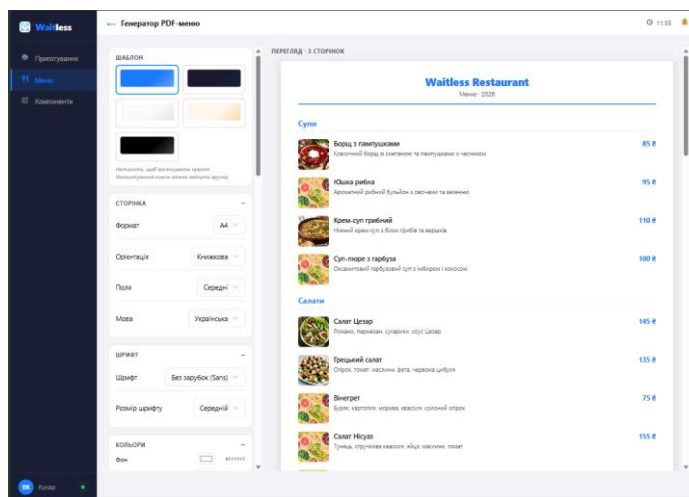


Рисунок 4.10 – Генерація PDF-меню

4.4.4 Інтерфейс адміністратора

Адміністратор має найширший доступ у системі: його інтерфейс повністю включає робочі екрани офіціанта (карта залу, черга замовлень, деталі столика) та кухаря (дошка приготувань, управління меню й компонентами), але з розширеними правами. Зокрема, лише адміністратор може створювати, редагувати та видаляти столики, а під час роботи із замовленнями він обходить обмеження, які діють для звичайного персоналу: редагування й видалення позицій дозволені незалежно від стану страви (без вимоги статусу «очікує»), а змінювати стан страв можна навіть для вже оплачених замовлень. Окрім успадкованих екранів, у бічному меню адміністратора з'являються власні розділи — «Персонал», «Налаштування», «Аналітика» та «Відгуки», доступність частини з яких залежить від тарифного плану ресторану.

Поділ можливостей за моделлю підписки є наскрізним для всього інтерфейсу. На безкоштовному плані діють кількісні обмеження — до 5 категорій, 50 страв, 3 зображень на страву та 3 облікових записів персоналу (адміністратор враховується в цю межу), приймання оплати лише готівкою, а функції аналітики, відгуків, автоматичного перекладу та генерації PDF-меню вимкнені або заблоковані. При досягненні ліміту чи спробі скористатися закритою функцією відкривається вікно UpgradeModal із пропозицією перейти на преміум; у бічному меню преміальні пункти позначені іконкою замка. Преміум-план знімає всі обмеження, вмикає онлайн-оплату через LiqPay, аналітику, відгуки, автопереклад і необмежену кількість страв, категорій та персоналу. Перемикання плану поширюється на інтерфейс миттєво через WebSocket, тож блокування знімаються без перезавантаження сторінки.

4.4.4.1 Управління персоналом

Розділ «Персонал» відображає таблицю співробітників ресторану з ім'ям, поштою, поточною роллю, статусом активності та діями. Роль кожного працівника

змінюється безпосередньо у випадному списку (офіціант, кухар, офіціант і кухар), а кнопки «Деактивувати» та «Скинути пароль» дозволяють тимчасово відключити обліковий запис або надіслати працівникові посилання для встановлення нового пароля; запис власника позначений окремою міткою й захищений від зміни ролі та деактивації. Кнопка «Додати співробітника» створює новий обліковий запис, проте на безкоштовному плані вона блокується після досягнення межі у три акаунти й натомість відкриває вікно пропозиції оновлення. Екран управління персоналом наведено на рисунку 4.11.

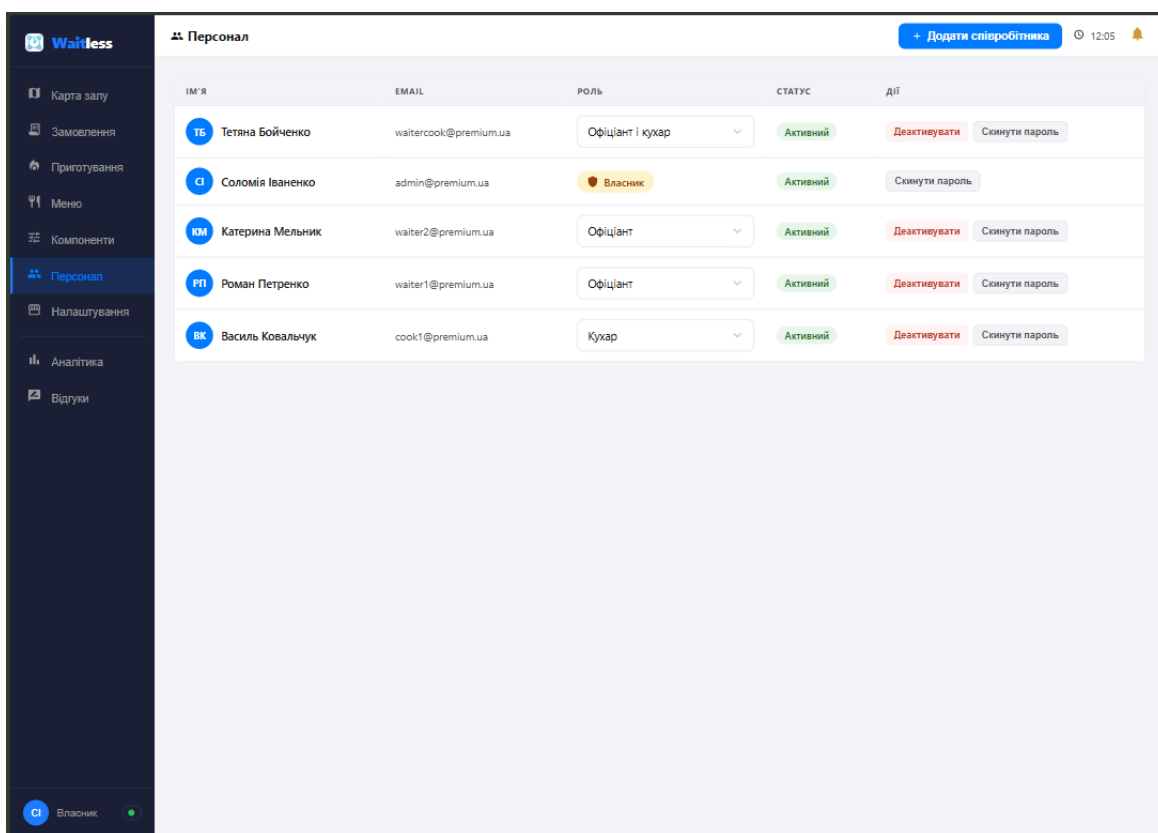


Рисунок 4.11 – Управління персоналом

4.4.4.2 Аналітика та відгуки

Розділи аналітики й відгуків доступні лише на преміум-плані та слугують інструментами оцінювання роботи закладу. Сторінка «Аналітика та звітність» зводить ключові показники — виручку, кількість замовлень, середній чек,

конверсію та середній час приготування — з перемиканням періоду (сьогодні, тиждень, місяць) і можливістю експорту у CSV, а нижче подає графік виручки за днями, рейтинг топ-категорій і таблицю найпопулярніших страв із кількістю замовлень, виручкою та оцінкою. Сторінка «Відгуки» дає змогу переглядати залишені гостями оцінки у двох вкладках — про ресторан загалом та про окремі страви — з фільтром за стравою й можливістю видалити недоречний відгук. Екрани аналітики та відгуків наведено на рисунку 4.12.

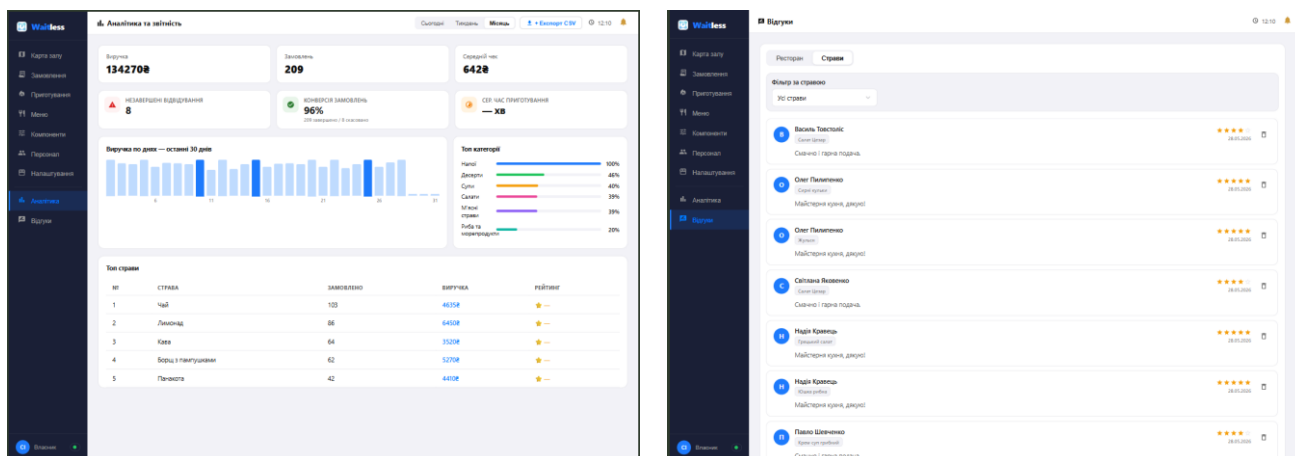


Рисунок 4.12 – Сторінки аналітики та звітності та модерації відгуків

4.4.4.3 Налаштування ресторану

Сторінка налаштувань ресторану зосереджує загальну конфігурацію закладу: основну інформацію (назву, посилання-slug для URL, адресу та кухню) з вкладками мов, завантаження логотипа, вибір основної мови контенту й мов перекладу, а також поля для ключів LiqPay, що зберігаються у зашифрованому вигляді й уможливають приймання онлайн-оплат. Зміни підтверджуються кнопками «Зберегти» та «Скасувати» у шапці сторінки, а функція автоматичного перекладу контенту доступна лише на преміум-плані. Екран налаштувань ресторану наведено на рисунку 4.13.

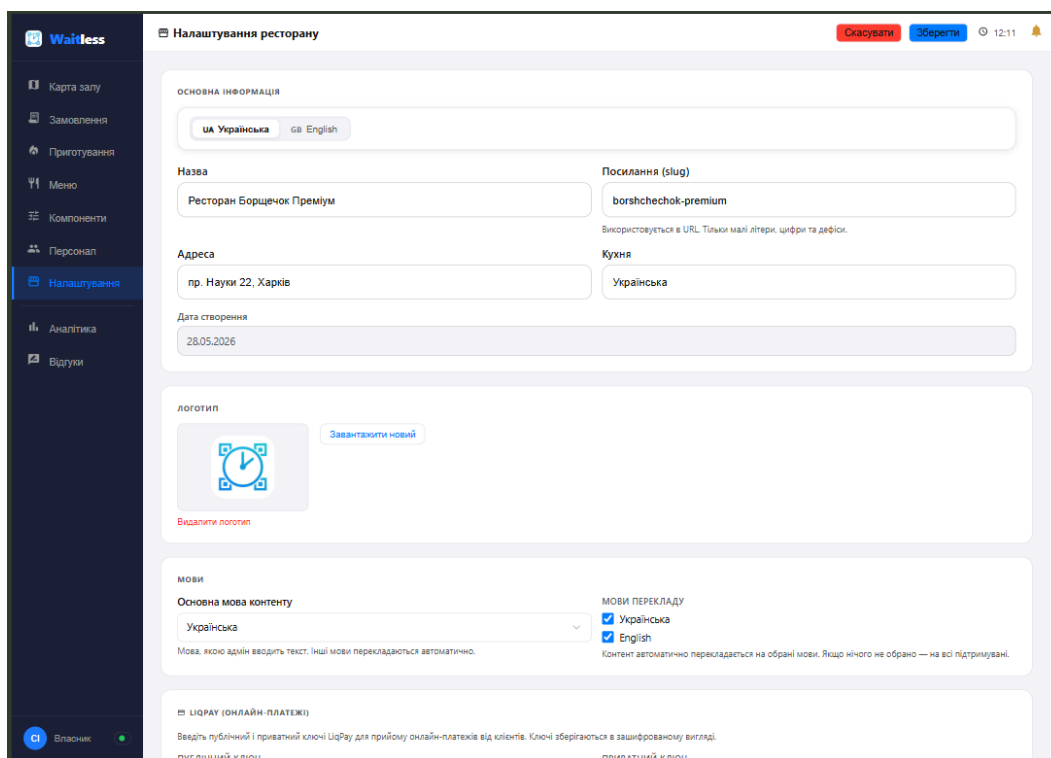


Рисунок 4.13 – Налаштування ресторану

4.4.4.4 Профіль користувача персоналу

Кожен працівник незалежно від ролі має особисту сторінку профілю, побудовану за єдиним зразком. Вона містить особисті дані (ім'я та посаду) з можливістю редагування, налаштування системи — мову інтерфейсу, тему оформлення (світла чи темна) та перемикач звукових сповіщень, блок інтеграцій для підключення облікового запису Google, а також інструменти безпеки: зміну пошти з підтвердженням за посиланням і скидання пароля. У нижньому куті відображається індикатор стану з'єднання в реальному часі, що підтверджує активність WebSocket-підключення. Екран профілю користувача наведено на рисунку 4.14.

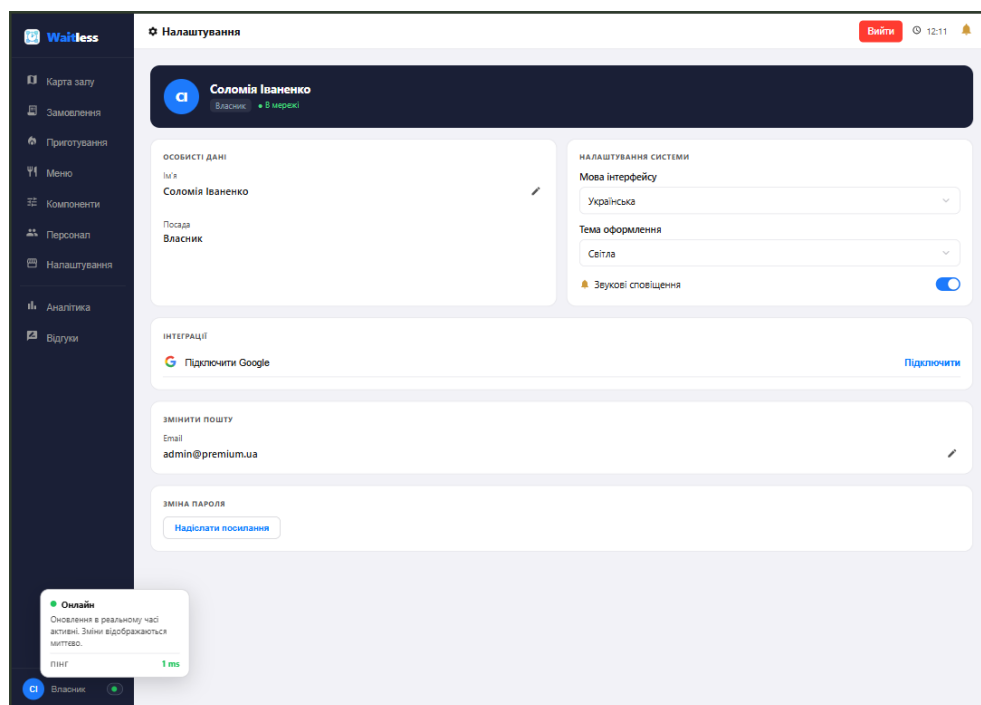


Рисунок 4.14 – Профіль користувача персоналу

5 ТЕСТУВАННЯ

5.1 Стратегія тестування

Стратегія тестування системи побудована на двох взаємодоповнюючих рівнях. Перший рівень — автоматизоване тестування серверної частини, що охоплює REST API-ендпоінти, доменні сервіси та механізм real-time обміну повідомленнями через WebSocket. Другий рівень — ручне функціональне тестування клієнтського веб-інтерфейсу за всіма сценаріями використання, під час якого перевіряється поведінка односторінкового застосунку (SPA) у браузері, коректність переходів між екранами, реакція інтерфейсу на події реального часу та робота в умовах нестабільної мережі.

Кожен перевірюваний випадок формулюється у вигляді критерію приймання (AC) у форматі Given/When/Then, що дозволяє однозначно зіставити вимогу з тестовим сценарієм — визначеними передумовами, дією та очікуваним результатом. Особливу увагу приділено перевірці критичних інваріантів системи: гарантії одного активного замовлення на обліковий запис, однаправленості переходів статусів страв і груп подачі, рольового розмежування доступу (RBAC) з ізоляцією між ресторанами, дотримання тарифних обмежень, незмінності цінових знімків (price snapshot) в оформлених замовленнях, цілісності та незмінності журналу аудиту фінансових операцій, а також безпечного зберігання паролів. Для функціоналу реального часу окремо перевірялися сценарії автентифікації з'єднання, маршрутизації подій за кімнатами (rooms), а також розриву й відновлення WebSocket-з'єднання з відтворенням пропущених подій (event replay).

5.2 Інструменти та середовище тестування

Автоматизоване тестування серверної частини виконується засобами фреймворку Jest із бібліотекою supertest для надсилання HTTP-запитів до застосунку та перевірки відповідей. Запуск здійснюється командою npm test (режим

--runInBand гарантує послідовне виконання наборів задля детермінованості). Загалом реалізовано 17 наборів тестів (205 тестових випадків), що покривають автентифікацію, сканування QR, замовлення, кухню, готівкові й онлайн-платежі, рольовий доступ, управління столиками, персоналом і меню, переклади, реєстрацію ресторану, Google OAuth, надсилання листів, завантаження зображень у S3 та WebSocket.

Для інтеграційних тестів застосовується бібліотека `mongodb-memory-server`, яка піднімає ізольований екземпляр MongoDB у пам'яті на час прогону. Такий підхід забезпечує повну ізоляцію від робочої бази, детермінованість результатів і високу швидкість виконання, оскільки після завершення тестів дані видаляються разом із зупинкою тимчасового екземпляра. Перед кожним тестом база наповнюється контрольованим набором даних, що відтворює передумови (Given) критерію приймання. Перевірка відповідей охоплює HTTP-статус-код та структуру тіла (поля `data`, `meta`, `error.code`), завдяки чому критерії, що оперують конкретними кодами, перевіряються безпосередньо — зокрема повернення HTTP 403 при недостатньому рівні доступу, HTTP 400 при некоректному переході статусу та HTTP 405 при спробі модифікувати журнал аудиту.

Ручне тестування клієнтського інтерфейсу проводилося у браузері на повному стеку (SPA + сервер + база), наповнена тестовим скриптом, з паралельним відкриттям сесій гостя й персоналу для перевірки *real-time* взаємодії, а також з імітацією втрати мережі для перевірки офлайн-режиму.

5.3 Тестові дані

Для відтворюваного тестування реалізовано скрипт наповнення бази даних `db-seed.js` (версія 6), що запускається командою `npm run db:seed`. Скрипт створює два ресторани з різними тарифами — безкоштовний (публічний ідентифікатор `BR5CH3OK`) та преміум (публічний ідентифікатор `PR3MIUM1`), — що дозволяє перевіряти тарифно-залежну поведінку системи. Для безкоштовного ресторану

створюється три облікові записи персоналу (адміністратор, кухар, офіціант) і вимкнено систему відгуків; для преміум-ресторану — п'ять акаунтів (включно з комбінованою роллю waiter_cook) та активовано відгуки.

Кожен ресторан наповнюється реалістичними даними: 5 столиків, 7 категорій і 32 страви з українськими та англійськими перекладами, 50 замовлень (44 завершених, 2 скасованих, 4 активних) із групами подачі та відповідними платежами, а також 3 виклики офіціанта різних типів. Додатково створюється 35 гостьових акаунтів. Пароль для всіх облікових записів — 12345678. Активні замовлення розподілені між непересічними групами гостей, що гарантує дотримання інваріанта одного активного замовлення на обліковий запис. Незалежно від скрипта, окремий набір передумов формується безпосередньо в інтеграційних тестах за допомогою фабричних хелперів, які створюють потрібні документи (ресторан, столик, користувач, замовлення) перед кожним тестом.

5.4 Автоматизоване тестування серверної частини

У таблиці 5.1 наведено репрезентативну вибірку критеріїв приймання, перевірених автоматизованими тестами бекенду. Для кожного критерію зазначено сценарій, очікуваний результат і статус.

Таблиця 5.1 — Результати тестування серверної частини за критеріями приймання

Ідентифікатор	Сценарій	Очікуваний результат	Статус
AC-QR-01	Сканування QR-коду вільного столика	HTTP 302 на /menu з новим sessionToken за ≤ 500 мс	Пройдено
AC-ORDER-03	Одночасне створення замовлення за одним столиком	Другий запит $\rightarrow$ HTTP 409; створено лише одне замовлення	Пройдено
AC-CART-01	Підтвердження кошика з 3 стравами	Замовлення з'являється на KDS за ≤ 300 мс	Пройдено
AC-KDS-01	Спроба зміни статусу групи з «waiting» на «cooking»	HTTP 200; статус страви оновлюється	Пройдено
AC-KDS-05	Спроба зниження статусу групи з «ready» на «cooking»	HTTP 400; статус не змінюється	Пройдено
AC-PAY-01	Надходження webhook підтвердження від LiqPay	Order.status $\rightarrow$ completed_pay; підтвердження за ≤ 3 с	Пройдено

Продовження таблиці 5.1

Ідентифікатор	Сценарій	Очікуваний результат	Статус
AC-WS-01	REPLAY_REQUEST після розриву з'єднання	Повторно надсилаються лише події після last_event_id	Пройдено
AC-AUTH-01	Реєстрація нового користувача	Пароль у БД — bcrypt-хеш; акаунт активний	Пройдено
AC-PWA-03	Підтвердження замовлення в офлайн-режимі	Замовлення відправляється автоматично після відновлення мережі	Пройдено
AC-REV-02	Повторне залишення відгуку про ресторан	HTTP 409	Пройдено

5.5 Ручне функціональне тестування клієнтського інтерфейсу

Сценарії, що залежать від поведінки браузерного інтерфейсу, рендерингу в реальному часі та роботи без мережі, перевірялися вручну на повному стеку. Сюди належать і кілька системних інваріантів, реалізованих на рівні сервісів, але спостережуваних саме через клієнтський інтерфейс. Основні результати наведено в таблиці 5.2.

Таблиця 5.2 — Результати ручного тестування клієнтських сценаріїв використання

Ідентифікатор	Сценарій	Очікуваний результат	Статус
UC-QR-01	Сканування дійсного QR-коду гостем	SPA зберігає сесію в localStorage і перенаправляє на сторінку меню	Пройдено
UC-ORDER-03	Спроба оформити друге активне замовлення з того ж акаунту	Сервер повертає HTTP 409 (ACTIVE_ORDER_EXISTS); інтерфейс показує банер із посиланням на наявне замовлення	Пройдено
UC-CART-01	Підтвердження кошика з кількома стравами	Замовлення майже миттєво з'являється на дошці кухаря через WebSocket	Пройдено
UC-STATUS-01	Кухар переводить групу подачі у наступний стан	Сторінка статусу замовлення в гостя оновлюється миттєво без перезавантаження	Пройдено
UC-CALL-01	Спроба зниження статусу групи з «ready» на «cooking»	HTTP 400; статус не змінюється	Пройдено
UC-TABLE-01	Надходження webhook підтвердження від LiqPay	Order.status → completed_pay; підтвердження за ≤ 3 с	Пройдено

Продовження таблиці 5.2

Ідентифікатор	Сценарій	Очікуваний результат	Статус
UC-PWA-01	Меню було завантажено але зник зв'язок	Клієнт має можливість переглядати меню, деталі кожної страви, редагувати кошик	Пройдено
UC-PWA-04	Підтвердження замовлення без мережі	Замовлення ставиться в чергу в localStorage і автоматично надсилається після відновлення з'єднання	Пройдено
UC-PLAN-02	Досягнення лімітів безкоштовного плану (5 категорій, 50 страв, 3 акаунти)	Відкривається вікно UpgradeModal; преміальні пункти меню (аналітика, відгуки) заблоковано іконкою замка	Пройдено
UC-REVIEW-01	Залишення відгуку на різних тарифах	На преміум-плані відгук додається; на безкоштовному функція відсутня	Пройдено
UC-I18N-01	Перемикання мови інтерфейсу UA/EN	Назви страв і контент оновлюються без повторного завантаження даних	Пройдено
UC-THEME-01	Перемикання теми інтерфейсу Dark/Light	Кольорова палітра інтерфейсу змінюється без перезавантаження	Пройдено

5.6 Результати тестування

За результатами тестування підтверджено відповідність реалізованої системи сформульованим критеріям приймання. Усі 278 автоматизованих тестів серверної частини завершуються успішно й покривають автентифікацію, життєвий цикл замовлень, кухонні переходи статусів, готівкові та онлайн-платежі, рольове розмежування доступу з ізоляцією між ресторанами, незмінність журналу аудиту, переклади та повний набір подій WebPocket. Ручна перевірка клієнтських сценаріїв підтвердила коректність наскрізних потоків і критичних інваріантів, які спостерігаються через інтерфейс: гарантії одного активного замовлення на обліковий запис, незмінності цінових знімків, безпечного блокування зайнятих столиків, а також працездатності механізмів реального часу й офлайн-режиму, що є ключовими конкурентними перевагами системи.

Окремо підтверджено стійкість системи до некоректних і зловмисних дій: відхилення платіжних webhook-ів із недійсним підписом без зміни стану замовлення, ідемпотентну обробку повторних сповіщень платіжної системи, блокування доступу до чужих ресторанів і адміністративних функцій, а також безпечне зберігання паролів у вигляді bcrypt-хешів. Виявлені під час тестування невідповідності — зокрема початково неточне трактування відповіді ендпоінта сканування QR як серверного перенаправлення (насправді сервер повертає HTTP 200 з даними сесії, а перехід виконує клієнтський застосунок) — було усунено, після чого повторне тестування підтвердило очікувану поведінку.

6 ВПРОВАДЖЕННЯ

6.1 Середовище розгортання

Система розгортається за моделлю multi-tenant SaaS: одна інстанція обслуговує багатьох клієнтів-ресторанів з ізоляцією даних на рівні префіксу restaurantId. Архітектура розгортання відповідає трирівневій структурі системи та складається з трьох логічно незалежних компонентів: статичної клієнтської частини, серверного застосунку Node.js та бази даних MongoDB. Такий поділ дозволяє масштабувати кожен компонент незалежно та забезпечує гнучкість при виборі хостинг-провайдера.

Клієнтська частина після збірки є набором статичних артефактів (HTML, JavaScript, CSS), які не потребують серверного виконання і можуть віддаватися через мережу доставки контенту (CDN) або статичний хостинг. Серверна частина розгортається як довготривалий процес Node.js, що обслуговує REST API та WebSocket-з'єднання. Файлові ресурси (зображення страв) зберігаються у зовнішньому об'єктному сховищі AWS S3, що знімає навантаження зберігання з серверного застосунку.

6.2 Конфігурація через змінні оточення

Конфігурація системи винесена у змінні оточення (environment variables), що відповідає принципу відокремлення конфігурації від коду та дозволяє розгортати ту саму збірку в різних середовищах (розробка, тестування, продакшен) без модифікації коду. Клієнтська частина використовує змінні VITE_API_URL (базова адреса REST API) та VITE_WS_URL (адреса WebSocket-сервера); за відсутності останньої адреса WebSocket виводиться автоматично з адреси API.

Серверна частина споживає змінні оточення для підключення до бази даних (рядок з'єднання MongoDB), секретних ключів JWT, облікових даних інтеграцій (LiqPay secret та public key, Google OAuth client ID та secret, AWS S3 credentials, API-

ключ Resend та Google Translate). Секретні значення ніколи не потрапляють до системи контролю версій і зберігаються у захищеному сховищі секретів середовища розгортання.

6.3 Збірка та розгортання

Збірка клієнтської частини виконується інструментом Vite командою `npm run build`, що формує оптимізований набір статичних файлів з мінімізацією, `tree-shaking` та `code-splitting`. Отримані артефакти розгортаються на статичному хостингу або CDN. Локальна розробка ведеться через команду `npm start`, що запускає dev-сервер Vite з миттєвим оновленням модулів (Hot Module Replacement).

Серверний застосунок запускається як довготривалий процес Node.js, що підключається до екземпляра MongoDB та зовнішніх сервісів (об'єктне сховище AWS S3, платіжний шлюз LiqPay, Google OAuth, сервіси надсилання листів і перекладу). Перед першим запуском у тестовому середовищі база даних наповнюється скриптом `db-seed.js`. Послідовність розгортання передбачає: налаштування змінних оточення, розгортання бази даних MongoDB, налаштування bucket-ів AWS S3 для зберігання зображень, запуск серверного застосунку Node.js та публікацію статичної клієнтської збірки на CDN.

6.4 Моніторинг та логування

Для забезпечення спостережуваності системи в робочому середовищі реалізовано health-check ендпоінт, що перевіряє стан критичних залежностей (база даних, платіжний шлюз, OAuth-сервіс) і використовується системою оркестрації для контролю працездатності застосунку. Структуроване логування реалізоване на основі бібліотеки `winston`: усі HTTP-запити та WebSocket-події записуються у форматі JSON з обов'язковими полями `timestamp`, `request_id` (для наскрізного трасування запиту крізь усі шари), типом події та статус-кодом.

Фінансові операції додатково фіксуються в окремому незмінному журналі (audit log) з мінімальним терміном зберігання 3 роки. Особисті дані (паролі, токени, платіжні реквізити) виключаються з логів на рівні форматувальника. Метрики системи (кількість активних з'єднань, час відповіді, частота помилок) збираються в реальному часі, а при перевищенні порогових значень формуються автоматичні алерти, що дозволяє оперативно реагувати на деградацію сервісу.

ВИСНОВКИ

У ході виконання кваліфікаційної роботи спроектовано та реалізовано програмну систему для безконтактного замовлення їжі в ресторані з використанням QR-технологій. Створена система забезпечує повний цикл обслуговування гостя від моменту сканування QR-коду на столику до завершення оплати та залишення відгуку, замінюючи традиційну модель «гість — офіціант — кухня — офіціант — гість».

У межах роботи вирішено наступні завдання.

- Виконано детальний аналіз предметної галузі та порівняльний аналіз чотирьох існуючих програмних рішень (Poster POS, OddMenu, Choice QR, Kavapp QR). За результатами аналізу виявлено ринкову прогалину: існуючі рішення або є фінансово недоступними для закладів малого формату через надлишковий функціонал, або обмежуються виключно функцією відображення меню без можливості повноцінного замовлення гостем.

- Сформовано повний комплект функціональних та нефункціональних вимог до системи. Сформульовано понад 80 функціональних вимог з унікальними ідентифікаторами, розподілених на 15 модулів, та понад 70 критеріїв приймання у форматі Given/When/Then. Результати оформлено у вигляді документа Software Requirements Specification (SRS).

- Спроектовано трирівневу клієнт-серверну архітектуру системи з використанням обґрунтовано підібраного технологічного стеку (ReactJS, Node.js, MongoDB). Розроблено схему бази даних із 23 колекціями та стратегією індексування для оптимізації типових запитів.

- Реалізовано клієнтську частину у вигляді односторінкового застосунку з підтримкою Progressive Web App, що забезпечує часткову роботу в офлайн-режимі та зберігає кошик гостя при втраті з'єднання. Інтерфейс адаптований під три типи

користувачів — гостя, персонал та адміністратора — з умовним рендерингом залежно від ролі.

- Реалізовано серверну частину з шаровою архітектурою, що включає понад 60 REST API-ендпоінтів та повноцінну систему передачі подій у реальному часі через WebSocket з моделлю кімнат та механізмом event replay. Розроблено систему автентифікації з підтримкою email+пароль та Google OAuth 2.0 для всіх типів користувачів.

- Реалізовано розширені модулі системи: аналітичний дашборд, генератор PDF-меню, систему відгуків, інтеграцію з платіжним шлюзом LiqPay для онлайн-оплати та повноцінну систему виклику офіціанта з підтримкою двох типів викликів.

- Реалізовано модель монетизації Freemium із технічним розподілом функціоналу на рівні бази даних та middleware: безкоштовний тариф включає базовий функціонал прийому та обслуговування замовлень з обмеженнями на обсяг меню та кількість акаунтів персоналу; розширений тариф знімає обмеження та активує модулі онлайн-оплати, аналітики, генератора PDF, системи відгуків та автоматизованого перекладу через Google Translate.

Усі поставлені у вступі завдання виконано в повному обсязі. Створений програмний продукт [11] є придатним для реального впровадження в закладах громадського харчування малого та середнього формату. Модульна архітектура та чітко документована специфікація вимог забезпечують можливість подальшого розширення функціоналу без перепроєктування системи.

Перспективи подальшого розвитку. Архітектура системи спроектована з урахуванням можливості подальшого розширення функціональності. Перспективними напрямками розвитку є розширення мовної підтримки (завдяки використанню i18next додавання нової мови потребує лише перекладу ресурсних файлів без модифікації коду), розширення тем інтерфейсу через систему CSS-змінних, а також реалізація функції розділення рахунку (Split Bill), інтеграція системи бронювання столиків та програми лояльності.

Окремими перспективними напрямками є поглиблення аналітичного блоку (погодинна аналітика навантаження, прогнозування попиту, АВС-аналіз страв за прибутковістю), інтеграція зі штучним інтелектом (автоматична генерація описів страв, персоналізовані рекомендації, аналіз тональності відгуків), двостороння інтеграція з популярними POS-системами (Poster, Syrve, Servio) та розробка нативних мобільних застосунків для персоналу з push-сповіщеннями. Реалізація запропонованих напрямів забезпечить трансформацію системи з вузькоспеціалізованого SaaS-рішення на повноцінну цифрову платформу для управління закладами громадського харчування.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАНЬ

1. Звягінцева О. Проблеми ресторанного бізнесу в Україні у 2025 році та рішення для них. URL: <https://hub.kyivstar.ua/articles/problemi-restorannogo-biznesu-v-ukrayini-u-2025-rocz-i-ta-rishennya-dlya-nih> (дата звернення: 16.04.2026).
2. Островська Г., Шерстюк Р., Летун О. Аналіз ринкових тенденцій інтелектуальної автоматизації ресторанного бізнесу. URL: [https://doi.org/10.32515/2663-1636.2023.10\(43\).143-155](https://doi.org/10.32515/2663-1636.2023.10(43).143-155) (дата звернення: 16.04.2026).
3. Силівейстр В. Топ 13 трендів у ресторанному бізнесі у 2026 році. URL: <https://joinposter.com/ua/post/restoranni-trendy> (дата звернення: 17.04.2026).
4. Турчиняк М., Примаєв А. Вплив пандемії covid-19 на готельно-ресторанну індустрію України. URL: <https://doi.org/10.32782/2524-0072/2023-47-29> (дата звернення: 17.04.2026).
5. Poster POS: офіційний сайт системи автоматизації. URL: <https://joinposter.com> (дата звернення: 18.04.2026).
6. OddMenu: цифрове QR-меню для ресторанів. URL: <https://oddmenu.com> (дата звернення: 18.04.2026).
7. Choice QR: система безконтактного замовлення та оплати. URL: <https://choiceqr.com> (дата звернення: 18.04.2026).
8. Каварр: система управління кав'ярнями та ресторанами. URL: <https://kavapp.com> (дата звернення: 18.04.2026).
9. Banks A., Porcello E. Learning React : Functional Web Development with React and Redux. Sebastopol : O'Reilly Media, 2017. 350 p.
10. Banker K., Bakum P., Verch S., Garrett D., Hawkins T. MongoDB in Action. 2nd ed. Shelter Island : Manning Publications, 2016. 480 p.
11. Github сторінка. URL: https://github.com/NureTokarDenys/2026_B_PI_PZPI-22-1_Tokar_D_Y (дата звернення: 10.07.2026).